

Pontificia Universidad Javeriana Facultad de Ingeniería
Departamento de Ingeniería de Sistemas

Estructuras de Datos

Proyecto Risk

Documento de diseño y plan de pruebas – Entrega 2

Componente 1: configuración del juego (estructuras lineales) y Componente 2: almacenamiento de partidas (árbol de Huffman)

Ivan David Wright Gaviria
Juan Sebastián Martínez Gómez

Estructuras de Datos

Bogotá D.C.

22 de Octubre de 2026

Repositorio de control de versiones
https://github.com/IvanDWright/Proyecto_Estructuras

Declaración de uso de IA generativa

Esta declaración responde al numeral 3.5 del enunciado (Pontificia Universidad Javeriana, 2026). En la Entrega 0 usamos Antigravity para generar plantillas de texto, organizar la presentación, definir los flujos de usuario y refinar el código base en C++. En la Entrega 2 usamos Claude Code (Anthropic, 2026) para comparar el diseño con el código, escribir y ejecutar los guiones que generan la evidencia de las pruebas (el estado y la salida de cada caso salen de ejecutar el programa compilado), generar los esquemáticos y redactar y maquetar este documento a partir de las correcciones de la Entrega 1. En la versión final también usamos Claude para depurar la redacción, ajustar el formato a APA 7, auditar el proyecto contra el enunciado y corregir el código. Revisamos cada cambio antes de incluirlo.

La Tabla 1 indica qué partes se hicieron con apoyo de IA y con qué prompt; los prompts están transcritos de forma literal en el Apéndice A. En el código, cada línea o bloque que Claude generó o modificó en la versión final lleva un comentario que empieza con [IA] y cita el prompt (Prompts 12 a 15).

Tabla 1

Partes de la entrega hechas con apoyo de IA generativa

Parte	Apoyo de la IA	Prompts
Acta (sección 1)	Los comentarios literales y las notas son de la retroalimentación de la Entrega 1. La redacción de la columna de corrección la hizo la IA a partir de los cambios en el código.	9, 12 y 13
Diseño (secciones 2 a 4)	Borrador generado por la IA a partir de nuestro código; luego lo revisamos y la IA lo depuró.	9, 10, 11 y 13
Huffman (sección 5)	Texto generado por la IA; las cifras (frecuencias, códigos y tamaños) salen de ejecutar el programa.	9 y 12
Pruebas (sección 6)	Planes, guiones de ejecución y tablas generados por la IA; los resultados salen de ejecutar el programa compilado.	9, 12 a 15
Secciones 7 y 8	Generadas por la IA a partir de la auditoría del código y del enunciado.	12 a 15
Esquemáticos	Dibujados con guiones de Graphviz que escribió la IA a partir del código.	9, 11 y 12 a 15
Código: estructura base y ayuda	Generados con Antigravity en la Entrega 0 y modificados por nosotros.	4, 5 y 6
Código: TADs, Huffman y traslado de la lógica a los TADs	Generados o corregidos con IA en las entregas 1 y 2.	7 y 8
Código: correcciones de la versión final	Generadas por Claude; marcadas con [IA] en el código.	12 a 15

Contenido

Declaración de uso de IA generativa..... 2
1. Acta de evaluación de la Entrega 1..... 5
2. Descripción general del sistema..... 7
    2.1 Archivos fuente..... 8
3. Diseño: procedimiento principal y operaciones..... 8
    3.1 Procedimiento principal..... 8
    3.2 Comandos (operaciones auxiliares)..... 12
    3.3 Operaciones auxiliares del procedimiento principal..... 33
    3.4 Diseño de las operaciones principales de los TADs..... 33
4. Diseño de los TADs..... 38
    4.1 TADs de estructuras lineales..... 39
    4.2 TADs del dominio del juego (Componente 1)..... 42
    4.3 TADs de estructuras jerárquicas (Componente 2)..... 50
5. Diagrama de relación entre TADs y esquemáticos de Huffman..... 54
    5.1 Árbol de Huffman de una partida real..... 56
    5.2 Esquemático del archivo binario (.bin)..... 58
6. Plan de pruebas..... 60
    6.1 Metodología..... 60
    6.2 Plan de pruebas del comando atacar..... 61
    6.3 Plan de pruebas del comando guardar_comprimido..... 64
    6.4 Casos de consola T01 a T11 y pruebas unitarias..... 66
    6.5 Pruebas de robustez y simulación de partidas..... 69
7. Observaciones, decisiones de diseño y limitaciones..... 71
8. Compilación y contenido de la entrega..... 75
Referencias..... 77
Apéndice A. Registro de prompts utilizados..... 78
Apéndice B. Salidas de consola de los casos de prueba..... 83
Apéndice C. Capturas de pantalla de la ejecución..... 93
Apéndice D. Archivo de inicio usado en las pruebas..... 103

1. Acta de evaluación de la Entrega 1

El numeral 3.3 del enunciado pide un acta con los comentarios recibidos en la Entrega 1. En la Tabla 2 copiamos cada comentario tal como apareció en la retroalimentación, con su puntaje, y explicamos qué hicimos para corregirlo. Aparte de esas correcciones, en esta entrega terminamos lo que faltaba: el componente 2 (guardar, guardar_comprimido e inicializar desde archivos .bin) y el canje de cartas dentro de obtener_unidades. Los comandos costo_conquista y conquista_mas_barata son de la Entrega 3; por ahora solo validan sus parámetros y el estado del juego.

Tabla 2

Comentarios de la Entrega 1 y su corrección

Componente (nota)	Comentario literal	Cómo se corrigió o completó	Evidencia
Diseño (2,5)	“no sigue la plantilla de TADs (solo ListaLineal)”	Los 12 TADs se describen con la plantilla de clase: datos mínimos (nombre, tipo en lenguaje natural y qué representa) y operaciones (entradas, salida y descripción), sin sintaxis de C++. Todas las operaciones listadas existen en el código. Se agregó el diagrama de relación entre TADs.	Sección 4; Figura 11
Diseño (2,5)	“faltan entradas, salidas y condiciones de los comandos”	El procedimiento principal y cada comando tienen una ficha con uso, entradas, condiciones en el orden en que se evalúan, salidas, efecto y TADs que hacen el trabajo, además de su esquemático (diagrama de flujo).	Sección 3; Figuras 1 a 10

Componente (nota)	Comentario literal	Cómo se corrigió o completó	Evidencia
Implementación (3)	“la lógica está en main y no en los TADs”	Las reglas pasaron a los TADs. JuegoRisk: calcularUnidadesBase, obtenerBonoSiguienteCanje, canjearCartas, lanzarDado, resolverRondaCombate, conquistarTerritorio, otorgarCartaPorConquista, moverTropas, unidadesInicialesPorJugador, cargarDesdeTexto, inicializarDesdeArchivo, serializarEstado, guardarEnTexto y guardarEnComprimido. TableroRisk: territoriosDeJugador, territoriosAptosParaAtacar y territoriosAptosParaFortificar. Territorio: tieneVecinoEnemigo y tieneVecinoPropio. Jugador: buscarGrupoCanjeable, eliminarCartas y registrarAtaque. En la versión final también pasaron a JuegoRisk la ubicación de refuerzos, el manejo de las tropas atacantes y el cierre de la fase de ataque (ubicarUnidades, apartarTropasAtacantes, devolverTropasAtacantes, actualizarDefensor y terminarFaseAtaque). Ahora main solo lee datos, decide el orden de las validaciones y muestra los mensajes; ya no modifica directamente tropas, fases ni banderas.	Secciones 3.2 a 3.4 y 4; PU-01 a PU-72
Implementación (3)	“nunca se verifica la victoria”	conquistarTerritorio llama a verificarGanadorGlobal después de cada conquista. Si el jugador ocupa los 42 territorios, el juego pasa a TERMINADO, se registra nombreGanador y atacar anuncia la victoria; los comandos siguientes responden “(Juego terminado)”.	Secciones 3.2 y 3.4; PU-08 a PU-10
Implementación (3)	“los dados no cambian (sin srand)”	main llama a srand(time(NULL)) una sola vez al iniciar la aplicación y lanzarDado usa rand() % 6 + 1, así que los dados cambian en cada ejecución.	Sección 3.1; PU-01 a PU-03; A-13 y A-14

Implementación (3)	“la validación del archivo es incompleta”	cargarDesdeTexto valida N entre 3 y 6; nombres de máximo 8 caracteres sin repetir; colores sin repetir; 42 líneas de territorio con código existente y no repetido, color de un jugador de la partida y al menos 1 tropa; que no haya datos sobrantes; que cada jugador ocupe al menos un territorio, y que la suma de unidades de cada jugador sea la inicial del enunciado (35, 30, 25 o 20 para 3, 4, 5 o 6 jugadores). En los .bin se valida también la cabecera ($1 \leq n \leq 256$, sin caracteres repetidos, $fi > 0$, $w > 0$ y $\Sigma fi = w$) y que el texto decodificado tenga w caracteres; como un .bin guarda una partida en curso, a ese archivo no se le exigen la suma inicial ni que todos los jugadores sigan en juego (sección 7).	Secciones 3.2 y 7; T02 a T05, T10, PU-24 a PU-29, PU-45 a PU-49, PU-54 a PU-61 y G-10 a G-15

Componente (nota)	Comentario literal	Cómo se corrigió o completó	Evidencia
Pruebas (2,5)	“8 casos sin resultado obtenido ni evidencia”	Cada caso tiene entrada, resultado esperado, resultado obtenido y estado, obtenidos al ejecutar el programa compilado con g++. Se presentan los planes de atacar (18 casos, exigido en la Entrega 1) y de guardar_comprimido (16 casos, exigido en la Entrega 2), los casos de consola T01 a T11 (T09 a T11 son nuevos), 72 pruebas unitarias (29 originales y 43 de regresión) y pruebas de robustez con entradas extremas. La salida de consola de cada caso está en el Apéndice B y las capturas de pantalla en el Apéndice C.	Sección 6; Apéndices B y C

2. Descripción general del sistema

El sistema es una aplicación de consola para jugar Risk. El usuario escribe un comando después del indicador $, y el programa lo valida y lo ejecuta sobre el estado de la partida. En esta entrega están completos los componentes 1 y 2:

- **Componente 1 – Configuración del juego (estructuras lineales):** inicializar, obtener_unidades, atacar, fortificar, estado_juego, salir y ayuda. Todas las colecciones (jugadores, territorios, continentes, vecinos y cartas) se guardan en ListaLineal, una lista simplemente enlazada genérica que implementamos nosotros.
- **Componente 2 – Almacenamiento de partidas (estructuras jerárquicas):** guardar, guardar_comprimido e inicializar desde un archivo .bin. La compresión usa un árbol binario de Huffman (Huffman, 1952) construido con una cola de prioridad.
- **Componente 3 – Estrategias de juego (grafos):** costo_conquista y conquista_mas_barata ya existen en la interfaz y validan sus parámetros, pero su algoritmo se implementará en la Entrega 3. Como cada territorio ya guarda referencias a sus vecinos, el mapa forma un grafo no dirigido sobre el que trabajaremos en esa entrega.

El tablero tiene 42 territorios repartidos en 6 continentes. Quien domina un continente completo recibe un bono: 2 por América del Sur o Australia, 3 por África, 5 por América del Norte o Europa y 7 por Asia. Como el diseño es incremental, el documento incluye lo construido en la Entrega 1 y lo nuevo de la Entrega 2.

2.1 Archivos fuente

Tabla 3

Archivos fuente del proyecto

Archivo	Contenido
main.cpp	Procedimiento principal (consola interactiva), operaciones auxiliares de entrada y salida y flujo de cada comando.
ListaLineal.h / .hxx	TAD Nodo y TAD ListaLineal (plantilla genérica).
Carta.h	TAD Carta y enumeración TipoTropa.
Territorio.h / .cpp	TAD Territorio.
Continente.h / .cpp	TAD Continente.
Jugador.h / .cpp	TAD Jugador y enumeración FaseTurno.
TableroRisk.h / .cpp	TAD TableroRisk: mapa base, fronteras y consultas sobre territorios.
JuegoRisk.h / .cpp	TAD JuegoRisk: estado de la partida, turnos, reglas de los comandos, carga y guardado.
NodoHuffman.h	TAD NodoHuffman.
ColaPrioridad.h / .cpp	TAD ColaPrioridad.
ArbolHuffman.h / .cpp	TAD ArbolHuffman (codificación y decodificación).
ArchivoHuffman.h / .cpp	TAD ArchivoHuffman (lectura y escritura del archivo .bin).

Nota. La forma de compilar y el contenido del archivo .zip se describen en la sección 8.

3. Diseño: procedimiento principal y operaciones

3.1 Procedimiento principal

La Tabla 4 resume el ciclo de la consola y la Figura 1 muestra su esquemático.

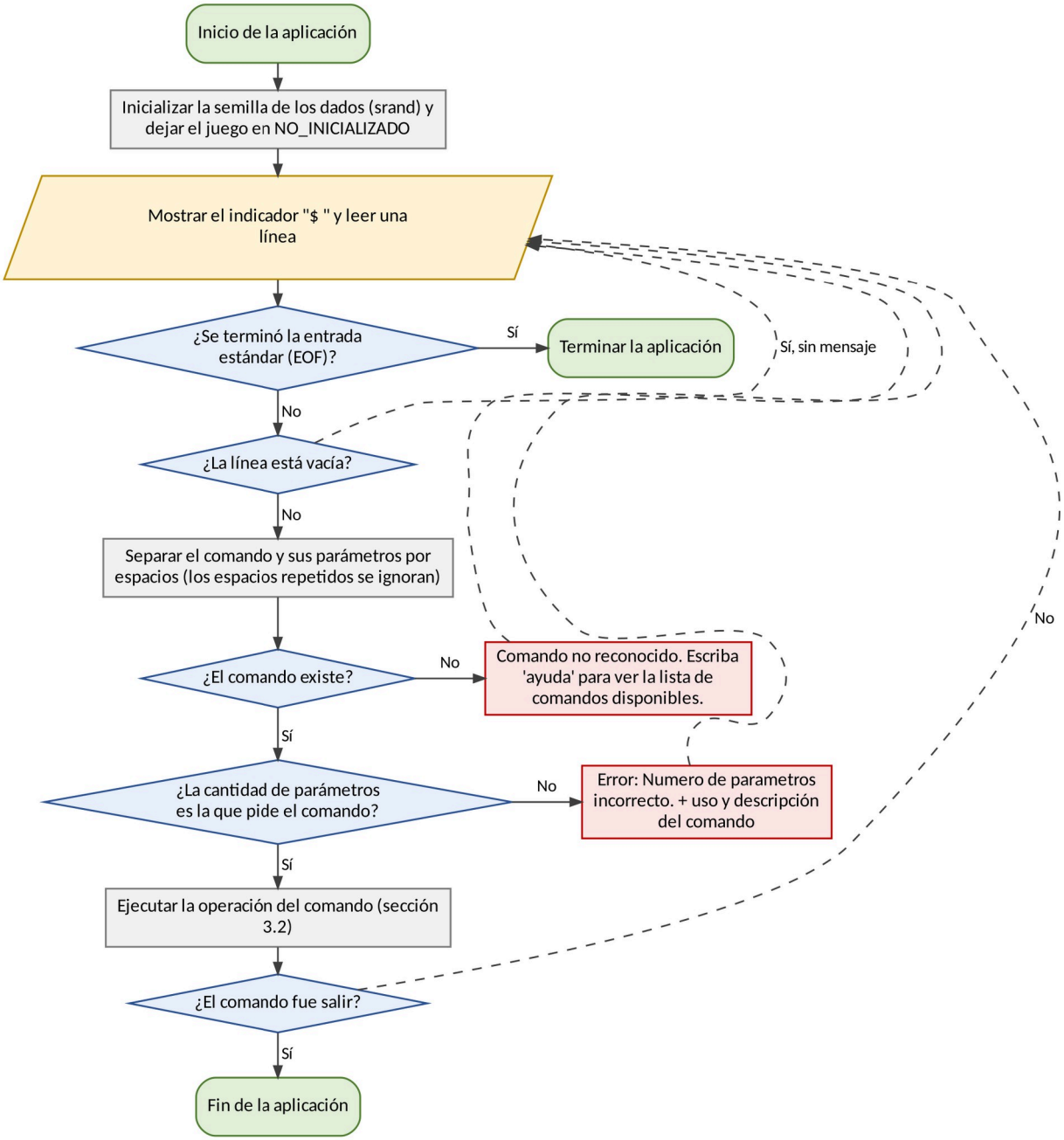
Tabla 4

Especificación del procedimiento principal

Entradas	Líneas de texto escritas en la consola. Cada línea tiene un comando y sus parámetros separados por espacios.
Salidas	Los mensajes de éxito o error del enunciado para cada comando, la ayuda de uso o “Comando no reconocido”.
Precondiciones	Ninguna. Al arrancar, el juego está en NO_INICIALIZADO y se inicializa la semilla de los dados.
Postcondiciones	Cada comando válido actualiza el estado de la partida (TAD JuegoRisk). El programa termina con salir o cuando se acaba la entrada estándar.
Funcionamiento	1. Inicializar una sola vez la semilla aleatoria (srand), para que los dados cambien en cada ejecución. 2. Mostrar el indicador $ y leer una línea; las líneas vacías se ignoran sin mensajes. 3. Separar el comando de los parámetros (los espacios repetidos no generan parámetros vacíos). 4. Si el comando no existe, mostrar “Comando no reconocido. Escriba 'ayuda' para ver la lista de comandos disponibles.”. 5. Si la cantidad de parámetros es incorrecta, mostrar “Error: Numero de parametros incorrecto.” y la ayuda del comando. 6. Ejecutar la operación del comando, mostrar su mensaje y repetir hasta recibir salir.

Figura 1

Esquemático del procedimiento principal

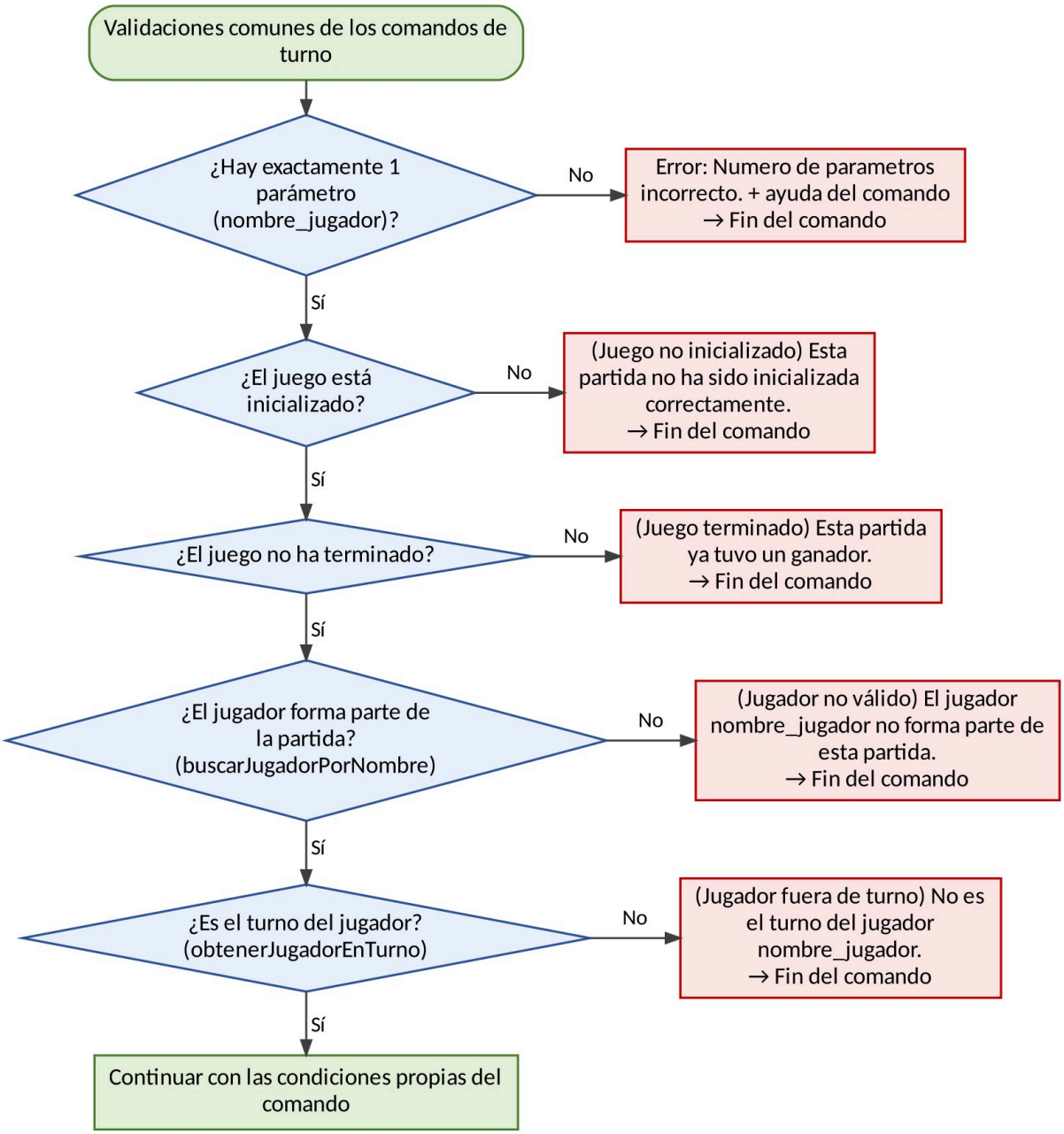


Los tres comandos de turno (obtener_unidades, atacar y fortificar) revisan primero las mismas cinco condiciones, siempre en el mismo orden. Para no repetirlas en cada esquemático

las dibujamos una sola vez en la Figura 2: la primera condición que falla muestra su mensaje y el comando termina sin tocar el estado.

Figura 2

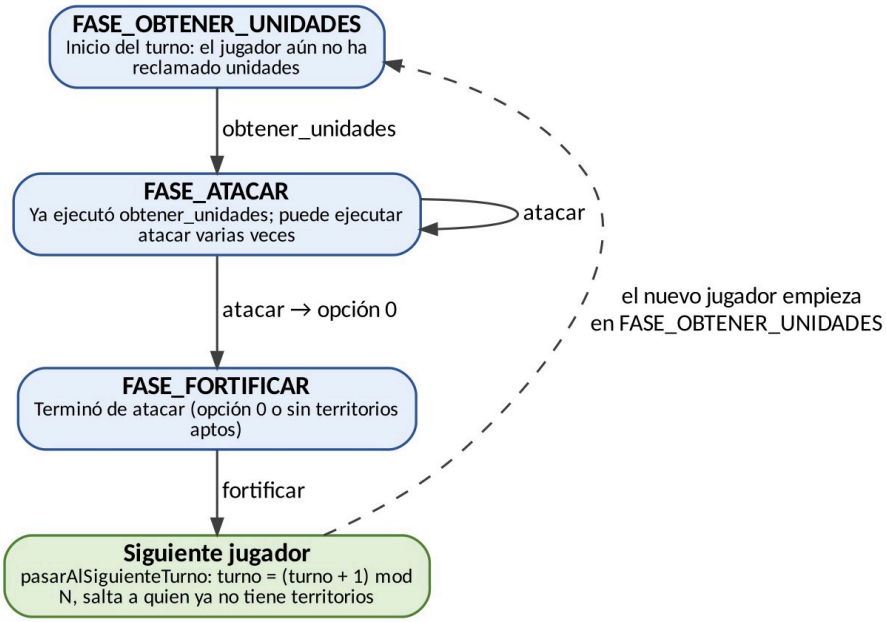
Validaciones comunes de los comandos obtener_unidades, atacar y fortificar



El avance del turno depende del dato faseTurnoActual del TAD Jugador y de la bandera haAtacadoTurnoActual. La Figura 3 muestra en qué orden se deben ejecutar los comandos.

Figura 3

Fases del turno de un jugador y comandos que las hacen avanzar



3.2 Comandos (operaciones auxiliares)

Cada comando tiene una ficha (uso, entradas, condiciones en el orden en que se evalúan, salidas, efecto y TADs que hacen el trabajo) y un esquemático. Los textos entre paréntesis son los mensajes exactos que pide el enunciado. La consola solo pide datos y muestra resultados; las reglas están en los TADs indicados en cada ficha. En los esquemáticos, la flecha punteada indica que el programa vuelve a un paso anterior (por ejemplo, a pedir otra vez un código) y "→ Fin del comando" indica que el comando termina ahí sin cambiar el estado. Los más largos los dividimos en partes; un círculo con letra continúa en el círculo con la misma letra de otra parte.

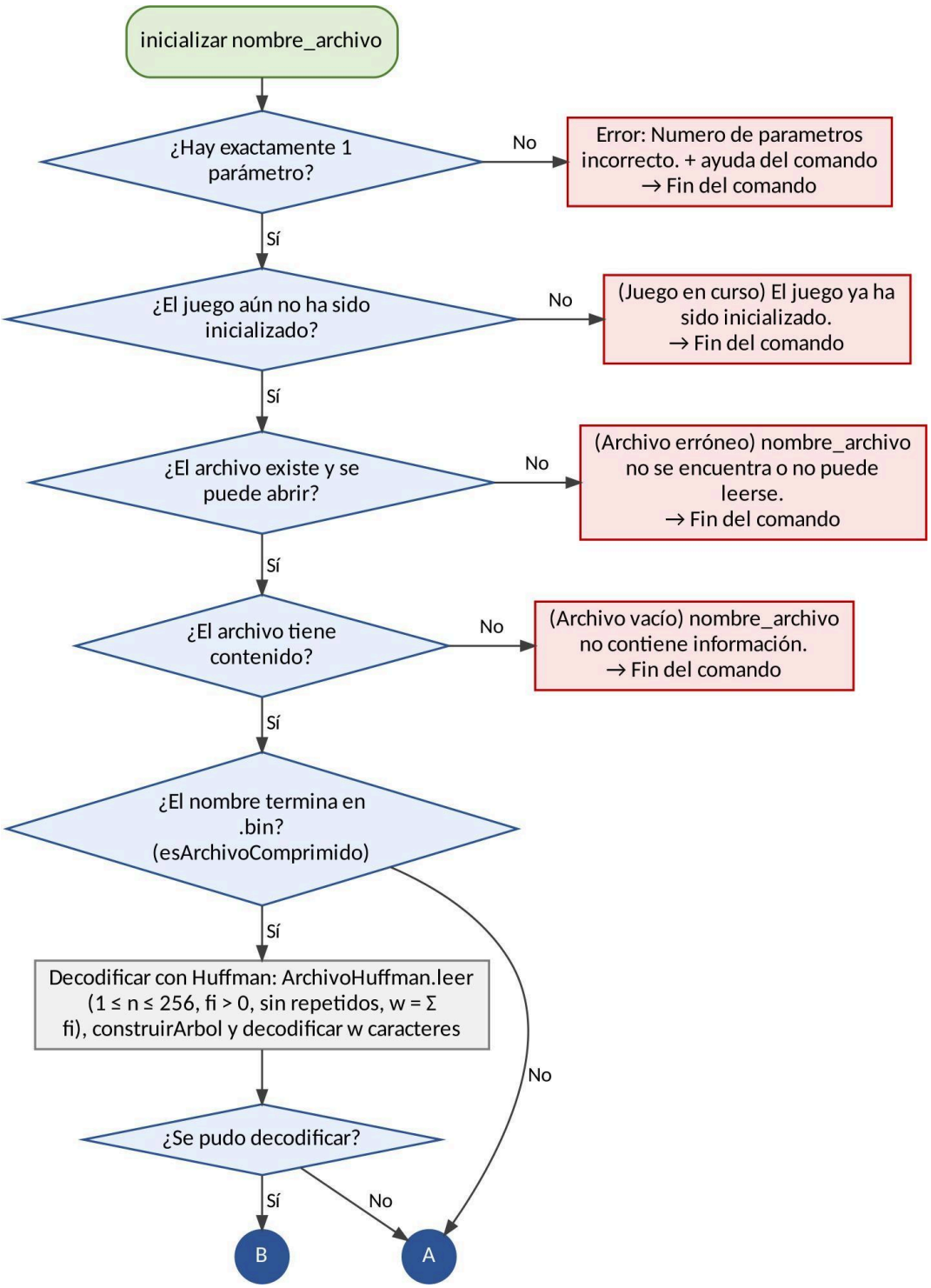
Tabla 5

Ficha del comando inicializar

Uso	inicializar <nombre_archivo>
Entradas	nombre_archivo: archivo de texto con el formato de inicio o archivo .bin generado por guardar_comprimido. La extensión .bin (sin distinguir mayúsculas) decide qué formato se intenta primero; si ese falla, se intenta el otro, así que un comprimido con otra extensión, o un texto llamado .bin, también se reconocen por su contenido.
Condiciones	1. Se recibe exactamente 1 parámetro. 2. El juego no ha sido inicializado. 3. El archivo existe y se puede abrir. 4. El archivo no está vacío. 5. Si se lee como comprimido: la cabecera es válida ($1 \leq n \leq 256$, sin caracteres repetidos, $f_i > 0$, $w > 0$ y $\Sigma f_i = w$) y el texto decodificado tiene w caracteres. 6. El contenido cumple el formato de inicio: N entre 3 y 6; N líneas "nombre color" (nombre de máximo 8 caracteres; nombres y colores sin repetir); 42 líneas "código color tropas" con código existente y no repetido, color de un jugador y al menos 1 tropa, y sin datos sobrantes. 7. Solo si el contenido se leyó como texto plano: cada jugador tiene al menos 1 territorio y la suma de sus unidades es 35, 30, 25 o 20 (para 3, 4, 5 o 6 jugadores).
Salidas	Éxito: (Comando correcto) El juego se ha inicializado correctamente. Errores: Error: Numero de parametros incorrecto. (condición 1); (Juego en curso) El juego ya ha sido inicializado. (2); (Archivo erróneo) nombre_archivo no se encuentra o no puede leerse. (3); (Archivo vacío) nombre_archivo no contiene información. (4); (Archivo sin formato) nombre_archivo no contiene información en el formato esperado. (5 a 7).
Efecto	Si es correcto, el juego queda EN_CURSO con los jugadores y los 42 territorios cargados, el turno en el primer jugador del archivo que tenga territorios y gruposCanjeados en 0. Si falla, el estado del juego no cambia: la lectura y la validación se hacen sobre un juego temporal (sección 3.4).
TADs	JuegoRisk.inicializarDesdeArchivo, cargarDesdeTexto y unidadesInicialesPorJugador; ArchivoHuffman.esArchivoComprimido, leer y desempaquetarBits; ArbolHuffman.fijarFrecuencias, construirArbol y decodificar; TableroRisk.configurarMapaBase.

Figura 4

Esquemático del comando inicializar (archivo de texto o comprimido) (parte 1 de 2)



Nota. Los conectores A y B continúan en la parte 2.

Figura 4 (continuación)

Esquemático del comando inicializar (archivo de texto o comprimido) (parte 2 de 2)

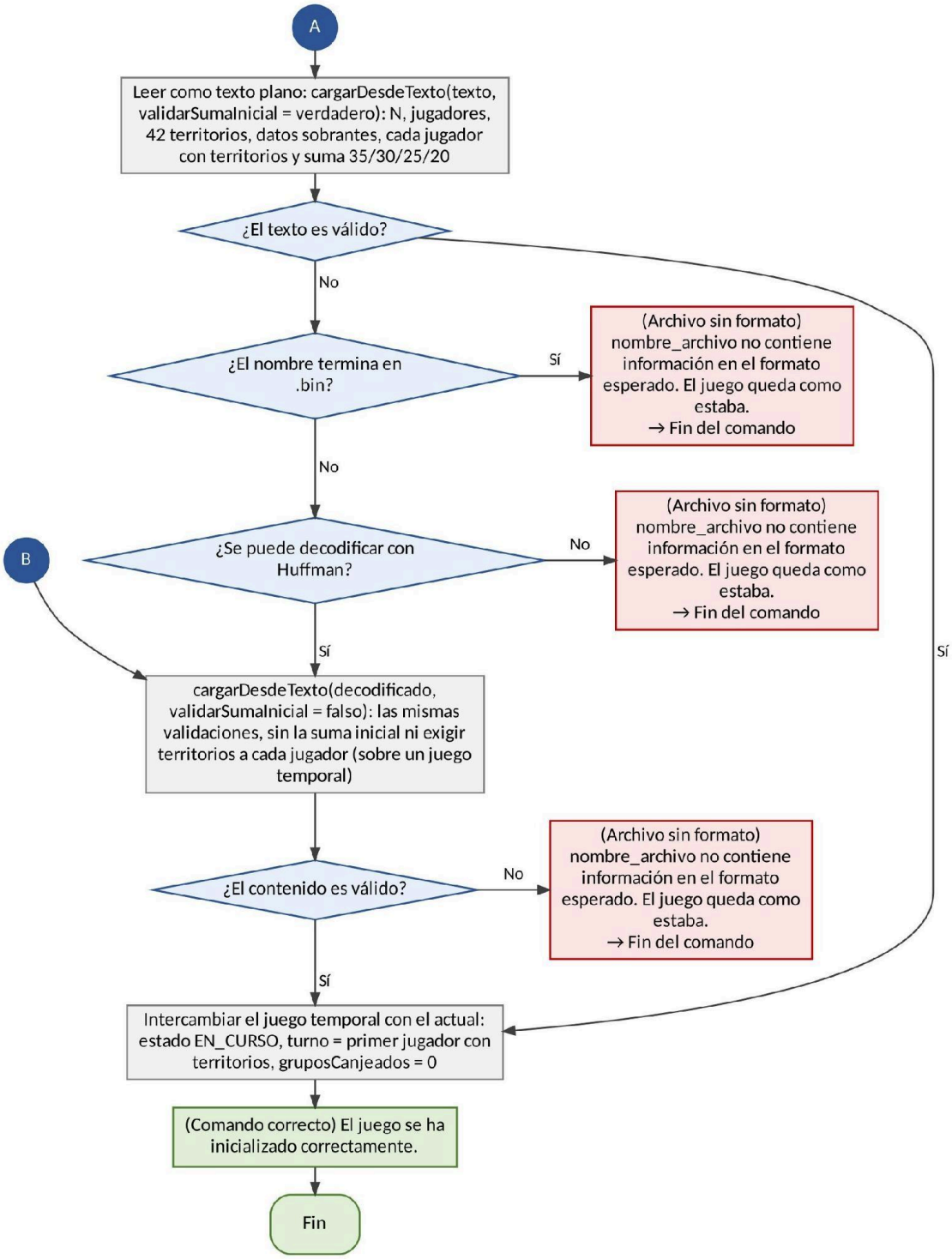


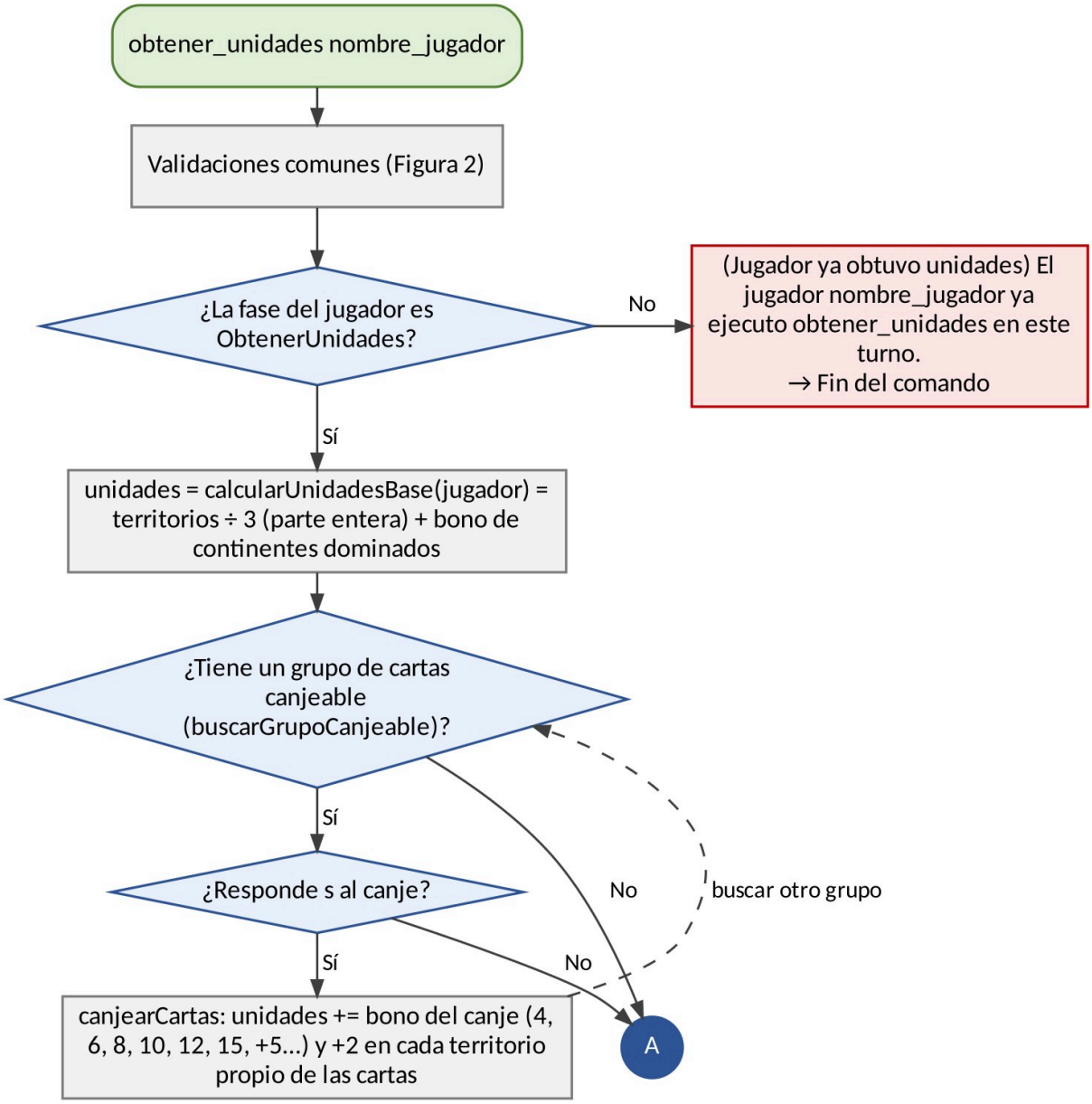
Tabla 6

Ficha del comando obtener_unidades

Uso	obtener_unidades <nombre_jugador>
Entradas	nombre_jugador. Luego, de forma interactiva: la respuesta s/n al canje de cartas (solo si hay un grupo canjeable), el código de un territorio propio y la cantidad de unidades para ese territorio, hasta ubicar todas.
Condiciones	1. Validaciones comunes (Figura 2). 2. El jugador no ha ejecutado obtener_unidades en este turno (su fase es ObtenerUnidades). 3. El código es de un territorio del jugador; si no, se muestra el error y se vuelve a pedir. 4. La cantidad es un entero entre 1 y las unidades por ubicar; si no, se muestra el error y se vuelve a pedir.
Salidas	Unidades por ubicar, territorios del jugador con su código y tropas, y "-> Se han anadido N tropas a nombre_territorio." en cada ubicación; con 0 unidades, "Tienes 0 unidades para ubicar en este turno.". Si hay canje: "Has canjeado un grupo de cartas por N unidades." y, por cada territorio propio que aparezca en las cartas, el aviso de +2 unidades en ese territorio. Éxito: (Comando correcto) El jugador nombre_jugador ha terminado de reclamar y ubicar sus unidades. Errores: los de las validaciones comunes; (Jugador ya obtuvo unidades) El jugador nombre_jugador ya ejecuto obtener_unidades en este turno.; (Codigo invalido, intenta de nuevo) y (Cantidad invalida, intenta de nuevo).
Cálculo	territorios ÷ 3 (parte entera) + bono de los continentes dominados. Cada canje suma el bono del canje y 2 tropas en cada territorio propio que aparezca en las cartas (sección 3.4).
Efecto	Las unidades quedan en los territorios escogidos y el jugador pasa a la fase Atacar.
TADs	JuegoRisk.calcularUnidadesBase, canjearCartas, obtenerBonoSiguienteCanje y ubicarUnidades; Jugador.buscarGrupoCanjeable, eliminarCartas y avanzarFase; TableroRisk.territoriosDeJugador y obtenerBonoPorContinentesDominados.

Figura 5

Esquemático del comando obtener_unidades (parte 1 de 2)



Nota. El conector A continúa en la parte 2.

Figura 5 (continuación)

Esquemático del comando obtener_unidades (parte 2 de 2)

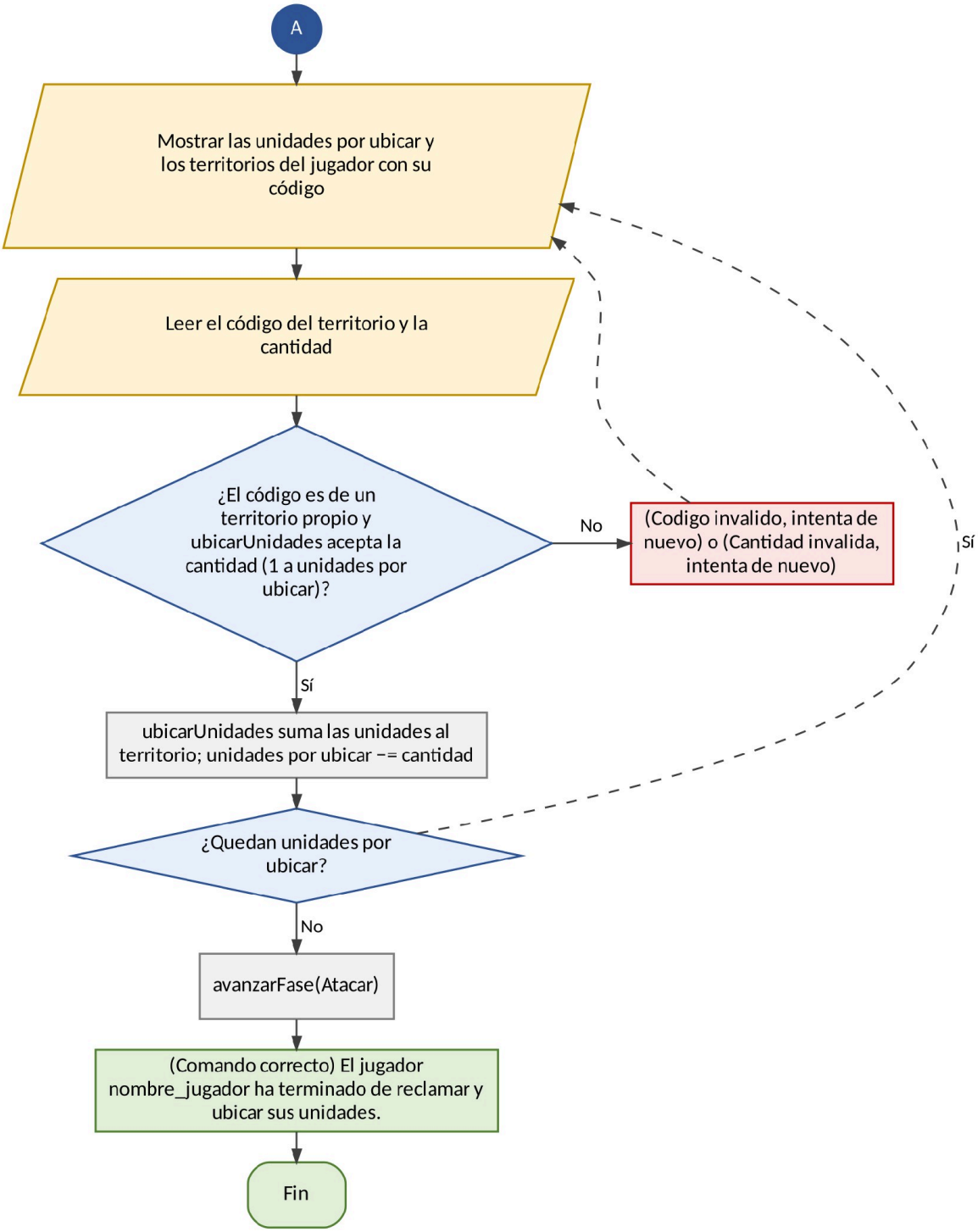


Tabla 7

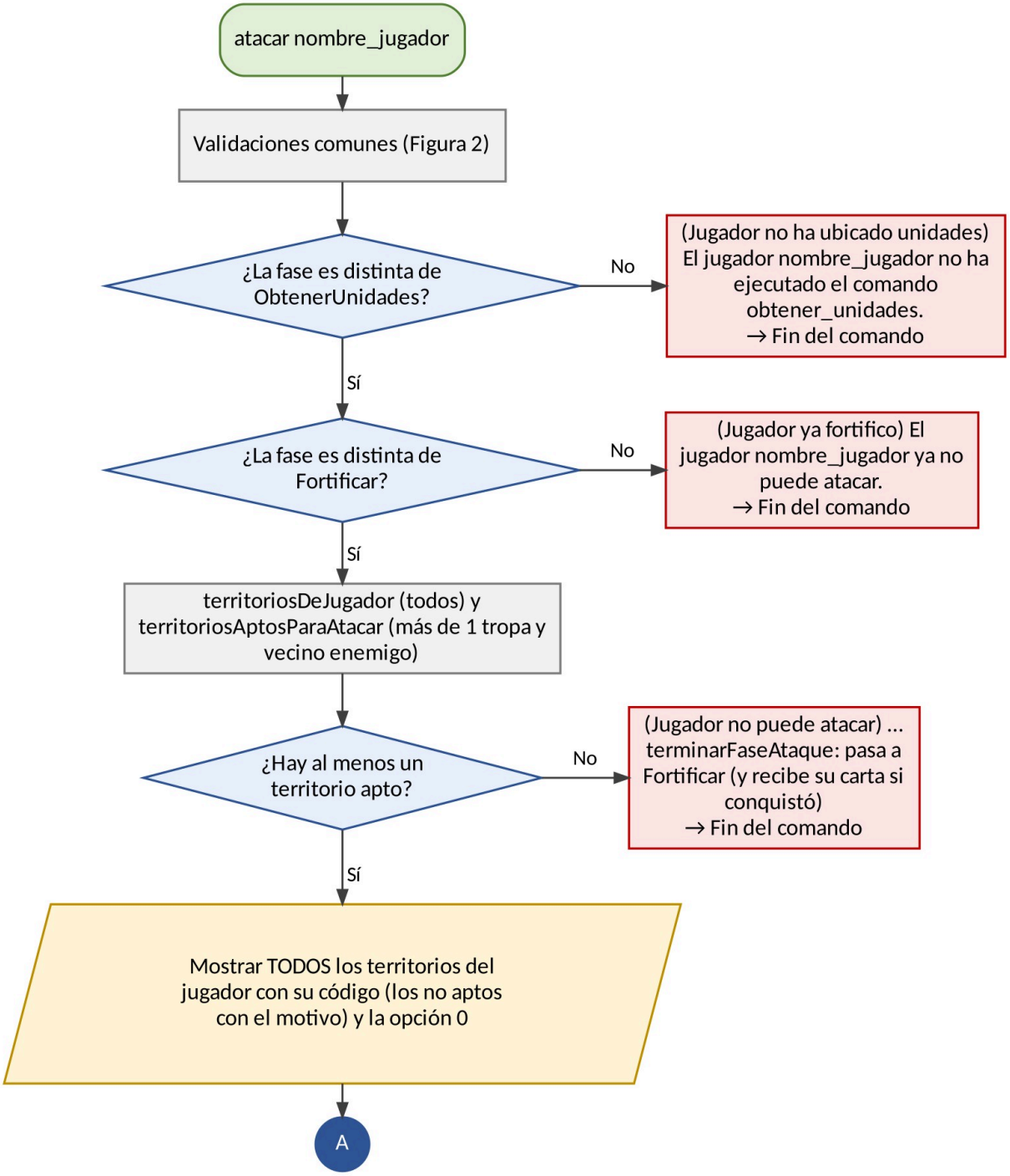
Ficha del comando atacar

Uso	atacar <nombre_jugador>
Entradas	nombre_jugador. Luego, de forma interactiva: el código del territorio de origen (o 0 para terminar la fase de ataque), las tropas atacantes, el código del territorio enemigo y, después de cada lanzamiento, la respuesta s/n para continuar.
Condiciones	1. Validaciones comunes (Figura 2). 2. El jugador ya ejecutó obtener_unidades (su fase no es ObtenerUnidades). 3. El jugador no ha terminado la fase de ataque (su fase no es Fortificar). 4. Tiene al menos un territorio con más de 1 tropa y un vecino de otro jugador; si no, se informa y pasa a fortificar. 5. El origen es uno de esos territorios (por código) o es 0 para terminar. 6. Las tropas atacantes están entre 1 y (tropas del origen − 1); lo valida apartarTropasAtacantes. 7. El destino es un vecino del origen ocupado por otro jugador (por código).
Salidas	Todos los territorios del jugador con código, nombre y tropas; los que no pueden atacar se marcan con el motivo ([No disponible: necesita mas de 1 tropa] o [No disponible: no tiene vecinos enemigos]) y aparece la opción 0. En cada ronda: dados de ambos lados, resultado de cada duelo y tropas restantes. Si conquista: "¡Has conquistado el territorio nombre!" y las tropas ubicadas; si llega a 42 territorios, se anuncia la victoria. Si el ataque se detiene, las tropas sobrevivientes regresan al origen. Éxito: (Comando correcto) El jugador nombre_jugador ha terminado de atacar. Errores: los de las validaciones comunes; (Jugador no ha ubicado unidades) El jugador nombre_jugador no ha ejecutado el comando obtener_unidades.; (Jugador ya fortifico) El jugador nombre_jugador ya no puede atacar.; (Jugador no puede atacar) nombre_jugador no tiene territorios con mas de 1 tropa y vecinos enemigos para atacar.; Error: Codigo invalido o territorio no disponible para atacar; Error: Opcion invalida. El ataque no se realizo; puedes ejecutar atacar de nuevo., y Error: Codigo invalido.
Dados	El atacante lanza min(3, atacantes) dados y el defensor min(2, defensores); el empate favorece al defensor (sección 3.4 y Figura 7).
Efecto	Se actualizan las tropas de ambos territorios. Si el defensor queda en 0, el territorio pasa al atacante con las tropas sobrevivientes, se marca la conquista del turno y se verifica la victoria. Ejecutar atacar habilita fortificar. Con la opción 0 (o sin territorios aptos) la fase pasa a Fortificar y, si conquistó en el turno, recibe 1 carta (máximo una por turno). Mientras no escriba 0 puede volver a atacar. Si las tropas atacantes son inválidas, el comando termina con el mensaje de error, sin el mensaje de éxito: no se modifica nada y el jugador sigue en la fase de ataque, así que

	puede volver a ejecutar atacar.
TADs	TableroRisk.territoriosDeJugador y territoriosAptosParaAtacar; Territorio.tieneVecinoEnemigo; JuegoRisk.apartarTropasAtacantes, resolverRondaCombate (usa lanzarDado), conquistarTerritorio (usa verificarGanadorGlobal), actualizarDefensor, devolverTropasAtacantes y terminarFaseAtaque; Jugador.registrarAtaque.

Figura 6

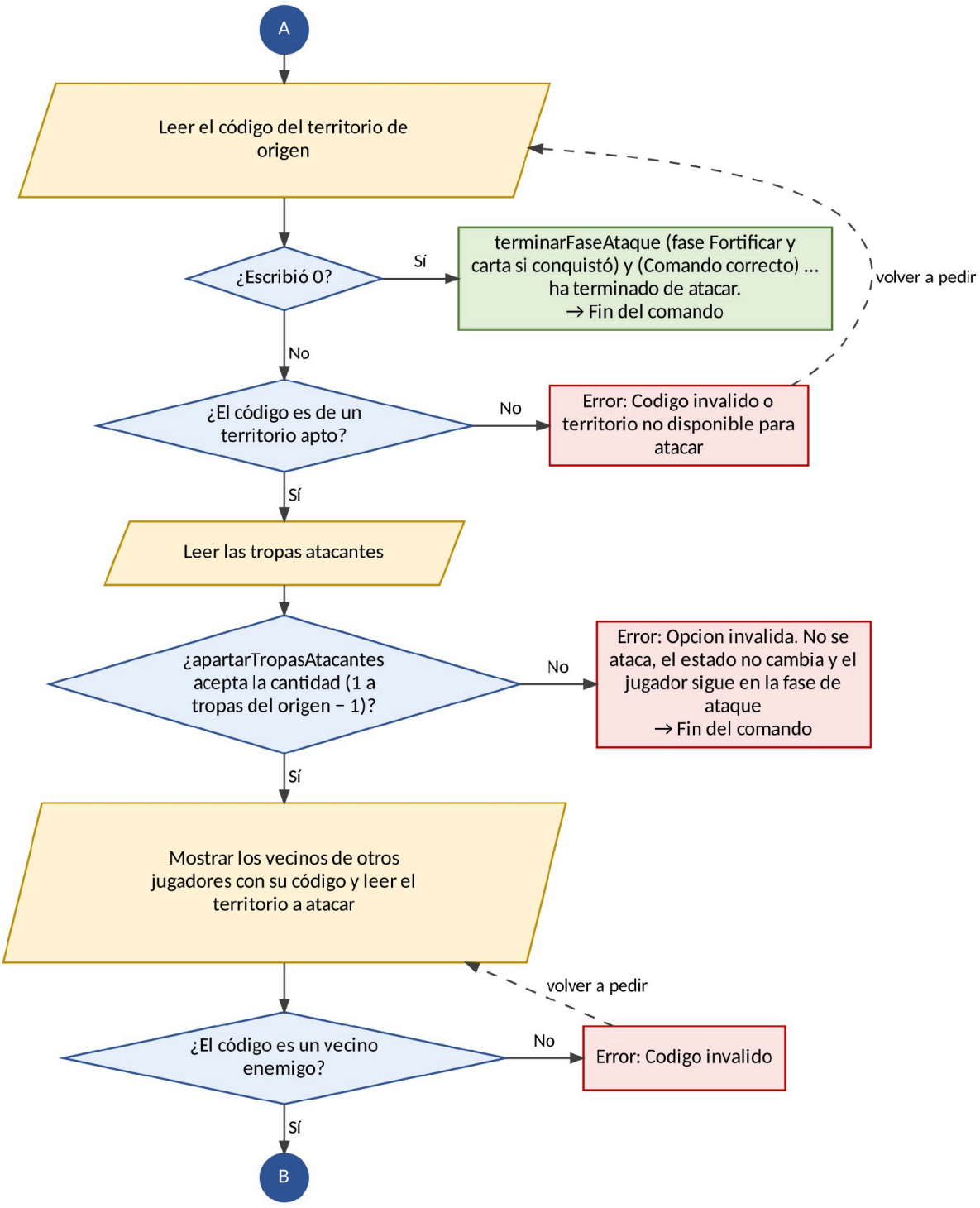
Esquemático del comando atacar (parte 1 de 4)



Nota. El conector A continúa en la parte 2.

Figura 6 (continuación)

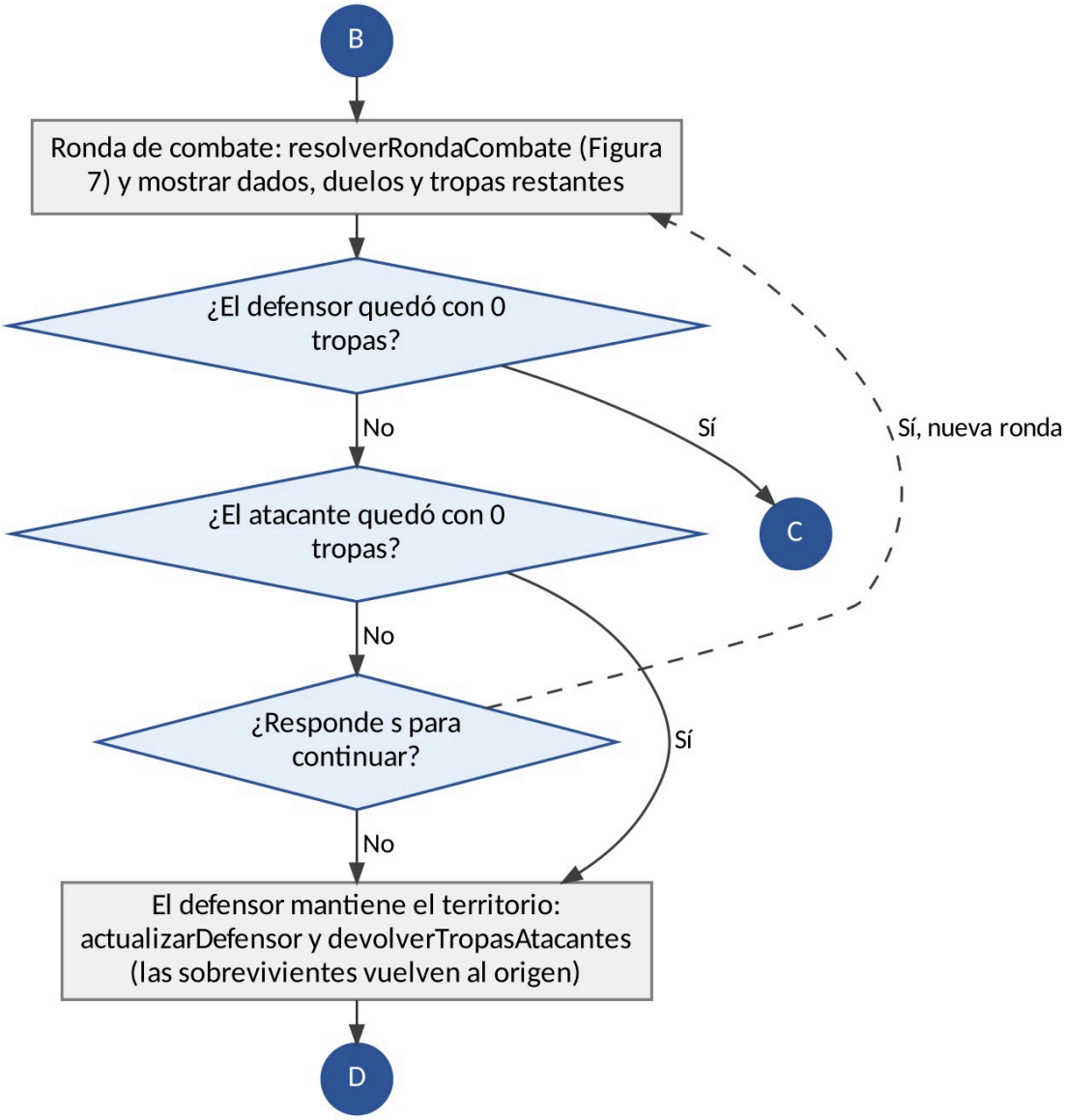
Esquemático del comando atacar (parte 2 de 4)



Nota. El conector B continúa en la parte 3.

Figura 6 (continuación)

Esquemático del comando atacar (parte 3 de 4)



Nota. Los conectores C y D continúan en la parte 4.

Figura 6 (continuación)

Esquemático del comando atacar (parte 4 de 4)

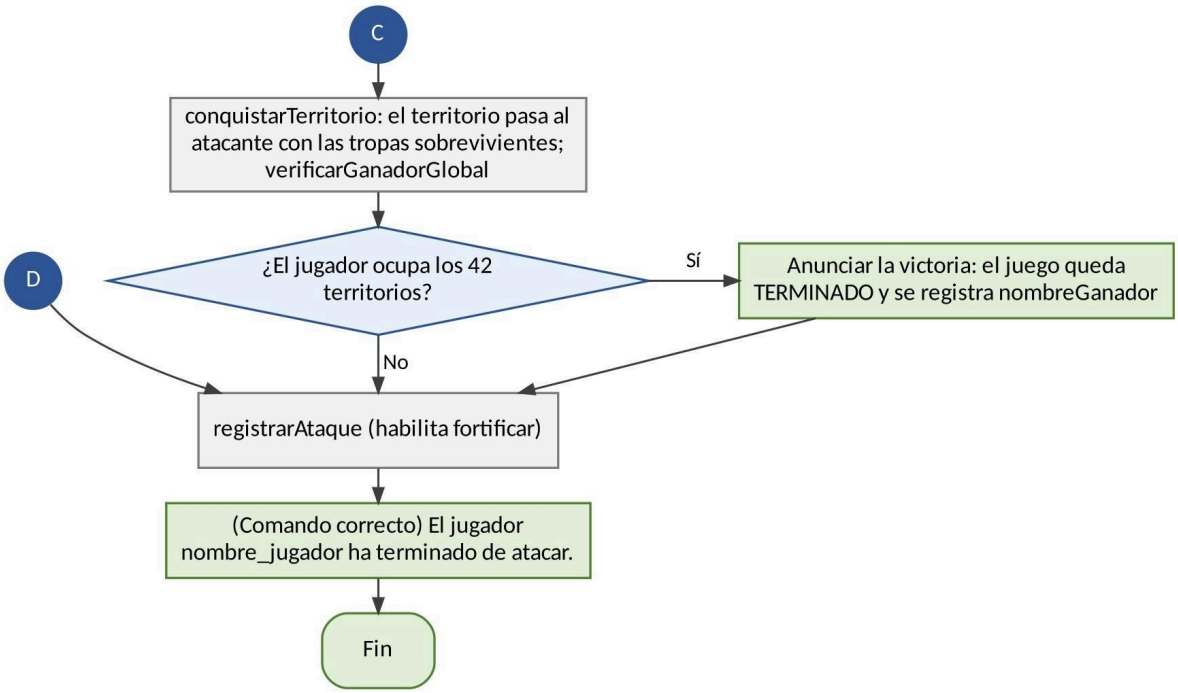


Figura 7

Ronda de combate (operación resolverRondaCombate del TAD JuegoRisk)

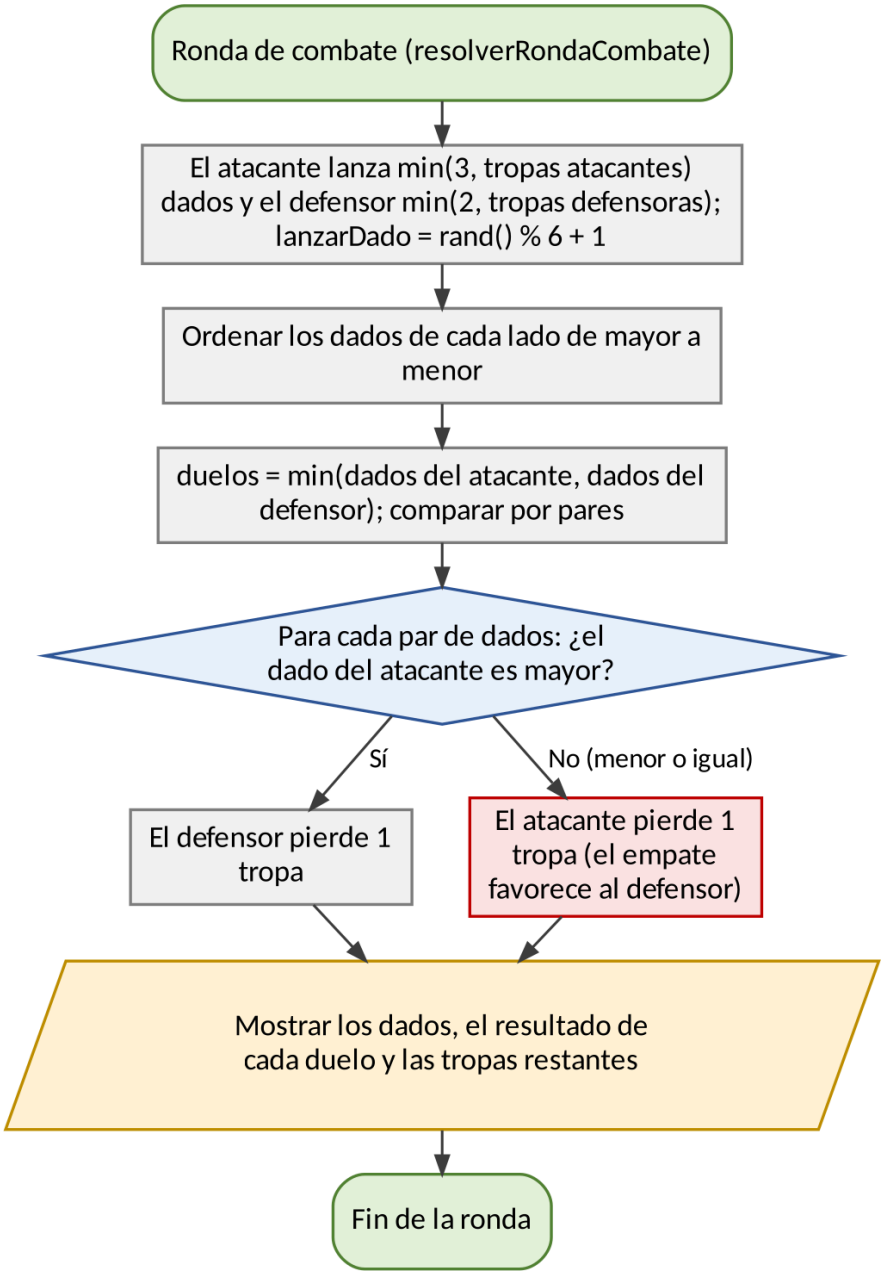


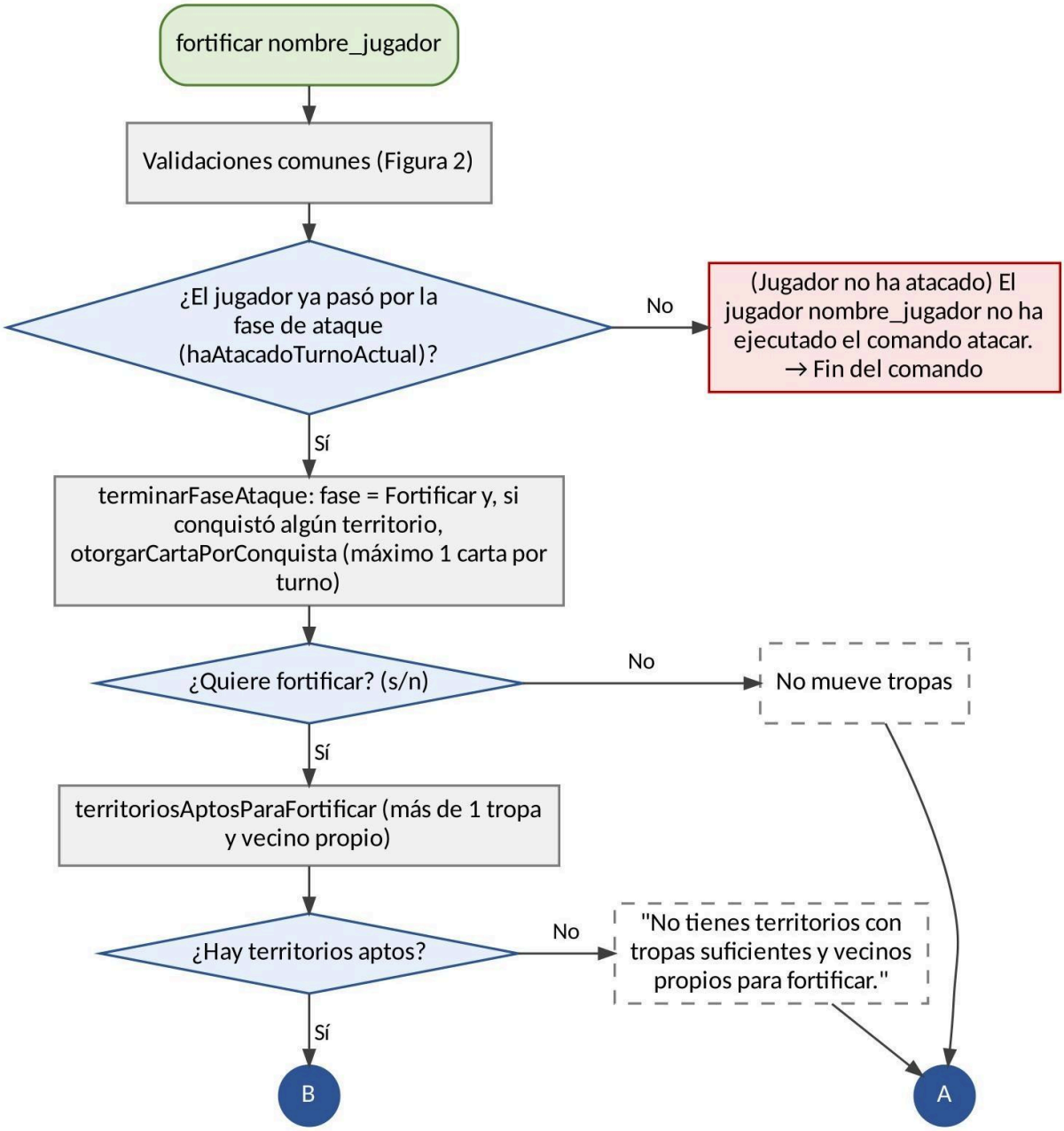
Tabla 8

Ficha del comando fortificar

Uso	fortificar <nombre_jugador>
Entradas	nombre_jugador. Luego: la respuesta s/n, el código del territorio de origen, el código del vecino propio de destino y la cantidad de tropas a mover.
Condiciones	1. Validaciones comunes (Figura 2). 2. El jugador ya pasó por la fase de ataque (haAtacadoTurnoActual). 3. El origen es un territorio del jugador con más de 1 tropa y al menos un vecino propio (por código). 4. El destino es un vecino propio del origen (por código). 5. La cantidad está entre 1 y (tropas del origen − 1).
Salidas	Territorios aptos con su código y tropas, y "Has movido N tropas de origen a destino." Éxito: (Comando correcto) El jugador nombre_jugador ha terminado de fortificar su posición. Errores: los de las validaciones comunes y (Jugador no ha atacado) El jugador nombre_jugador no ha ejecutado el comando atacar. Si un código o la cantidad no son válidos se muestra "Error: Codigo invalido. Intenta de nuevo." o "Error: Opcion invalida. Intenta de nuevo:" y se vuelve a pedir el dato.
Efecto	Se mueven tropas de un territorio a un solo vecino propio y el origen conserva al menos 1 tropa. Si el jugador conquistó y aún no había recibido su carta, la recibe. Al final el turno pasa al siguiente jugador de forma circular, saltando a quienes ya no ocupan territorios.
TADs	TableroRisk.territoriosAptosParaFortificar; Territorio.tieneVecinoPropio; JuegoRisk.terminarFaseAtaque (usa otorgarCartaPorConquista), moverTropas y pasarAlSiguienteTurno.

Figura 8

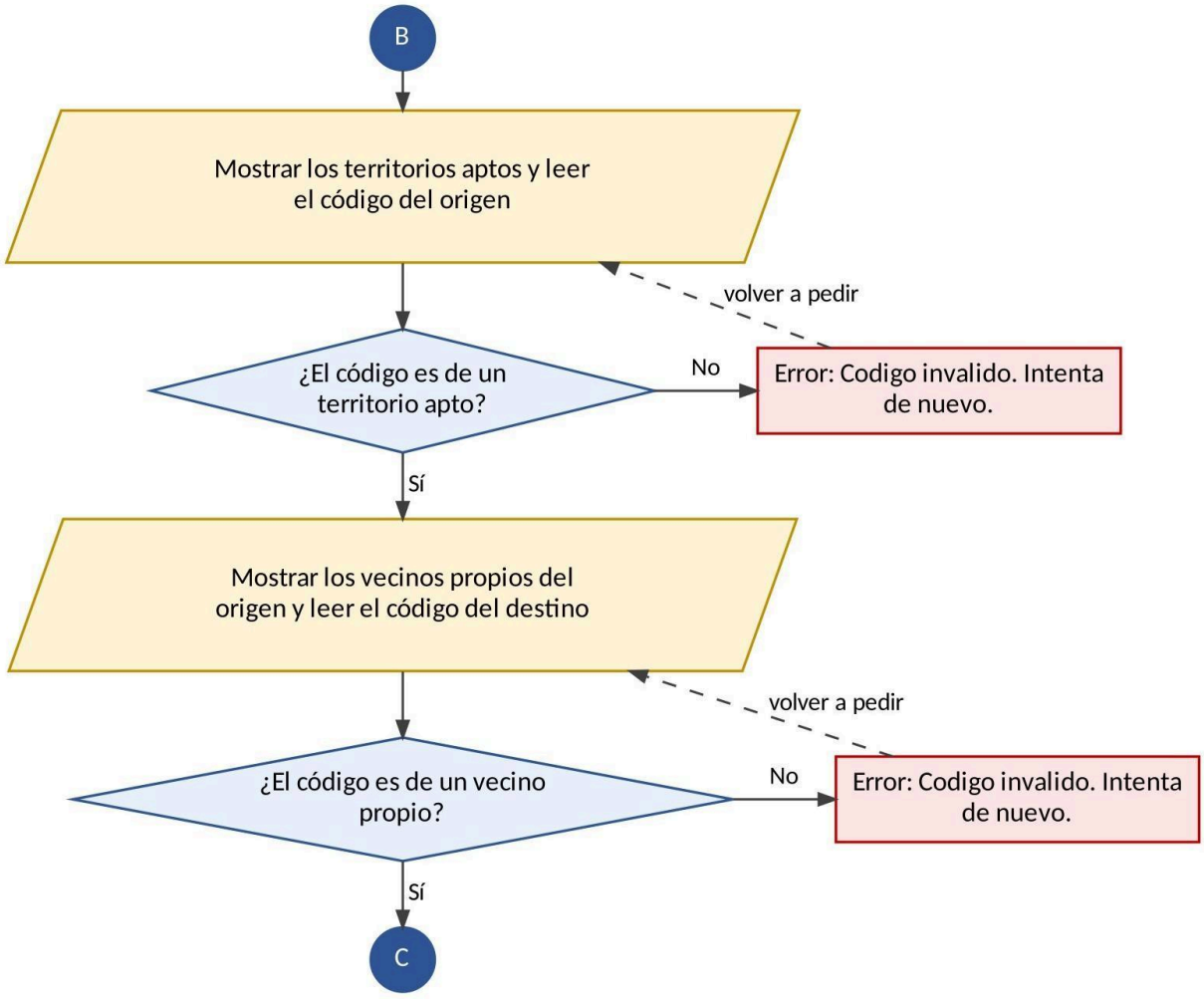
Esquemático del comando fortificar (parte 1 de 3)



Nota. El conector B continúa en la parte 2; el conector A continúa en la parte 3.

Figura 8 (continuación)

Esquemático del comando fortificar (parte 2 de 3)



Nota. El conector C continúa en la parte 3.

Figura 8 (continuación)

Esquemático del comando fortificar (parte 3 de 3)

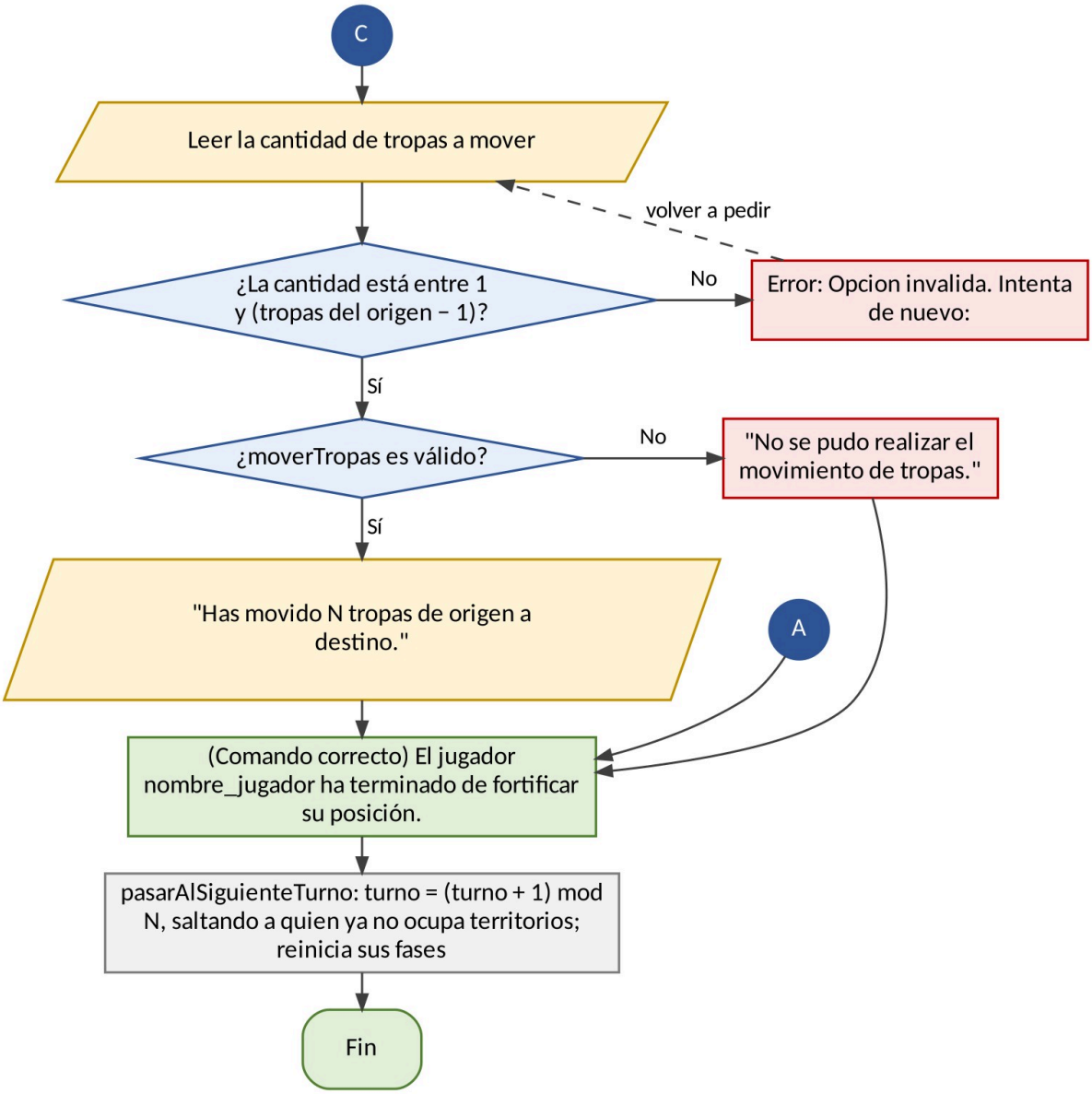


Tabla 9

Ficha del comando estado_juego

Uso	estado_juego
Entradas	Ninguna.
Condiciones	1. No se reciben parámetros. 2. El juego está inicializado. 3. El juego no ha terminado.
Salidas	Jugadores con nombre y color, jugador en turno y, para cada uno de los 42 territorios, código, nombre, color del ocupante y unidades. Errores: (Juego no inicializado) Esta partida no ha sido inicializada correctamente. y (Juego terminado) Esta partida ya tuvo un ganador.
Efecto	Ninguno; solo consulta el estado.
TADs	JuegoRisk.obtenerJugadorEnTurno; ListaLineal.obtener sobre jugadores y territorios.

Tabla 10

Ficha de los comandos guardar y guardar_comprimido

Uso	guardar <nombre_archivo> guardar_comprimido <nombre_archivo>
Entradas	nombre_archivo: ruta del archivo que se va a crear (para guardar_comprimido, con extensión .bin).
Condiciones	1. Se recibe exactamente 1 parámetro. 2. El juego está inicializado. 3. El archivo se puede crear y escribir.
Salidas	(Juego no inicializado) Esta partida no ha sido inicializada correctamente. guardar: (Comando correcto) La partida ha sido guardada correctamente. o (Error al guardar) La partida no ha sido guardada correctamente. guardar_comprimido: (Comando correcto) La partida ha sido codificada y guardada correctamente. o (Error al codificar y/o guardar) La partida no ha sido codificada ni guardada correctamente.
Efecto	Se crea o sobrescribe el archivo con el estado en el mismo formato del archivo de inicio: N, N líneas “nombre color” y 42 líneas “código color tropas”. En guardar_comprimido ese texto se codifica con Huffman y se escribe con la estructura n c1 f1 … cn fn w binary_code; al decodificarlo se obtiene exactamente el texto que produce guardar.
TADs	JuegoRisk.serializarEstado, guardarEnTexto y guardarEnComprimido; ArbolHuffman (calcularFrecuencias, construirArbol, generarCodigos y codificar); ColaPrioridad; ArchivoHuffman.escribir y empaquetarBits.

Figura 9

Esquemático de los comandos guardar y guardar_comprimido

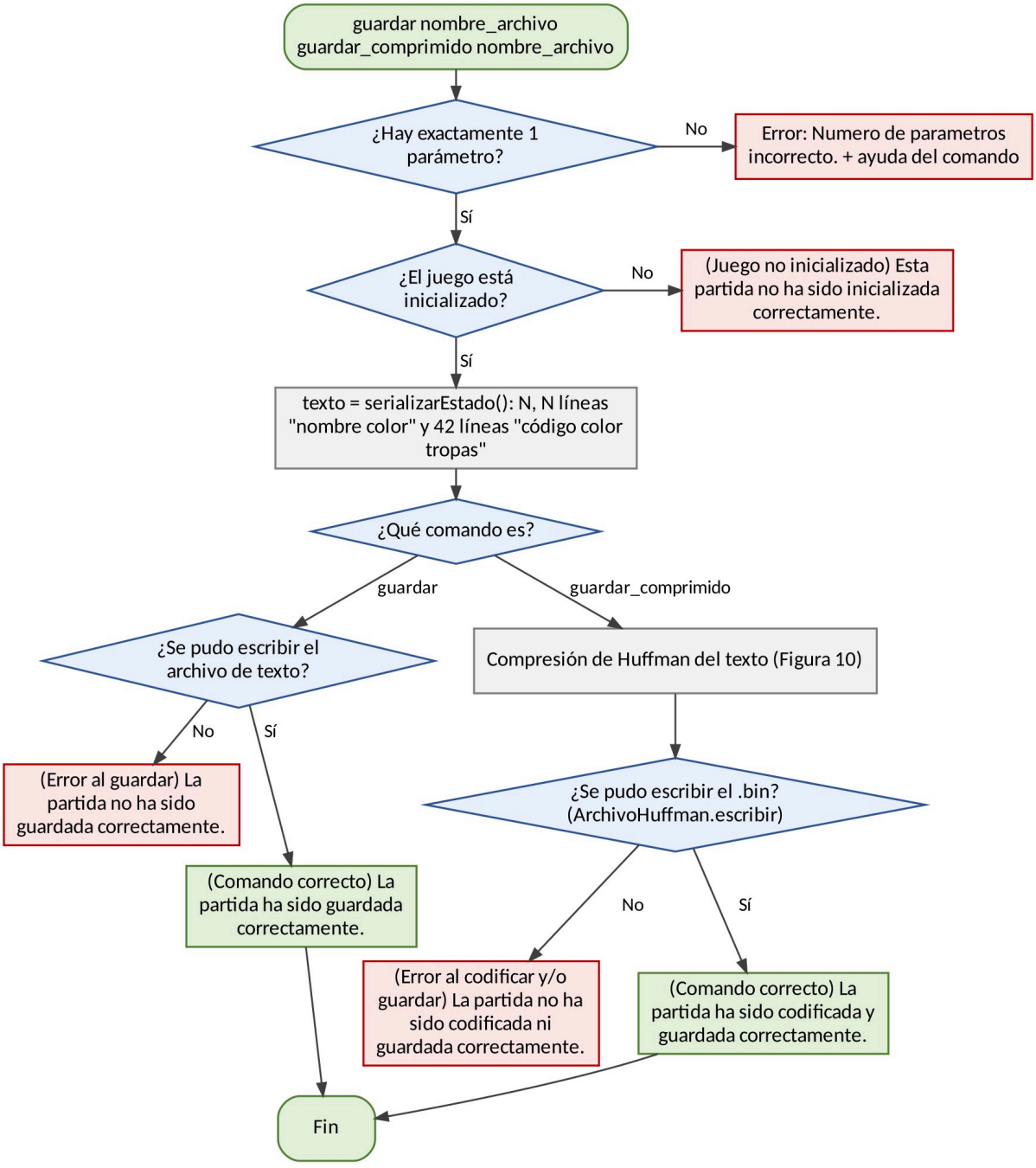


Figura 10

Compresión con el algoritmo de Huffman (usada por guardar_comprimido)

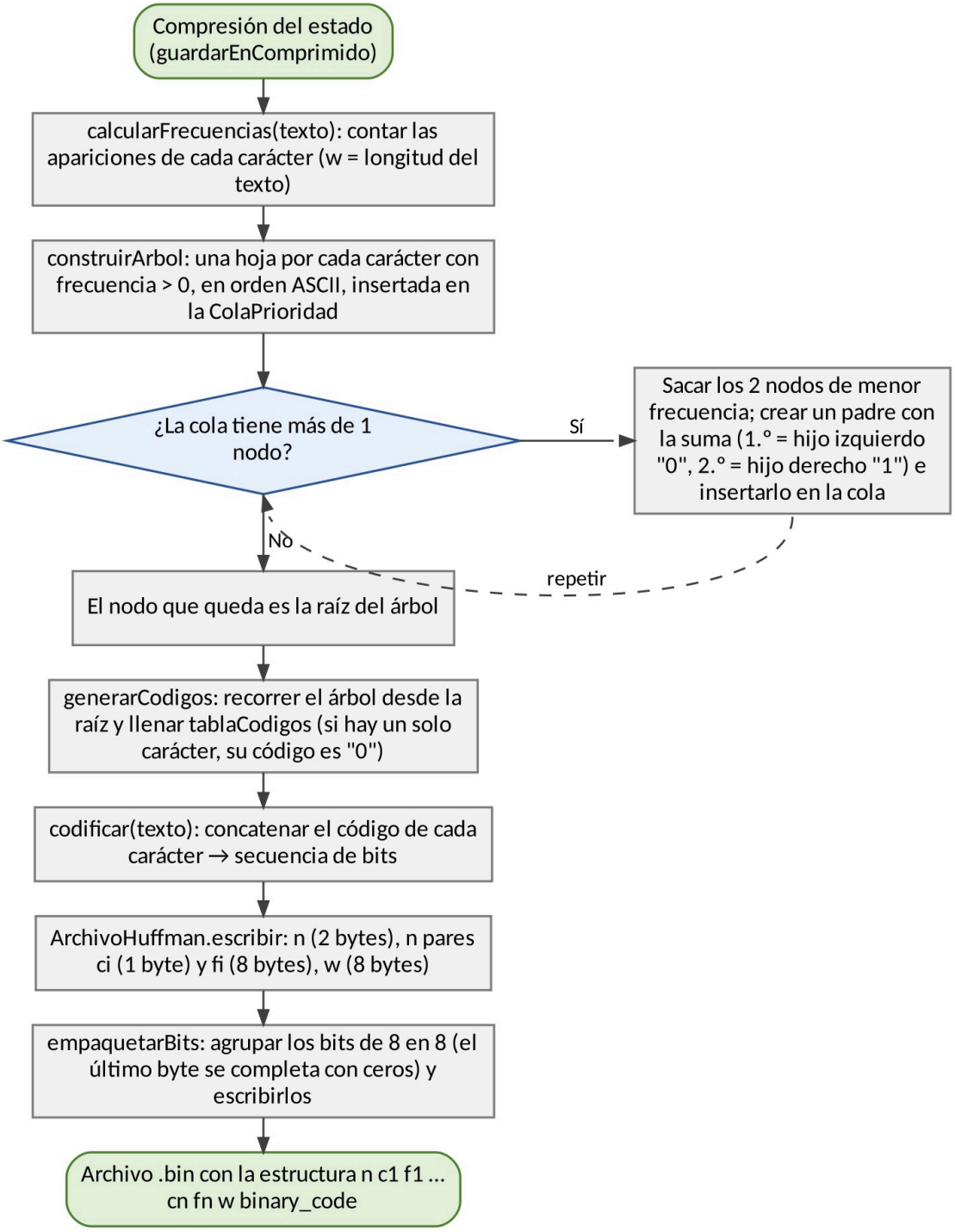


Tabla 11

Comandos salir, ayuda, costo_conquista y conquista_mas_barata

Comando	Entradas	Condiciones	Salidas y efecto
salir	Ninguna.	Sin parámetros; no depende del estado del juego.	No muestra nada. Cambia la bandera del ciclo principal y termina la aplicación de forma limpia.
ayuda [comando]	Ninguna o el nombre de un comando.	0 o 1 parámetro; no depende del estado del juego.	Sin parámetros, lista los comandos. Con un parámetro, muestra el uso y la descripción del comando o indica que no existe.
costo_conquista <nombre_jugador> <territorio>	Nombre del jugador y código del territorio.	2 parámetros; juego inicializado y no terminado.	Entrega 3. Hoy muestra el mensaje con el formato del enunciado, sin calcular rutas. Errores: (Juego no inicializado) y (Juego terminado).
conquista_mas_barata <nombre_jugador>	Nombre del jugador.	1 parámetro; juego inicializado y no terminado.	Igual que costo_conquista: pendiente para la Entrega 3.

3.3 Operaciones auxiliares del procedimiento

principal Tabla 12

Operaciones auxiliares de main.cpp

Operación	Entradas	Salida	Descripción
leerLinea	Ninguna	cadena	Lee una línea completa y quita los espacios del inicio y del final. Si la entrada se acaba, termina la ejecución de forma limpia.
leerEntero	referencia a un entero	booleano	Lee una línea y la convierte en entero. Retorna verdadero solo si la línea contiene exactamente un entero válido.
buscarTerritorioPorCodigo	lista de territorios, código	territorio o nulo	Busca el territorio con el código dado; valida las selecciones por código.
atacar (combate interactivo)	tropas atacantes y defensoras (por referencia)	booleano	Repite rondas con resolverRondaCombate, muestra dados, duelos y tropas restantes y pregunta si se continúa. Retorna verdadero si el defensor queda en 0.
finalizarFaseAtaque	juego, jugador	ninguna	Marca que el jugador ya atacó, pasa su fase a Fortificar y le entrega la carta si conquistó.
mostrar_ayuda / mostrar_ayuda_comando	(opcional) nombre del comando	ninguna	Muestran la lista de comandos o el uso y la descripción de uno.

3.4 Diseño de las operaciones principales de los TADs

Esta sección describe en lenguaje natural la lógica de las operaciones que hacen el trabajo de los comandos.

calcularUnidadesBase(jugador) – JuegoRisk

1. Contar los territorios del jugador (TableroRisk.cantidadTerritoriosOcupadosPor) y dividir entre 3 (parte entera), como dice el numeral 1.5 del enunciado.
2. Sumar el bono de los continentes que domina por completo (TableroRisk.obtenerBonoPorContinentesDominados) y retornar el total.

canjearCartas(jugador, i1, i2, i3, territoriosBono) – JuegoRisk

1. Vaciar territoriosBono y recorrer las 3 cartas de las posiciones i1, i2 e i3 (los comodines se ignoran). Por cada territorio de las cartas que sea del jugador y no esté ya en territoriosBono, sumar 2 tropas en ese territorio y agregar su código a la lista.
2. Eliminar las 3 cartas de la mano (Jugador.eliminarCartas).
3. Obtener el bono con obtenerBonoSiguienteCanje, que aumenta el contador de grupos canjeados y retorna 4, 6, 8, 10, 12 y 15 para los seis primeros grupos y luego 20, 25, 30… El grupo lo elige Jugador.buscarGrupoCanjeable con esta prioridad: 3 cartas del mismo tipo, una de cada tipo, o un comodín con 2 cartas cualesquiera.

resolverRondaCombate(tropasAtq, tropasDef, dadosAtq, dadosDef) – JuegoRisk

1. El atacante lanza min(3, tropasAtq) dados y el defensor min(2, tropasDef); lanzarDado retorna un entero aleatorio entre 1 y 6.
2. Ordenar los dados de cada lado de mayor a menor y compararlos por pares, tantas veces como dados tenga el lado con menos dados.
3. Si el dado del atacante es estrictamente mayor, el defensor pierde 1 tropa; si es menor o igual, la pierde el atacante.
4. Actualizar tropasAtq y tropasDef y dejar los dados lanzados en dadosAtq y dadosDef para que la consola los muestre (Figura 7).

conquistarTerritorio(jugador, territorio, tropas) y verificarGanadorGlobal() – JuegoRisk

1. Asignar el territorio al jugador con las tropas sobrevivientes (Territorio.asignarOcupante) y marcar la conquista del turno.
2. verificarGanadorGlobal: si la partida está en curso y el jugador en turno ocupa los 42 territorios, pasar a TERMINADO y guardar su nombre como ganador.

3. Retornar verdadero si el jugador ganó.

otorgarCartaPorConquista(jugador) y moverTropas(jugador, origen, destino, cantidad) – JuegoRisk

1. otorgarCartaPorConquista: si el jugador no conquistó en el turno, retornar falso. Si conquistó, apagar la bandera (así recibe máximo una carta por turno), agregar a su mano una carta de un territorio al azar (con baja probabilidad, un comodín) y retornar verdadero.
2. moverTropas: retornar falso sin modificar nada si algún territorio es nulo, no es del jugador, no son vecinos, cantidad ≤ 0 o el origen quedaría con menos de 1 tropa. Si no, restar cantidad al origen, sumarla al destino y retornar verdadero.

inicializarDesdeArchivo(nombre) y cargarDesdeTexto(contenido, validarSumaInicial) – JuegoRisk

1. inicializarDesdeArchivo: verificar que el archivo se pueda abrir y no esté vacío. Si el nombre termina en .bin, intentar decodificarlo (cabecera con ArchivoHuffman.leer, árbol con las mismas frecuencias y w caracteres) y cargarlo con validarSumaInicial falso. Si no termina en .bin o no se pudo decodificar, leerlo como texto y cargarlo con validarSumaInicial verdadero. Si eso falla y el nombre no termina en .bin, intentar como comprimido. Resultado: correcta, archivo erróneo, archivo vacío o sin formato (Figura 4).
2. cargarDesdeTexto: crear un JuegoRisk temporal y, sobre él (cargarEnJuegoVacio), leer N (entre 3 y 6) y los N jugadores (nombre de máximo 8 caracteres; nombres y colores sin repetir).

3. Configurar el mapa base y leer las 42 líneas de territorio, validando código existente y no repetido, color de un jugador y al menos 1 tropa.
4. Verificar que no sobre información.
5. Si validarSumaInicial es verdadero, verificar que cada jugador ocupe al menos un territorio y que la suma de sus unidades sea unidadesInicialesPorJugador(N): 35 − 5 × (N − 3), es decir 35, 30, 25 o 20.
6. Todo lo anterior se hace sobre un JuegoRisk temporal (cargarEnJuegoVacio). Si algo falla, retornar falso: el juego actual no se tocó. Si todo es correcto, el temporal queda EN_CURSO con el turno en el primer jugador que tenga territorios, y cargarDesdeTexto intercambia sus listas con las del juego actual (ListaLineal.intercambiar, sin copiar nodos, así los punteros a vecinos siguen siendo válidos) y copia estado, turno, ganador y canjes. El estado anterior queda en el temporal y su destructor lo libera.

serializarEstado(), guardarEnTexto(nombre) y guardarEnComprimido(nombre) – JuegoRisk

1. serializarEstado: escribir N, una línea “nombre color” por jugador y una línea “código color tropas” por cada territorio, en orden.
2. guardarEnTexto: escribir el resultado de serializarEstado en el archivo y retornar verdadero si no hubo errores.
3. guardarEnComprimido: con el texto de serializarEstado, calcular frecuencias, construir el árbol, generar los códigos, codificar y escribir el .bin con ArchivoHuffman.escribir. Retornar falso si el árbol queda vacío o no se pudo escribir.

Operaciones que encapsulan los cambios de estado de los comandos – JuegoRisk

1. ubicarUnidades(jugador, destino, cantidad, disponibles): retorna falso sin modificar nada si el territorio no es del jugador o la cantidad no está entre 1 y las unidades disponibles; si no, suma la cantidad al territorio.
2. apartarTropasAtacantes(jugador, origen, tropas): retorna falso sin modificar nada si el origen no es del jugador o las tropas no están entre 1 y (tropas del origen − 1); si no, las resta del origen mientras dura el combate. devolverTropasAtacantes(origen, tropas) suma de vuelta las que sobrevivieron cuando no hubo conquista.
3. actualizarDefensor(defensor, tropasRestantes): deja en el territorio defensor las tropas que le quedaron después del combate (el combate trabaja sobre una copia).
4. terminarFaseAtaque(jugador): llama a registrarAtaque (habilita fortificar), cambia la fase a Fortificar y llama a otorgarCartaPorConquista; retorna verdadero si el jugador recibió carta.

Operaciones del árbol de Huffman – ArbolHuffman, ColaPrioridad y ArchivoHuffman

1. construirArbol: liberar el árbol anterior y crear una hoja por cada carácter con frecuencia mayor a 0, en orden ASCII, insertándola en la ColaPrioridad. Mientras la cola tenga más de un nodo, extraer los dos de menor frecuencia y crear un padre con la suma (el primero es el hijo izquierdo y el segundo el derecho). El último nodo es la raíz. Como las hojas entran en orden ASCII y la inserción es estable, las mismas frecuencias producen siempre el mismo árbol; por eso basta guardar las frecuencias en el archivo.

2. generarCodigos: si la raíz es una hoja (un solo carácter), su código es “0”; si no, recorrer el árbol acumulando ‘0’ a la izquierda y ‘1’ a la derecha y guardar el camino al llegar a cada hoja.
3. codificar(texto) reemplaza cada carácter por su código. decodificar(bits, longitud) baja desde la raíz según cada bit y, al llegar a una hoja, agrega su símbolo y vuelve a la raíz hasta completar longitud caracteres; si un bit no tiene camino, retorna el texto obtenido hasta ese punto. Si el árbol tiene un solo carácter, devuelve ese carácter repetido longitud veces.
4. ArchivoHuffman.escribir: escribir n (2 bytes), cada carácter (1 byte) con su frecuencia (8 bytes), w (8 bytes) y los bits empaquetados de 8 en 8; los enteros van con el byte menos significativo primero.
5. ArchivoHuffman.leer: exigir $1 \leq n \leq 256$, frecuencias mayores a 0 sin caracteres repetidos, $w > 0$ y $w = \Sigma fi$; el resto del archivo es el código binario. Si algo no se cumple, retornar falso.

4. Diseño de los TADs

Cada TAD sigue la plantilla de clase: datos mínimos (nombre, tipo en lenguaje natural y qué representa) y operaciones (entradas, salida y descripción). Solo listamos operaciones que existen en el código y con el mismo nombre que tienen allí (por ejemplo, calcularUnidadesBase). En ListaLineal los datos son privados y se manejan con sus operaciones; en los demás TADs son atributos públicos, por eso no incluimos operaciones Obtener/Fijar. Las operaciones Crear… corresponden a los constructores.

4.1 TADs de estructuras lineales

TAD Nodo

Elemento básico de la lista enlazada: guarda un dato genérico y la referencia al siguiente nodo.

Tabla 13

TAD Nodo: datos mínimos y operaciones

Datos mínimos			
Dato	**Tipo**	**Represen ta**	
valorAlmacenado	T (genérico)	Dato guardado en el nodo.	
siguienteNodo	referencia a Nodo	Nodo siguiente; es nulo en el último y por defecto.	
Operaciones			
Operación	**Entradas**	**Salida**	**Descripción**
CrearNodo(valor)	valor de tipo T	Nodo	Crea un nodo con el valor dado y sin siguiente.

TAD ListaLineal

Lista simplemente enlazada genérica de tamaño dinámico. Es la estructura lineal de todo el modelo: jugadores, territorios, continentes, vecinos, cartas y la cola de prioridad.

Tabla 14

TAD ListaLineal: datos mínimos y operaciones

Datos mínimos			
Dato	**Tipo**	**Representa**	
nodoInicial	referencia a Nodo	Primer nodo de la lista.	
nodoFinal	referencia a Nodo	Último nodo; permite insertar al final en tiempo constante.	
cantidadElementos	entero	Cantidad de elementos guardados.	
Operaciones			
Operación	**Entradas**	**Salida**	**Descripción**
CrearLista()	—	ListaLineal	Crea una lista vacía.
estaVacia()	—	booleano	Indica si la lista no tiene elementos. O(1).
longitud()	—	entero	Retorna la cantidad de elementos. O(1).
agregarAlFinal(valor)	valor de tipo T	—	Inserta el valor al final. O(1).
insertarOrdenado(valor, menorQue)	valor de tipo T, criterio de comparación	—	Inserta manteniendo el orden ascendente; ante elementos iguales queda después de ellos (inserción estable). La usa la cola de prioridad. O(N).
obtener(indice)	entero	referencia al elemento	Retorna el elemento de esa posición para leerlo o modificarlo; si la posición no existe lanza la excepción out_of_range. O(N).
eliminarEn(indice)	entero	booleano	Elimina el elemento de esa posición y libera su memoria; retorna falso si la posición no existe. O(N).
limpiar()	—	—	Elimina todos los nodos; también se ejecuta al destruir la lista.
CopiarLista(otra) / asignar(otra)	ListaLineal	ListaLineal	Constructor de copia y operador =: crean nodos nuevos con copias de los valores (copia profunda), así dos listas nunca comparten nodos. La asignación primero libera los nodos propios y no hace nada si se asigna la lista a sí misma.
intercambiar(otra)	ListaLineal	—	Intercambia el contenido con otra lista en O(1): cambia los apuntadores inicial y final y la cantidad, sin crear ni liberar nodos.

4.2 TADs del dominio del juego (Componente 1)

TAD Carta

Carta de territorio o comodín, con el tipo de tropa dibujado.

Tabla 15

TAD Carta: datos mínimos y operaciones

Datos mínimos			
Dato	**Tipo**	**Represen ta**	
idAsociado	cadena de caracteres	Código del territorio de la carta, o “COMODIN”.	
clasificacion	TipoTropa (Infantería, Caballería, Artillería, Comodín)	Tipo de tropa de la carta.	
Operaciones			
Operación	**Entradas**	**Salida**	**Descripción**
CrearCarta(codigo, tipo)	cadena, TipoTropa	Carta	Crea la carta; por defecto el código es vacío y el tipo es Infantería.

TAD Territorio

Una de las 42 casillas del tablero, con su ocupante, sus tropas y sus vecinos.

Tabla 16

TAD Territorio: datos mínimos y operaciones

Datos mínimos			
Dato	**Tipo**	**Represent a**	
idTerritorio	cadena de caracteres	Código oficial (por ejemplo “1.1” o “5.6”).	
nombreTerritorio	cadena de caracteres	Nombre oficial (por ejemplo “Alaska”).	
idContinente	entero	Continente al que pertenece (1 a 6).	
colorPropietario	cadena de caracteres	Color del jugador que lo ocupa.	
cantidadTropas	entero	Unidades ubicadas en el territorio.	
adyacentes	lista de referencias a Territorio	Vecinos; se guardan como referencias para no duplicar territorios.	
Operaciones			
Operación	**Entradas**	**Salida**	**Descripción**
CrearTerritorio(id, nombre, idContinente)	cadena, cadena, entero	Territorio	Crea el territorio sin ocupante, con 0 tropas y sin vecinos.
asignarOcupante(color, cantidad)	cadena, entero	—	Asigna el ocupante y la cantidad de tropas.
modificarEjercitos(delta)	entero	—	Suma tropas (o resta, si delta es negativo).
agregarVecino(vecino)	referencia a Territorio	booleano	Agrega un vecino; retorna falso si es nulo, es el mismo territorio o ya estaba.
esVecinoDe(codigo)	cadena	booleano	Indica si tiene frontera con ese territorio (falso si el código es vacío o el propio).
tieneVecinoEnemigo(color)	cadena	booleano	Indica si algún vecino es de un color distinto al dado.
tieneVecinoPropio(color)	cadena	booleano	Indica si algún vecino es del color dado.

TAD Continente

Agrupa territorios y define el bono por controlarlos todos.

Tabla 17

TAD Continente: datos mínimos y operaciones

Datos mínimos			
Dato	**Tipo**	**Representa**	
idContinente	entero	1 Norteamérica, 2 Sudamérica, 3 Europa, 4 África, 5 Asia, 6 Australia.	
nombreContinente	cadena de caracteres	Nombre oficial.	
tropasAdicionales	entero	Bono por dominar el continente (2, 3, 5 o 7).	
territoriosInternos	lista de referencias a Territorio	Territorios del continente.	
Operaciones			
Operación	**Entradas**	**Salida**	**Descripción**
CrearContinente(id, nombre, bono)	entero, cadena, entero	Continente	Crea el continente sin territorios.
agregarTerritorio(territorio)	referencia a Territorio	—	Agrega el territorio si no es nulo ni estaba en la lista.
esControladoTotalmentePor(c olor)	cadena	booleano	Indica si todos sus territorios son de ese color (falso si el continente o el color están vacíos).

TAD Jugador

Participante de la partida y avance de su turno.

Tabla 18

TAD Jugador: datos mínimos y operaciones

Datos mínimos			
Dato	**Tipo**	**Represent a**	
nombreJugador	cadena de caracteres	Nombre (máximo 8 caracteres, sin espacios).	
colorFicha	cadena de caracteres	Color que identifica sus territorios.	
manoCartas	lista de Carta	Cartas obtenidas (1 por turno si conquistó).	
faseTurnoActual	FaseTurno (ObtenerUnidades, Atacar, Fortificar, Finalizada)	Fase del turno en la que está.	
logroConquistaTurnoActual	booleano	Indica si conquistó al menos un territorio en el turno.	
haAtacadoTurnoActual	booleano	Indica si ya pasó por la fase de ataque (habilita fortificar).	
Operaciones			
Operación	**Entradas**	**Salida**	**Descripción**
CrearJugador(nombre, color)	cadena, cadena	Jugador	Crea el jugador en ObtenerUnidades, con las banderas en falso y sin cartas.
reiniciarFaseTurno()	—	—	Vuelve a ObtenerUnidades y apaga las banderas de conquista y ataque.
avanzarFase(fase)	FaseTurno	—	Cambia la fase del turno.
agregarCarta(carta)	Carta	—	Agrega una carta al final de la mano.
buscarGrupoCanjeable(i1, i2, i3)	tres enteros (salida)	booleano	Busca 3 cartas canjeables (3 del mismo tipo, una de cada tipo o un comodín con 2 cualesquiera) y deja sus posiciones en i1, i2 e i3.
registrarAtaque()	—	—	Marca que el jugador ya pasó por la fase de ataque en el turno (habilita fortificar).

eliminarCartas(i1, i2, i3)	tres enteros	—	Elimina esas 3 cartas, de mayor a menor posición para no desplazar las siguientes.

TAD TableroRisk

Contiene el mapa: los 42 territorios, los 6 continentes y las fronteras.

Tabla 19

TAD TableroRisk: datos mínimos y operaciones

Datos mínimos			
Dato	**Tipo**	**Representa**	
listaTerritoriosGlob	lista de Territorio	Los 42 territorios, en orden de código.	
listaContinentesGlob	lista de Continente	Los 6 continentes.	
Operaciones			
Operación	**Entradas**	**Salida**	**Descripción**
configurarMapaBase()	—	—	Crea los continentes y territorios, los asocia y crea todas las fronteras oficiales.
establecerFrontera(cod1, cod2)	cadena, cadena	—	Conecta dos territorios en ambos sentidos.
obtenerTerritorioPorId(codigo)	cadena	referencia a Territorio o nulo	Busca un territorio por su código. O(N).
cantidadTerritoriosOcupadosPor(color)	cadena	entero	Cuenta los territorios de un jugador.
obtenerBonoPorContinentesDominados(color)	cadena	entero	Suma los bonos de los continentes que el jugador domina.
territoriosDeJugador(color, resultado)	cadena, vector de referencias a Territorio (salida)	—	Llena el vector con todos los territorios del jugador, en el orden del tablero.
territoriosAptosParaAtacar(color, resultado)	cadena, vector de referencias a Territorio (salida)	—	Territorios del jugador con más de 1 tropa y al menos un vecino enemigo.
territoriosAptosParaFortificar(color, resultado)	cadena, vector de referencias a Territorio (salida)	—	Territorios del jugador con más de 1 tropa y al menos un vecino propio.
vaciarTablero()	—	—	Libera los territorios y los continentes.

intercambiar(otro)	TableroRisk	—	Intercambia territorios y continentes con otro tablero en O(1). Un tablero no se puede copiar (constructor de copia privado), porque la copia compartiría punteros a los territorios del original.

TAD JuegoRisk

Controla la partida: estado, jugadores, turno, tablero, reglas de los comandos y carga y guardado.

Tabla 20

TAD JuegoRisk: datos mínimos y operaciones

Datos mínimos		
Dato	**Tipo**	**Representa**
estadoJuego	EstadoJuego (NoInicializado, EnCurso, Terminado)	Estado de la partida.
listaJugadores	lista de Jugador	Participantes (3 a 6).
turnoActualIndex	entero	Posición del jugador en turno.
tableroJuego	TableroRisk	Mapa de la partida.
nombreGanador	cadena de caracteres	Ganador, si la partida terminó.
gruposCanjeados	entero	Grupos de cartas canjeados en la partida (define el bono del siguiente canje).

Operaciones			
Operación	**Entradas**	**Salida**	**Descripción**
CrearJuego()	—	JuegoRisk	Crea el juego en NoInicializado.
estaInicializado()	—	booleano	Indica si la partida está en curso.
estaTerminado()	—	booleano	Indica si ya hay ganador.
obtenerJugadorEnTurno()	—	referencia a Jugador o nulo	Jugador en turno (nulo si el juego no está en curso).
buscarJugadorPorNombre(nombre)	cadena	referencia a Jugador o nulo	Busca un jugador por su nombre.
pasarAlSiguienteTurno()	—	—	turno = (turno + 1) mod N, saltando a quienes no tienen territorios; reinicia la fase del nuevo jugador.
verificarGanadorGlobal()	—	booleano	Si el jugador en turno ocupa los 42 territorios, termina la partida y registra al ganador.

Operaciones

Operación	Entradas	Salida	Descripción
unidadesInicialesPorJugador(N)	entero	entero	Unidades iniciales de cada jugador: 35, 30, 25 o 20 para 3, 4, 5 o 6 jugadores (0 si N no es válido).
calcularUnidadesBase(jugador)	Jugador	entero	territorios ÷ 3 (parte entera) + bono de continentes dominados.
obtenerBonoSiguienteCanje()	—	entero	Aumenta gruposCanjeados y retorna 4, 6, 8, 10, 12, 15 y luego de 5 en 5.
canjearCartas(jugador, i1, i2, i3, territoriosBono)	Jugador, tres enteros, lista de cadenas (salida)	entero	Suma 2 tropas en cada territorio propio de las cartas (deja sus códigos en territoriosBono), elimina las cartas y retorna el bono.
lanzarDado()	—	entero	Entero aleatorio entre 1 y 6; requiere la semilla inicializada al arrancar.
resolverRondaCombate(tropas Atq, tropasDef, dadosAtq, dadosDef)	dos enteros (entrada y salida), dos vectores de enteros (salida)	—	Resuelve una ronda con las reglas de los dados y deja los dados lanzados, ordenados de mayor a menor, en los vectores.
conquistarTerritorio(jugador, territorio, tropas)	Jugador, Territorio, entero	booleano	Asigna el territorio, marca la conquista y verifica la victoria (verdadero si ganó).
otorgarCartaPorConquista(jug ador)	Jugador	booleano	Entrega 1 carta si conquistó en el turno (máximo una) y apaga la bandera.
moverTropas(jugador, origen, destino, cantidad)	Jugador, dos Territorio, entero	booleano	Mueve tropas entre territorios propios vecinos dejando al menos 1 en el origen.
ubicarUnidades(jugador, destino, cantidad, disponibles)	Jugador, Territorio, dos enteros	booleano	Suma refuerzos a un territorio propio si $1 \leq$ cantidad $\leq$ disponibles.
apartarTropasAtacantes(jugad or, origen, tropas)	Jugador, Territorio, entero	booleano	Resta del origen las tropas atacantes si $1 \leq$ tropas $\leq$ tropas del origen $- 1$; si no, no modifica nada.
devolverTropasAtacantes (orig en, tropas)	Territorio, entero	—	Devuelve al origen las tropas atacantes que sobrevivieron.
actualizarDefensor(defensor, tropasRestantes)	Territorio, entero	—	Deja en el defensor las tropas que le quedaron tras el combate.

terminarFaseAtaque(jugador)	Jugador	booleano	Registra el ataque, pasa a la fase Fortificar y entrega la carta si hubo conquista (verdadero si la entregó).
cargarDesdeTexto(contenido, validarSumaInicial)	cadena, booleano	booleano	Configura la partida desde un texto con el formato de inicio. Con validarSumaInicial también exige la suma inicial de unidades y que cada jugador tenga territorios. Si el texto es inválido, el juego queda como estaba.

Operaciones			
Operación	**Entradas**	**Salida**	**Descripción**
inicializarDesdeArchivo(nomb re)	cadena	ResultadoInicial izacion (Correcta, ArchivoErróneo, ArchivoVacío, SinFormato)	Lee un archivo de texto o uno comprimido (por la extensión y, si falla, por el contenido) y llama a cargarDesdeTexto; la suma inicial solo se exige al texto plano.
serializarEstado()	—	cadena	Genera el texto del estado con el formato de inicio.
guardarEnTexto(nombre)	cadena	booleano	Escribe serializarEstado() en un archivo de texto (guardar).
guardarEnComprimido(nombr e)	cadena	booleano	Codifica serializarEstado() con Huffman y lo escribe en binario (guardar_comprimido).

4.3 TADs de estructuras jerárquicas (Componente 2)

TAD NodoHuffman

Nodo del árbol de Huffman. Las hojas guardan un símbolo; los nodos internos, la suma de las frecuencias de sus hijos.

Tabla 21

TAD NodoHuffman: datos mínimos y operaciones

Datos mínimos			
Dato	**Tipo**	**Representa**	
simbolo	carácter (1 byte)	Carácter ASCII; solo tiene sentido en las hojas.	
frecuencia	entero largo sin signo	Apariciones del símbolo o suma de los hijos.	
hijoIzq	referencia a NodoHuffman	Hijo izquierdo (arista 0).	
hijoDer	referencia a NodoHuffman	Hijo derecho (arista 1).	
Operaciones			
Operación	**Entradas**	**Salida**	**Descripción**
CrearNodo(simbolo, frecuencia, izq, der)	carácter, entero, referencias (opcionales)	NodoHuffman	Crea una hoja o un nodo interno.
esHoja()	—	booleano	Indica si el nodo no tiene hijos.

TAD ColaPrioridad

Cola de nodos de Huffman ordenada de menor a mayor frecuencia, implementada con una ListaLineal ordenada.

Tabla 22

TAD ColaPrioridad: datos mínimos y operaciones

Datos mínimos			
Dato	**Tipo**	**Representa**	
elementos	lista de referencias a NodoHuffman	Nodos pendientes por combinar; el primero es el de menor frecuencia.	
Operaciones			
Operación	**Entradas**	**Salida**	**Descripción**
CrearCola()	—	ColaPrioridad	Crea una cola vacía.
esVacia()	—	booleano	Indica si la cola no tiene elementos.
tamano()	—	entero	Cantidad de nodos.
insertar(nodo)	referencia a NodoHuffman	—	Inserta según la frecuencia, de forma estable (insertarOrdenado). O(N).
extraerMinimo()	—	referencia a NodoHuffman o nulo	Saca y retorna el nodo de menor frecuencia; nulo si la cola está vacía.

TAD ArbolHuffman

Árbol binario que asigna a cada carácter un código de longitud variable; los más frecuentes reciben códigos más cortos.

Tabla 23

TAD ArbolHuffman: datos mínimos y operaciones

Datos mínimos			
Dato	**Tipo**	**Representa**	
raiz	referencia a NodoHuffman	Raíz; es nula si el árbol está vacío.	
tablaFrecuencias	arreglo de 256 enteros largos sin signo	Frecuencia de cada carácter ASCII.	
tablaCodigos	arreglo de 256 cadenas	Código de bits de cada carácter.	
Operaciones			
Operación	**Entradas**	**Salida**	**Descripción**
CrearArbol()	—	ArbolHuffman	Crea un árbol vacío con todas las frecuencias en 0.
esVacio()	—	booleano	Indica si el árbol no tiene nodos.
calcularFrecuencias(texto)	cadena	—	Cuenta las apariciones de cada carácter.
fijarFrecuencias(frecuencias)	arreglo de 256 enteros	—	Asigna las frecuencias leídas de un .bin.
cantidadSimbolos()	—	entero	Caracteres con frecuencia mayor a 0 (el n del archivo).
construirArbol()	—	—	Construye el árbol con una ColaPrioridad (sección 3.4).
generarCodigos() / generarCodigos(nodo, prefijo)	— / NodoHuffman, cadena	—	Llena tablaCodigos con un recorrido en preorden (izquierda ‘0’, derecha ‘1’); con un solo carácter, su código es “0”.
codificar(texto)	cadena	cadena de bits	Reemplaza cada carácter por su código.
decodificar(bits, longitud)	cadena de bits, entero	cadena	Recorre el árbol bit a bit hasta completar la longitud.
destruir() / destruir(nodo)	— / NodoHuffman	—	Libera los nodos en posorden.

TAD ArchivoHuffman

Lee y escribe el archivo comprimido con la estructura del enunciado: n c1 f1 … cn fn w binary_code.

Tabla 24

TAD ArchivoHuffman: datos mínimos y operaciones

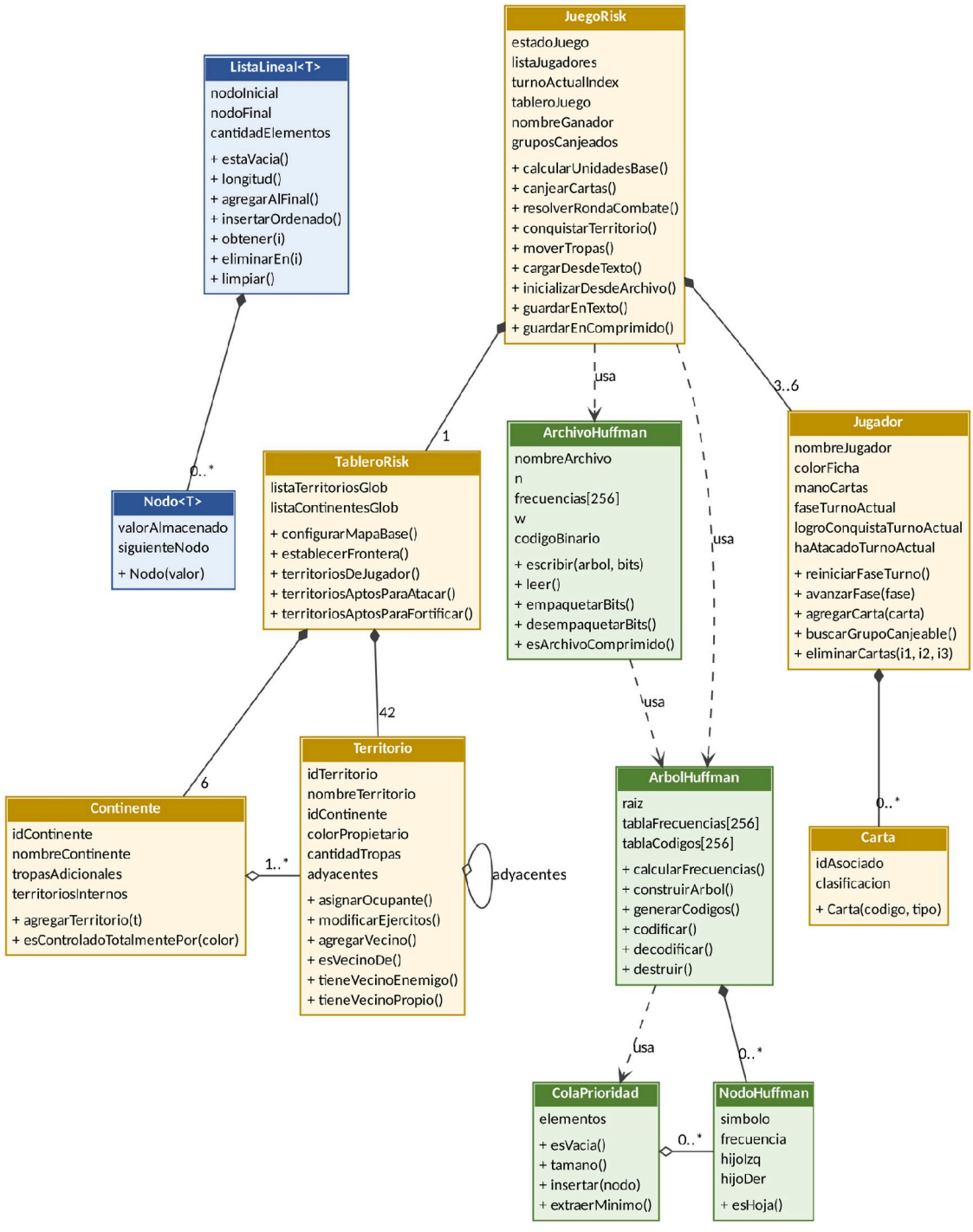
Datos mínimos			
Dato	**Tipo**	**Representa**	
nombreArchivo	cadena de caracteres	Ruta del archivo .bin.	
n	entero corto sin signo (2 bytes)	Cantidad de caracteres distintos.	
frecuencias	arreglo de 256 enteros largos sin signo (8 bytes)	Frecuencia fi de cada carácter ci; la posición es el código ASCII.	
w	entero largo sin signo (8 bytes)	Longitud del texto original.	
codigoBinario	secuencia de bytes	Bits codificados, empaquetados de 8 en 8.	
Operaciones			
Operación	**Entradas**	**Salida**	**Descripción**
CrearArchivo(nombre)	cadena	ArchivoHuffman	Guarda la ruta e inicializa n, w y las frecuencias en 0.
escribir(arbol, bits)	ArbolHuffman, cadena de bits	booleano	Escribe n, los pares (ci, fi), w y el código; verdadero si no hubo errores.
leer()	—	booleano	Lee cabecera y código y valida $1 \leq n \leq 256$, sin repetidos, $fi > 0$ y $w = \Sigma fi$.
empaquetarBits(bits)	cadena de bits	secuencia de bytes	Agrupa los bits de 8 en 8 (del más al menos significativo) y completa el último byte con ceros.
desempaquetarBits()	—	cadena de bits	Convierte los bytes leídos en una cadena de bits.
esArchivoComprimido(nombre)	cadena	booleano	Indica si el nombre termina en .bin (sin distinguir mayúsculas).

5. Diagrama de relación entre TADs y esquemáticos de Huffman

La Figura 11 muestra los 12 TADs con sus datos y operaciones principales. El rombo lleno es composición (el contenedor crea y libera lo que contiene), el rombo vacío es agregación por referencia (solo se guardan apuntadores a objetos que pertenecen a otro TAD) y la línea punteada indica que un TAD usa a otro. Los colores separan las estructuras lineales (azul), el dominio del juego (amarillo) y la compresión (verde). Todas las colecciones usan ListaLineal, pero no dibujamos una flecha desde cada TAD hacia ella porque el diagrama se volvería ilegible.

Figura 11

Diagrama de relación entre los TADs del sistema



Nota. La composición entre ArbolHuffman y NodoHuffman corresponde a los nodos que cuelgan de la raíz del árbol.

5.1 Árbol de Huffman de una partida real

La Figura 12 es el árbol que construye el programa cuando comprime la partida inicial de archivo_inicio.txt (3 jugadores, 514 caracteres y 28 caracteres distintos). Cada hoja muestra su carácter (␣ es el espacio y ↵ el salto de línea) y su frecuencia; cada nodo interno, la suma de sus hijos. El código de un carácter es el camino desde la raíz: el espacio, que aparece 87 veces, queda como 111, mientras que la J, que aparece una sola vez, queda como 00100101. Para comprobarlo, reconstruimos el árbol solo con las frecuencias guardadas en el .bin y, al decodificar, obtuvimos exactamente el texto original.

Figura 12

Árbol de Huffman de la partida inicial (514 caracteres, 28 símbolos)

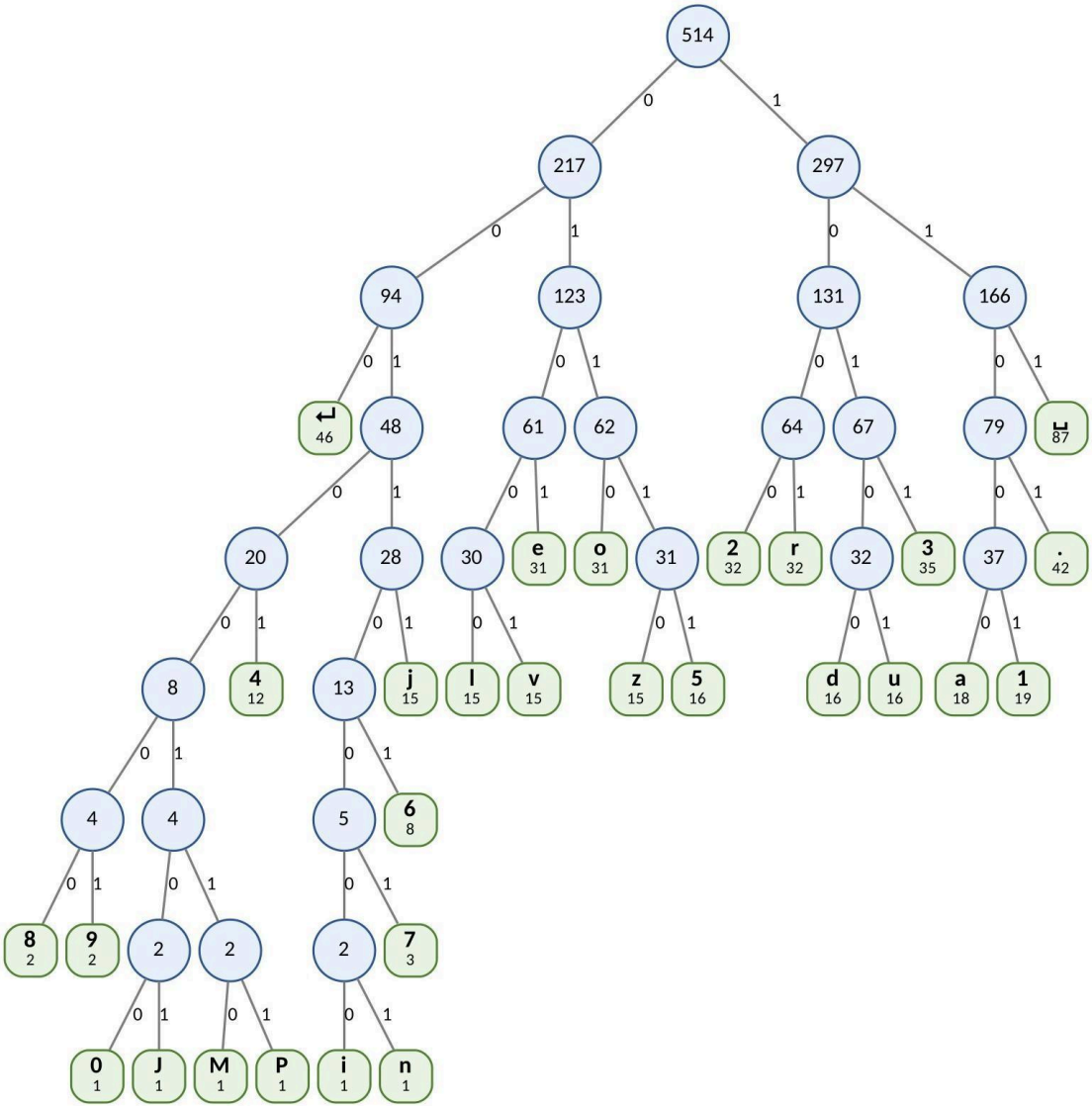


Tabla 25

Códigos generados por generarCodigos() para la partida inicial

Carácter	Frec.	Código	Carácter	Frec.	Código
espacio	87	111	l	15	01000
salto de línea	46	000	v	15	01001
.	42	1101	z	15	01110
3	35	1011	4	12	00101
2	32	1000	6	8	001101
r	32	1001	7	3	0011001
e	31	0101	8	2	0010000
o	31	0110	9	2	0010001
1	19	11001	0	1	00100100
a	18	11000	J	1	00100101
5	16	01111	M	1	00100110
d	16	10100	P	1	00100111
u	16	10101	i	1	00110000
j	15	00111	n	1	00110001

Nota. Con estos códigos el texto ocupa 2141 bits, es decir, 268 bytes. Codificamos p3.txt con esta tabla y el resultado coincidió byte a byte con el código guardado por el programa en p3.bin.

5.2 Esquemático del archivo binario (.bin)

La Figura 13 muestra cómo queda el archivo que genera guardar_comprimido para esa misma partida. Los bytes en hexadecimal se tomaron del archivo real del caso G-05.

Figura 13

Estructura del archivo binario n c1 f1 … cn fn w binary_code

Campo	Tamaño	Bytes (hexadecimal)	Valor
n	2 bytes	`1C 00`	n = 28 caracteres distintos
c1	1 byte	`0A`	'\n' (salto de línea)
f1	8 bytes	`2E 00 00 00 00 00 00 00`	46
c2	1 byte	`20`	' ' (espacio)
f2	8 bytes	`57 00 00 00 00 00 00 00`	87
c3 f3 … cn fn	26 × 9 bytes	…	26 pares más
w	8 bytes	`02 02 00 00 00 00 00 00`	w = 514 caracteres
binary_code	268 bytes	`B0 4B 5C 18 F9 …`	bits de 8 en 8; el último byte se completa con ceros

Cabecera: 2 + 9 × 28 + 8 = 262 bytes **Código:** 268 bytes **Total del archivo:** 262 + 268 = 530 bytes

Nota. Los enteros se escriben con el byte menos significativo primero.

El archivo comprimido (530 bytes) quedó más grande que el texto original (514 bytes), y es lo esperado. Huffman sí reduce el contenido: los 514 caracteres pasan a 2141 bits, es decir, 268 bytes (52,1 % del texto). Pero para poder decodificar hay que guardar la tabla de frecuencias, y el formato del enunciado usa 9 bytes por cada carácter distinto (1 del carácter y 8 de la frecuencia). Con 28 caracteres distintos, esa cabecera ocupa 262 bytes y se come lo que se ahorró. Huffman no garantiza un archivo más pequeño cuando el texto es corto. La cabecera depende del número de caracteres distintos y no de la longitud del texto, así que en un texto más largo con los mismos caracteres pesaría proporcionalmente menos. El caso G-16 verifica justamente que el código (268 bytes) sea menor que el texto.

6. Plan de pruebas

Presentamos dos planes: el del comando atacar, que se pidió en la Entrega 1, y el de guardar_comprimido, que se pide en esta entrega. También volvimos a ejecutar los casos de consola T01 a T08 de la Entrega 1, ahora con su resultado obtenido, y agregamos T09 a T11, las pruebas unitarias de los TADs y pruebas de robustez con entradas extremas.

6.1 Metodología

- **Tipo y técnica:** pruebas funcionales de caja negra sobre la consola, con partición en clases de equivalencia y valores límite (por ejemplo 0, 1 y el máximo de tropas permitido).
- **Ambiente:** las pruebas de la versión final se ejecutaron en Linux con g++ 13.3; el código compila sin advertencias con -Wall -Wextra en los modos -std=c++98, -std=c++11 y -std=c++17. El ejecutable de Windows se compiló desde los mismos fuentes con MinGW-w64 13.2 y se ejecutó con Wine (no en un Windows real): T01 a T07 y T09 a T11 dieron la misma salida que en Linux y el .bin de la partida inicial es idéntico byte a byte. El guion pruebas_doc.py abre el programa, escribe las entradas y guarda la salida real; el estado de cada caso sale de comparar esa salida con la esperada.
- **Datos base:** archivo_inicio.txt, con 3 jugadores (Juan rojo, Pedro azul y Maria verde) y los 42 territorios. Juan ocupa América del Norte, América del Sur y Gran Bretaña (14 territorios y 35 unidades), por lo que recibe 11 unidades en su primer turno (14 ÷ 3 = 4, más 5 y 2 por los continentes). El contenido completo del archivo está en el Apéndice D.
- **Evidencia y regresión:** todas las tablas y el Apéndice B corresponden a la ejecución del código final. El guion tests/ejecutar_todo.sh compila y ejecuta todas las pruebas de una vez; la salida de la última ejecución está en docs/evidencias/ejecucion_completa.log.

Después de las correcciones, las salidas de T01 a T07 quedaron idénticas a las de la versión anterior. Las pruebas unitarias se ejecutaron con AddressSanitizer y con Valgrind, y las de robustez con AddressSanitizer, sin errores ni fugas de memoria.

- **Dados:** como son aleatorios, en los casos con lanzamientos no esperamos valores fijos sino propiedades que siempre se cumplen: cuántos dados se lanzan, cuántas tropas se pierden y qué mensajes aparecen.
- **Criterio de aceptación:** un caso se aprueba si el mensaje coincide exactamente con el del enunciado, el tablero queda como se espera y la aplicación sigue funcionando.

6.2 Plan de pruebas del comando atacar

Con este plan comprobamos que atacar valida el estado del juego y del turno, muestra todos los territorios del jugador con su código, solo deja escoger origen y destino por código, aplica bien las reglas de los dados y actualiza las tropas y el dueño de cada territorio.

Tabla 26

Clases de equivalencia del comando atacar

Dato de entrada	Clases válidas	Clases inválidas
Estado del juego	Inicializado y en curso	No inicializado; terminado
Parámetros	Exactamente 1 (nombre)	0 parámetros; 2 o más
nombre_jugador	Jugador de la partida, con el turno y que ya obtuvo unidades	No existe; no tiene el turno; no ha ejecutado obtener_unidades; ya terminó el ataque
Código de origen	Territorio propio con más de 1 tropa y un vecino enemigo; 0 para terminar	Código inexistente; territorio propio sin vecinos enemigos; territorio con 1 tropa; territorio ajeno
Tropas atacantes	$1 \leq t \leq$ tropas del origen $- 1$	$t \leq 0$; $t \geq$ tropas del origen (límites 0 y 14 con 14 tropas); texto no numérico
Código de destino	Vecino del origen ocupado por otro jugador	Código inexistente; territorio propio; enemigo que no es vecino
Continuar (s/n)	s o S continúa; cualquier otra respuesta detiene el ataque	—

Tabla 27

Casos de prueba del comando atacar

ID	Objetivo y precondición	Entrada	Resultado esperado	Resultado obtenido	Estado
A-01	No se ejecuta si el juego no está inicializado. *Precondición:* juego recién abierto.	atacar Juan	(Juego no inicializado) Esta partida no ha sido inicializada correctamente.	Igual al esperado. *Evidencia:* Apéndice B.	**Aprobado**
A-02	Valida la cantidad de parámetros (faltan). *Precondición:* juego inicializado.	atacar	Error: Numero de parametros incorrecto. y la ayuda del comando.	Igual al esperado. *Evidencia:* Apéndice B.	**Aprobado**
A-03	Valida la cantidad de parámetros (sobran). *Precondición:* juego inicializado.	atacar Juan extra	Error: Numero de parametros incorrecto. y la ayuda del comando.	Igual al esperado. *Evidencia:* Apéndice B.	**Aprobado**
A-04	El jugador debe formar parte de la partida. *Precondición:* Pepe no es jugador.	atacar Pepe	(Jugador no válido) El jugador Pepe no forma parte de esta partida.	Igual al esperado. *Evidencia:* Apéndice B.	**Aprobado**
A-05	El jugador debe tener el turno. *Precondición:* turno de Juan.	atacar Pedro	(Jugador fuera de turno) No es el turno del jugador Pedro.	Igual al esperado. *Evidencia:* Apéndice B.	**Aprobado**
A-06	No se puede atacar sin ejecutar obtener_unidades. *Precondición:* Juan en fase obtener unidades.	atacar Juan	(Jugador no ha ubicado unidades) El jugador Juan no ha ejecutado el comando obtener_unidades.	Igual al esperado. *Evidencia:* Apéndice B.	**Aprobado**
A-07	No se puede fortificar sin pasar por atacar. *Precondición:* Juan no ha atacado.	fortificar Juan	(Jugador no ha atacado) El jugador Juan no ha ejecutado el comando atacar.	Igual al esperado. *Evidencia:* Apéndice B.	**Aprobado**
A-08	Se muestran todos los territorios con su código; los no aptos, con el motivo. *Precondición:* Juan ubicó sus 11 unidades en 1.1 (14 tropas).	atacar Juan	Los 14 territorios con código, nombre y tropas; los no aptos marcados [No disponible: …]; opción 0.	Igual al esperado. *Evidencia:* Apéndice B; Figuras C2, C3.	**Aprobado**

A-09	Un código de origen inexistente se rechaza. *Precondición:* lista de origen visible.	9.9	Error: Codigo invalido o territorio no disponible para atacar; se vuelve a pedir.	Igual al esperado. *Evidencia:* Apéndice B.	**Aprobado**
A-10	Un territorio propio sin vecinos enemigos no puede ser origen. *Precondición:* 1.4 es de Juan y todos sus vecinos también.	1.4	El mismo error de A-09; se vuelve a pedir.	Igual al esperado. *Evidencia:* Apéndice B; Figuras C3.	**Aprobado**

ID	Objetivo y precondición	Entrada	Resultado esperado	Resultado obtenido	Estado
A-11	Tropas atacantes = 0 (límite inferior inválido). *Precondición:* origen 1.1 con 14 tropas (máximo 13).	1.1; 0	Error: Opcion invalida, sin el mensaje (Comando correcto). Las tropas no cambian y el jugador sigue en la fase de ataque.	Igual al esperado. *Evidencia:* Apéndice B.	**Aprobado**
A-12	Tropas atacantes = tropas del origen (límite superior inválido). *Precondición:* origen 1.1 con 14 tropas.	atacar Juan; 1.1; 14	Error: Opcion invalida, sin el mensaje (Comando correcto). La lista vuelve a mostrar 1.1 con 14 tropas.	Igual al esperado. *Evidencia:* Apéndice B.	**Aprobado**
A-13	El destino debe ser un vecino enemigo; un lanzamiento y el atacante se detiene. *Precondición:* 13 tropas atacantes desde 1.1; 5.6 Kamchatka tiene 3.	atacar Juan; 1.1; 13; 1.2; 5.6; n	1.2 se rechaza (Error: Codigo invalido). Con 5.6: 3 dados contra 2 y se pierden 2 tropas en total; con n, el defensor conserva el territorio.	Igual al esperado (atacante 13, defensor 1). *Evidencia:* Apéndice B; Figuras C3.	**Aprobado**
A-14	El atacante continúa hasta conquistar y el tablero se actualiza. *Precondición:* ataque desde 1.1 contra 5.6 con el máximo de tropas; 5.6 quedó con 1 tropa después de A-13.	atacar Juan; 1.1; 13; 5.6; s…; estado_juego	Mensaje de conquista; estado_juego muestra 5.6 de rojo con las tropas sobrevivientes.	Igual al esperado (13 tropas, conquista en el primer lanzamiento). *Evidencia:* Apéndice B; Figuras C4, C6.	**Aprobado**
A-15	La opción 0 termina el ataque y pasa a fortificar. *Precondición:* Juan acaba de conquistar Kamchatka.	atacar Juan; 0	(Comando correcto) El jugador Juan ha terminado de atacar. y aviso de carta recibida.	Igual al esperado. *Evidencia:* Apéndice B.	**Aprobado**
A-16	Después de terminar el ataque ya no se puede atacar. *Precondición:* Juan en	atacar Juan	(Jugador ya fortifico) El jugador Juan ya no puede atacar.	Igual al esperado. *Evidenci*	**Aprobado**

	fase fortificar.			*a:* Apéndice B.	
A-17	Tras atacar, fortificar es aceptado y el turno pasa al siguiente. *Precondición:* Juan terminó de atacar.	fortificar Juan; n	(Comando correcto) El jugador Juan ha terminado de fortificar su posición. y turno de Pedro.	Igual al esperado. *Evidenci a:* Apéndice B; Figuras C5, C6.	**Aprobad o**
A-18	Sin territorios aptos, se informa y pasa a fortificar. *Precondición:* inicio_sin_ataque.txt: territorios de Juan con 1 tropa salvo 1.4 (22, sin vecinos enemigos; suma 35); sus 11 unidades en 1.4.	atacar Juan	(Jugador no puede atacar) Juan no tiene territorios con mas de 1 tropa y vecinos enemigos para atacar.	Igual al esperado. *Evidenci a:* Apéndice B.	**Aprobad o**

Nota. 18 de 18 casos aprobados. En el Apéndice B la salida de consola de cada caso aparece con su mismo identificador; las figuras C1 a C10 son las capturas del Apéndice C.

6.3 Plan de pruebas del comando guardar_comprimido

Con este plan comprobamos que guardar_comprimido valida el estado y los parámetros; que el archivo .bin tiene la estructura del enunciado (n en 2 bytes, pares ci y fi de 1 y 8 bytes, w en 8 bytes y el código empaquetado); que su contenido equivale al de guardar, y que al cargarlo con inicializar se recupera la misma partida. También probamos que inicializar rechace archivos .bin vacíos, inexistentes o dañados.

Tabla 28

Clases de equivalencia del comando guardar_comprimido

Dato de entrada	Clases válidas	Clases inválidas
Estado del juego	Inicializado	No inicializado
Parámetros	Exactamente 1 (nombre de archivo)	0 parámetros; 2 o más
nombre_archivo	Ruta que se puede escribir	Ruta en una carpeta inexistente o sin permisos
Archivo .bin al cargarlo	Generado por guardar_comprimido (partida inicial o con un turno jugado)	Vacío; inexistente; cabecera truncada; código truncado; w distinto de Σ fi; n = 0

Tabla 29

Casos de prueba del comando guardar_comprimido

ID	Objetivo y precondición	Entrada	Resultado esperado	Resultado obtenido	Estado
G-01	No se ejecuta si el juego no está inicializado. *Precondición:* juego sin inicializar.	guardar_com pri mido p.bin	(Juego no inicializado) Esta partida no ha sido inicializada correctamente. No se crea archivo.	Igual al esperado. *Evidencia:* Apéndice B.	**Aprobado**
G-02	Valida la cantidad de parámetros (faltan). *Precondición:* juego inicializado.	guardar_com pri mido	Error: Numero de parametros incorrecto. y la ayuda del comando.	Igual al esperado. *Evidencia:* Apéndice B; Figuras C1.	**Aprobado**

G-03	Valida la cantidad de parámetros (sobran). *Precondición:* juego inicializado.	guardar_compri mido a.bin extra	Error: Numero de parametros incorrecto. No se crea el archivo.	Igual al esperado. *Evidencia:* Apéndice B.	**Aprobado**

ID	Objetivo y precondición	Entrada	Resultado esperado	Resultado obtenido	Estado
G-04	Una ruta que no se puede escribir produce el error y la aplicación sigue. *Precondición:* carpeta_inexistente no existe.	guardar_comprimido carpeta_inexiste nte/p.bin	(Error al codificar y/o guardar) La partida no ha sido codificada ni guardada correctamente.	Igual al esperado. *Evidencia:* Apéndice B.	**Aprobado**
G-05	Guardado correcto de la partida inicial con la estructura n c1 f1 … cn fn w binary_code. *Precondición:* partida de archivo_inicio.txt.	guardar_comprimido p3.bin	(Comando correcto) La partida ha sido codificada y guardada correctamente. p3.bin con la estructura indicada.	Igual al esperado: n = 28, w = 514, cabecera 262 B, código 268 B, total 530 B. *Evidencia:* Apéndice B; Figuras C7, C8.	**Aprobado**
G-06	Partida en curso: la cabecera coincide con el texto de guardar. *Precondición:* turno completo de Juan; turno de Pedro.	guardar t.txt; guardar_comprimido t.bin	Ambos correctos; en t.bin, w es la longitud de t.txt y Σ fi = w.	Igual al esperado. *Evidencia:* Apéndice B.	**Aprobado**
G-07	Ciclo guardar_comprimido → inicializar → guardar reproduce la partida. *Precondición:* nueva ejecución; p3.bin de G-05.	inicializar p3.bin; guardar r3.txt	(Comando correcto) El juego se ha inicializado correctamente. y r3.txt idéntico byte a byte a p3.txt.	Igual al esperado. *Evidencia:* Apéndice B; Figuras C9, C10.	**Aprobado**
G-08	Con un juego en curso no se puede reinicializar desde un .bin. *Precondición:* juego inicializado desde p3.bin.	inicializar p3.bin	(Juego en curso) El juego ya ha sido inicializado.	Igual al esperado. *Evidencia:* Apéndice B; Figuras C1, C7.	**Aprobado**

G-09	Ciclo completo con una partida distinta de la inicial. *Precondición:* nueva ejecución; t.bin de G-06.	inicializar t.bin; guardar r_t.txt	(Comando correcto) El juego se ha inicializado correctamente. y r_t.txt idéntico a t.txt.	Igual al esperado. *Evidencia:* Apéndice B; Figuras C9, C10.	**Aprobado**
G-10	Un .bin con la cabecera truncada se rechaza sin errores de memoria. *Precondición:* corrupto.bin = primeros 40 bytes de p3.bin.	inicializar corrupto.bin	(Archivo sin formato) corrupto.bin no contiene información en el formato esperado.	Igual al esperado. *Evidencia:* Apéndice B.	**Aprobado**
G-11	Un .bin con el código binario truncado se rechaza. *Precondición:* truncado.bin = cabecera de p3.bin + 10 bytes.	inicializar truncado.bin	(Archivo sin formato) truncado.bin no contiene información en el formato esperado.	Igual al esperado. *Evidencia:* Apéndice B.	**Aprobado**
G-12	Un .bin con w distinto de Σ fi se rechaza. *Precondición:* w_distinto.bin = p3.bin con w + 1.	inicializar w_distinto.bin	(Archivo sin formato) w_distinto.bin no contiene información en el formato esperado.	Igual al esperado. *Evidencia:* Apéndice B.	**Aprobado**

ID	Objetivo y precondición	Entrada	Resultado esperado	Resultado obtenido	Estado
G-13	Un .bin con n = 0 se rechaza. *Precondición:* n_cero.bin = p3.bin con n = 0.	inicializar n_cero.bin	(Archivo sin formato) n_cero.bin no contiene información en el formato esperado.	Igual al esperado. *Evidencia:* Apéndice B.	**Aprobado**
G-14	Un .bin vacío se informa como archivo vacío. *Precondición:* vacio.bin de 0 bytes.	inicializar vacio.bin	(Archivo vacío) vacio.bin no contiene información.	Igual al esperado. *Evidencia:* Apéndice B.	**Aprobado**
G-15	Un .bin inexistente se informa como archivo erróneo. *Precondición:* noexiste.bin no existe.	inicializar noexiste.bin	(Archivo erróneo) noexiste.bin no se encuentra o no puede leerse.	Igual al esperado. *Evidencia:* Apéndice B.	**Aprobado**
G-16	El código de Huffman ocupa menos que el texto original. *Precondición:* p3.bin y p3.txt de G-05.	Comparar tamaños	Bytes del código < bytes del texto.	Igual al esperado: 268 B frente a 514 B (52,1 %). *Evidencia:* Apéndice B.	**Aprobado**

Nota. 16 de 16 casos aprobados. En el Apéndice B la salida de cada caso aparece con su mismo identificador. El resultado de G-16 se explica en la sección 5.2.

6.4 Casos de consola T01 a T11 y pruebas unitarias

Los ocho casos de consola de la Entrega 1 no tenían resultado ni evidencia, así que los volvimos a ejecutar con el código actual; T09 a T11 son nuevos. Cada caso tiene su archivo de entrada (.in) y su salida real (.out) en la carpeta de pruebas. T08 usa dados y su salida cambia en cada ejecución; por eso ahí verificamos las propiedades que siempre se cumplen.

Tabla 30

Casos de consola T01 a T11

ID	Objetivo	Entrada (.in)	Resultado esperado	Resultado obtenido	Estado

T01	Comandos sin inicializar el juego	obtener_unidades Juan, atacar Juan, fortificar Juan, estado_juego, guardar x.txt	(Juego no inicializado) en los 5 comandos	El mensaje aparece 5 veces.	**Aprobado**
T02	Archivo inexistente	inicializar no_existe.txt	(Archivo erróneo)	(Archivo erróneo) no_existe.txt no se encuentra o no puede leerse.	**Aprobado**

ID	Objetivo	Entrada (.in)	Resultado esperado	Resultado obtenido	Estado
T03	Archivo vacío	inicializar vacio.txt	(Archivo vacío)	(Archivo vacío) vacio.txt no contiene información.	**Aprobado**
T04	Archivos sin formato: territorio repetido, nombre repetido, datos sobrantes y solo 2 jugadores	4 comandos inicializar inv_*.txt	(Archivo sin formato) en los 4	El mensaje aparece 4 veces.	**Aprobado**
T05	Inicialización, reinicialización y estado	inicializar archivo_inicio.txt (2 veces), estado_juego	Correcto; luego (Juego en curso); 3 jugadores, turno de Juan y 42 territorios	Coincide. *Evidencia:* Figura C1.	**Aprobado**
T06	Validaciones de jugador, turno y fase	obtener_unidades Luis, obtener_unidades Pedro, atacar Juan, fortificar Juan, obtener_unidades Juan 1.1 extra	(Jugador no válido), (Jugador fuera de turno), (Jugador no ha ubicado unidades), (Jugador no ha atacado) y error de parámetros	Los 5 mensajes esperados.	**Aprobado**
T07	obtener_unidades: cálculo, entradas inválidas y repetición	obtener_unidades Juan; código XX; código 1.1 con cantidad 0; código 1.1 con 11; repetir el comando	11 unidades; (Codigo invalido); (Cantidad invalida); ubicación correcta; (Jugador ya obtuvo unidades)	Todos los mensajes esperados. *Evidencia:* Figura C2.	**Aprobado**
T08	Flujo completo de un turno: refuerzos, un lanzamiento de dados, fortificar y cambio de turno	obtener_unidades Juan (1.1, 11), atacar Juan (1.1, 13, 5.6, n), fortificar Juan (n), estado_juego	Dados entre 1 y 6; 2 bajas en total; las tropas sobrevivientes vuelven a 1.1; turno de Pedro; ningún “Comando no reconocido”	Atacante 13, defensor 1; 1.1 con 14 y 5.6 con 1; turno de Pedro. Repetido 50 veces sin entradas sobrantes. *Evidencia:*	**Aprobado**

				Apéndice B.	
T09	Regresión: tropas atacantes inválidas no cambian el estado	obtener_unidades Juan (1.1, 11), atacar Juan (1.1, 0), atacar Juan (1.1, 14), fortificar Juan, estado_juego	Error: Opcion invalida dos veces, sin (Comando correcto); fortificar responde (Jugador no ha atacado); 1.1 sigue con 14	Todos los mensajes esperados. *Evidencia:* Apéndice B.	**Aprobado**
T10	Suma inicial de unidades incorrecta	inicializar inv_suma_tropas.txt (Juan con 36), estado_juego, inicializar archivo_inicio.txt	(Archivo sin formato); (Juego no inicializado); luego inicialización correcta	Los 3 mensajes esperados. *Evidencia:* Apéndice B.	**Aprobado**
T11	Refuerzos en 0: jugador con 2 territorios y sin continentes	inicializar inicio_pocos_territorios.txt, obtener_unidades Ana, atacar Ana (0), fortificar Ana (n)	Tienes 0 unidades para ubicar; (Comando correcto) en los tres comandos	Coincide. *Evidencia:* Apéndice B.	**Aprobado**

Nota. En la versión anterior, T08 respondía s cuatro veces para llegar a la conquista. Como los dados son aleatorios, a veces la conquista llegaba antes y las s sobrantes se leían como comandos (“Comando no reconocido”). Ahora T08 responde n después del primer lanzamiento. Con 13 atacantes contra 3 defensores, el defensor pierde como máximo

2 tropas por ronda, así que la pregunta de continuar siempre aparece y consume esa n. La conquista y la carta siguen cubiertas por A-14, A-15, PU-08 y PU-11.

Tabla 31

Pruebas unitarias de los TADs (archivo pruebas_tads.cpp, 72 de 72 correctas)

Grupo	IDs	Qué verifica
Dados	PU-01 a PU-03	Valores entre 1 y 6, aparición de los 6 valores y secuencias distintas con semillas distintas.
Combate	PU-04 y PU-05	3 dados contra 2 ordenados, número de bajas correcto y caso 1 contra 1.
Unidades	PU-06, PU-07 y PU-50	Juan recibe 11 (4 + 5 + 2) y Pedro 7 (4 + 3 por África); con 4 territorios y sin continentes se recibe 1 (no hay mínimo de 3).
Victoria	PU-08 a PU-10	Con los 42 territorios el juego termina con ganador; una sola conquista no lo termina.
Cartas	PU-11 a PU-20, PU-51 y PU-52	1 carta por turno, grupos canjeables, bonos 4/6/8/10/12/15/20 y +2 en cada territorio propio de las cartas (dos territorios propios reciben +2 cada uno; dos cartas del mismo territorio dan +2 una vez).
Fortificación	PU-21 a PU-23	Movimiento válido, el origen nunca queda vacío y no se mueve a un territorio enemigo.
Archivos	PU-24 a PU-29	Archivo válido, territorio repetido, nombre repetido, datos sobrantes, menos de 3 jugadores y archivo vacío.
Suma inicial	PU-46 a PU-49	35/30/25/20 según el número de jugadores; con 3, 4, 5 y 6 jugadores se acepta la suma exacta y se rechazan una unidad de más y una de menos.
Copia de listas (regresión)	PU-30 a PU-33	La copia y la asignación de ListaLineal son independientes del original; la autoasignación no pierde elementos; copiar un Jugador copia su mano de cartas.
Refuerzos (regresión)	PU-34 a PU-36	ubicarUnidades rechaza un territorio ajeno, 0 unidades y más de las disponibles, y acepta una cantidad válida.
Ataque (regresión)	PU-37 a PU-39	Una cantidad de tropas inválida no modifica el origen; apartar y devolver tropas restaura el origen; terminarFaseAtaque pasa a Fortificar.
Guardado	PU-40, PU-41, PU-45 y PU-53	Una partida en curso guardada con guardar_comprimido se recupera igual, también con un jugador eliminado (el turno empieza en el primero que sigue en juego). Una partida guardada con guardar antes de los refuerzos se recupera igual; en texto, una partida con refuerzos ya ubicados se rechaza (sección 7).
Detección del formato	PU-54 a PU-57	Un comprimido llamado .dat se reconoce por su contenido; un texto inicial llamado .bin se carga como texto; un texto en curso llamado .bin sigue exigiendo la suma; bytes que no son ni texto ni Huffman se rechazan y el juego queda vacío.

Grupo	IDs	Qué verifica
Huffman (regresión)	PU-42 a PU-44	decodificar(codificar(texto)) recupera el texto; un solo carácter usa el código 0; empaquetarBits completa el último byte con ceros.
Inicialización segura	PU-58 a PU-61	Sobre una partida activa (refuerzos ubicados, fase Atacar, una carta, 2 canjes), tres archivos inválidos se rechazan y el tablero, el turno, la fase, las cartas y los canjes no cambian; un texto válido reemplaza todo y el tablero nuevo funciona (vecinos y fortificación).
Ataques y fortificaciones inválidos	PU-62 a PU-65	Origen ajeno o nulo, tropas negativas o enormes, territorios propios no vecinos, cantidades 0 o negativas y territorios nulos: se rechazan y nada cambia.
Refuerzos y canjes (límites)	PU-66 y PU-67	ubicarUnidades rechaza negativos y nulos; dos comodines y una carta ajena forman grupo, dan 4 unidades y ningún +2.
Huffman (límites)	PU-68 a PU-70	Los 256 valores de byte sin pérdida; un texto de 200 000 caracteres se recupera y comprime (bytes = ⌈bits ÷ 8⌉); sin partida no se genera un archivo que inicializar acepte.
Memoria (límites)	PU-71 y PU-72	Lista de 100 000 elementos con eliminaciones, intercambio y reutilización; 200 cargas fallidas y correctas seguidas. Las 72 pruebas pasan con AddressSanitizer, UndefinedBehaviorSanitizer y Valgrind, sin errores ni fugas.

Nota. PU-30 a PU-72 se agregaron con las correcciones de la versión final. PU-30 y PU-33 fallan con la versión anterior de ListaLineal (AddressSanitizer detecta un uso de memoria ya liberada), y PU-18, PU-45 y PU-48 a PU-51 fallan si se restauran las reglas anteriores (mínimo de 3, +2 en un solo territorio y sin validar la suma). Con la carga anterior, que trabajaba directamente sobre el juego, PU-58 a PU-61 hacen fallar la ejecución: la partida se borra y la prueba ya no encuentra al jugador.

6.5 Pruebas de robustez y simulación de partidas

Para buscar fallas que los planes anteriores no cubren, el guion robustez.py ejecuta la aplicación compilada con AddressSanitizer y UndefinedBehaviorSanitizer con entradas extremas. Un caso se aprueba si el programa termina (no se queda en un ciclo), no se cae, el sanitizador no reporta errores y, cuando aplica, aparece el mensaje esperado.

Tabla 32

Pruebas de robustez (robustez.py, 22 de 22 aprobadas)

ID	Entrada	Resultado obtenido	Estado
R-01	Líneas vacías, solo espacios y tabuladores entre comandos.	Sin caídas ni errores de memoria.	**Aprobado**
R-02	Comandos desconocidos, mayúsculas, caracteres especiales y de control.	Sin caídas ni errores de memoria.	**Aprobado**

ID	Entrada	Resultado obtenido	Estado
R-03	Línea de 200 000 caracteres.	Sin caídas ni errores de memoria.	**Aprobado**
R-04	obtener_unidades con cantidades negativas, cero, enormes, decimales y texto antes de una válida.	Sin caídas ni errores de memoria.	**Aprobado**
R-05	obtener_unidades con códigos de territorio inválidos (ajeno, inexistente, vacío).	Sin caídas ni errores de memoria.	**Aprobado**
R-06	Fin de la entrada (EOF) en medio de obtener_unidades.	Sin caídas ni errores de memoria.	**Aprobado**
R-07	Fin de la entrada (EOF) en medio de atacar.	Sin caídas ni errores de memoria.	**Aprobado**
R-08	Fin de la entrada sin escribir salir.	Sin caídas ni errores de memoria.	**Aprobado**
R-09	Entrada vacía (0 bytes).	Sin caídas ni errores de memoria.	**Aprobado**
R-10	atacar con tropas negativas, enormes y texto: no cambia el estado.	Sin caídas ni errores de memoria.	**Aprobado**
R-11	Comandos fuera de turno y con jugadores inexistentes o con nombres extremos.	Sin caídas ni errores de memoria.	**Aprobado**
R-12	inicializar dos veces y comandos después de terminar con salir.	Sin caídas ni errores de memoria.	**Aprobado**
R-13	Archivo de inicio con tabuladores y líneas en blanco extra (mismo contenido).	Sin caídas ni errores de memoria.	**Aprobado**
R-14	Territorio con 0 tropas.	Sin caídas ni errores de memoria.	**Aprobado**
R-15	Territorio con tropas negativas o no numéricas.	Sin caídas ni errores de memoria.	**Aprobado**
R-16	Nombre de jugador de 9 caracteres.	Sin caídas ni errores de memoria.	**Aprobado**
R-17	Color de territorio que no es de ningún jugador.	Sin caídas ni errores de memoria.	**Aprobado**
R-18	Archivo binario aleatorio con extensión .txt.	Sin caídas ni errores de memoria.	**Aprobado**

R-19	inicializar con el nombre de una carpeta.	Sin caídas ni errores de memoria.	**Aprobado**
R-20	300 archivos .bin corruptos (bytes cambiados, recortados, con bytes extra o con n o w alterados).	Ninguno provocó caídas ni errores de memoria.	**Aprobado**
R-21	20 sesiones seguidas: inicializar el .bin anterior y volver a guardarlo.	Los 20 textos y los 20 .bin generados son idénticos byte a byte.	**Aprobado**
R-22	5 sesiones con 4000 líneas aleatorias cada una (comandos, códigos y números válidos e inválidos).	Sin caídas, ciclos infinitos ni errores de memoria.	**Aprobado**

Además, simulacion_partidas.cpp juega 60 partidas completas con decisiones al azar usando solo las operaciones de los TADs. En cada turno revisa que los refuerzos, los combates y las fortificaciones cuadren con el total de tropas y que el tablero siga siendo válido, y cada 25 turnos guarda la partida con guardar_comprimido y la vuelve a cargar. En la ejecución final, 30 de las 60 partidas terminaron con un ganador y las demás llegaron al límite de 1500 turnos; hubo 56 724 conquistas y 9549 canjes; las 1921 recargas desde .bin dieron el mismo estado y no se

violó ningún invariante. La primera ejecución de esta simulación sí encontró un error: un .bin guardado después de eliminar a un jugador no se podía volver a cargar, porque se exigía que cada jugador tuviera territorios. Lo corregimos (sección 7) y agregamos PU-53.

7. Observaciones, decisiones de diseño y limitaciones

Al comparar el diseño con el código encontramos algunas decisiones y limitaciones que preferimos dejar por escrito.

Suma inicial de unidades (corregido, con una limitación que no se puede evitar). El numeral 2.1 pide validar que la suma de unidades de cada jugador sea la cantidad inicial (35, 30, 25 o 20), y el numeral 2.2 pide que inicializar cargue los archivos de guardar y de guardar_comprimido, que tienen el mismo formato pero guardan partidas en curso, donde la suma ya cambió. Un archivo de inicio con la suma mal hecha y una partida guardada en texto pueden tener exactamente el mismo contenido, así que ninguna regla basada en el archivo cumple las dos cosas a la vez. Decidimos exigir la suma inicial cuando el contenido se lee como texto plano, como pide el componente 1, y no exigirla cuando se lee como comprimido, porque ese formato solo lo produce guardar_comprimido. Por eso una partida en curso se recupera siempre con guardar_comprimido (PU-40, PU-53 y G-09), y con guardar solo si se guardó antes de ubicar refuerzos (PU-41); un texto con la partida en curso se rechaza (PU-45 y PU-56). El formato del enunciado no tiene ningún campo que permita distinguir los dos casos, y no quisimos agregar uno ni preguntas que el enunciado no pide. La consulta que queremos hacerle a la profesora es: “¿La suma inicial se valida solo en el archivo de inicio, o también en los archivos de guardar? Si es lo segundo, ¿cómo se debe recuperar una partida en curso guardada en texto?”. Si responde que solo en el archivo de inicio, el cambio es un parámetro en inicializarDesdeArchivo.

Inicialización segura (corregido). Antes, cargarDesdeTexto limpiaba la partida antes de validar el archivo, así que un archivo inválido dejaba el juego vacío. En la consola no se notaba, porque inicializar solo se ejecuta si el juego no está inicializado, pero la operación del TAD no era segura. Ahora todo se lee y valida en un juego temporal y solo un archivo válido reemplaza el estado, en un solo paso (PU-58 a PU-61).

Detección del formato (mejorado). Antes el formato se decidía solo por la extensión: un .bin generado por guardar_comprimido con otro nombre, o un texto llamado .bin, se rechazaban aunque fueran válidos. Ahora la extensión decide qué formato se intenta primero y, si falla, se intenta el otro; la suma inicial se exige según cómo se leyó el contenido, no según el nombre (PU-54 a PU-57). Un archivo que no es ninguno de los dos se sigue rechazando con (Archivo sin formato).

Refuerzos en 0 (mensaje agregado). Sin el mínimo de 3, un jugador con menos de 3 territorios y ningún continente recibe 0 unidades. Antes el comando terminaba sin decirlo; ahora muestra “Tienes 0 unidades para ubicar en este turno.”, porque el enunciado pide informar primero cuántas unidades se pueden reclamar (T11).

Refuerzos sin mínimo (corregido). Antes aplicábamos un mínimo de 3 unidades, tomado de las reglas comerciales de Risk. El numeral 1.5 solo dice que se divide el número de territorios entre 3, así que lo quitamos. En la partida de prueba no cambia nada (Juan 11 y Pedro 7), pero un jugador con menos de 9 territorios ahora recibe menos (PU-50).

+2 unidades por cada territorio de las cartas (corregido). Antes el canje sumaba 2 unidades solo en el primer territorio propio de las cartas. El numeral 1.5 dice “2 unidades extra por cada territorio”, así que ahora se suman en cada territorio propio distinto que aparezca en el grupo, y la consola muestra un aviso por cada uno (PU-18, PU-51 y PU-52).

Partida guardada con un jugador eliminado (corregido). Lo encontró la simulación de partidas (sección 6.5): un .bin guardado después de que un jugador perdiera todos sus territorios se rechazaba. Ahora un .bin acepta jugadores sin territorios y, al cargarlo, el turno empieza en el primer jugador que sigue en juego (PU-53).

Número de dados. El numeral 1.5 dice que el atacante lanza 3 dados y el defensor 2, pero no dice qué pasa cuando hay menos tropas. Seguimos el manual de Risk que cita el enunciado: el atacante lanza min(3, tropas atacantes) y el defensor min(2, tropas defensoras), y se comparan tantos pares como dados tenga el lado con menos.

Tropas que ocupan el territorio conquistado. El enunciado dice que el atacante reclama el territorio “moviendo algunas de sus piezas”. Nosotros movemos todas las tropas atacantes que sobrevivieron, porque el jugador ya escogió cuántas arriesgar al empezar el ataque.

Cartas al azar. La carta que se recibe al conquistar se escoge al azar entre los 42 territorios, con reposición; por eso dos cartas pueden ser del mismo territorio. En ese caso el +2 se da una sola vez por territorio.

Turno, fase y cartas al guardar. El formato del enunciado solo tiene los jugadores y los 42 territorios: no tiene campos para el jugador en turno, la fase, las cartas ni el número de canjes. No los agregamos porque el numeral 2.2 pide el mismo formato del archivo de inicio, y un campo extra haría que el archivo no lo cumpliera. Por eso, al cargar una partida guardada, el turno empieza en el primer jugador que tiene territorios, en la fase de obtener unidades, con las manos vacías y el contador de canjes en 0.

Fin de línea. En Windows, guardar escribe en modo texto (CRLF), mientras que el texto que se comprime usa LF. Fuera de esa diferencia el contenido es el mismo, e inicializar lee los dos sin problema (caso G-07).

Carta por eliminar a un jugador. Entregamos una carta por turno cuando hubo al menos una conquista. La carta por eliminar a otro jugador no la implementamos porque no está entre los requisitos del enunciado.

Responsabilidades de main. Después de las correcciones, main ya no cambia directamente tropas, fases ni banderas: lo hace a través de operaciones de los TADs (ubicarUnidades, apartarTropasAtacantes, actualizarDefensor, terminarFaseAtaque, registrarAtaque y avanzarFase). Lo que sí sigue en main son dos consultas para armar menús: la lista de vecinos enemigos del origen en atacar y la de vecinos propios en fortificar. Como solo leen datos, las dejamos ahí.

Copia de listas (corregido). La versión anterior de ListaLineal no tenía constructor de copia ni operador de asignación, y la copia que genera el compilador compartía los nodos. Hoy no causaba fallas porque los jugadores y territorios se copian cuando sus listas internas están vacías, pero era un riesgo real: AddressSanitizer detecta un uso de memoria liberada en PU-30. Agregamos la copia profunda y las pruebas PU-30 a PU-33.

Mensaje de atacar con tropas inválidas (corregido). Antes, si la cantidad de tropas estaba fuera del rango, el programa mostraba el error y enseguida “(Comando correcto) … ha terminado de atacar.”, aunque no hubo ataque. Ahora solo muestra el error y el jugador sigue en la fase de ataque (A-11, A-12 y T09).

Uso de vector de la STL. Las colecciones del modelo (jugadores, territorios, continentes, vecinos, cartas y la cola de prioridad) usan ListaLineal. Para listas temporales usamos std::vector: los resultados de territoriosDeJugador y de las otras consultas de TableroRisk, los dados de resolverRondaCombate y la tabla de ayuda de main.cpp. No las cambiamos porque no

tenemos una restricción explícita del enunciado sobre el uso de la STL en listas auxiliares; si el profesor lo exige, se pueden reemplazar por ListaLineal.

Tildes en la consola de Windows (corregido). El programa escribe en UTF-8. Al iniciar en Windows, ahora cambia la consola a UTF-8 (SetConsoleOutputCP), así que ya no hace falta ejecutar chcp 65001. Las capturas del Apéndice C se tomaron antes de este cambio, y por eso en algunas las tildes y los signos ¿ y ¡ se ven mal.

Compatibilidad con g++ antiguo (corregido). El numeral 3.3 pide que el código compile en gnu-g++ 4.0.0 o superior. main.cpp usaba elementos de C++11 (lista inicializadora, auto y nullptr), que g++ 4.x no acepta sin opciones. Los cambiamos por un arreglo, ciclos con índice y NULL, y ahora el código compila sin advertencias también con -std=c++98. No tenemos un g++ 4.0 para probarlo directamente; el único elemento fuera de C++98 que queda es el tipo long long de las frecuencias, que g++ acepta desde sus primeras versiones.

Costo de las consultas. obtener(indice) y obtenerTerritorioPorId recorren la lista, así que son O(N). Con 42 territorios y máximo 6 jugadores ese costo no se nota.

8. Compilación y contenido de la entrega

El código compila sin advertencias con -Wall -Wextra en g++ 13.3 (Linux, en los modos C++98, C++11 y C++17) y en MinGW-w64 13.2 (Windows). Desde la carpeta del .zip de BrightSpace, donde todos los archivos están juntos:

```
g++ -Wall *.cpp -o risk_app
./risk_app
```

Desde la raíz del repositorio, donde los TADs están en la carpeta tads/:

```
g++ -Wall -I tads src/main.cpp tads/*.cpp -o risk_app
./risk_app
```

El numeral 3 exige un solo .zip con los archivos en un mismo directorio, sin carpetas, y solo con el documento en PDF y los fuentes .h, .hxx, .cxx y .cpp. Por eso el .zip de la entrega contiene únicamente:

- **Documento:** Proyecto_Risk_Entrega2.pdf (este documento).
- **Fuentes (21 archivos):** main.cpp, ListaLineal.h, ListaLineal.hxx, Carta.h, Territorio.h y .cpp, Continente.h y .cpp, Jugador.h y .cpp, TableroRisk.h y .cpp, JuegoRisk.h y .cpp, NodoHuffman.h, ColaPrioridad.h y .cpp, ArbolHuffman.h y .cpp, ArchivoHuffman.h y .cpp.

No van ejecutables, archivos de datos (.txt, .bin) ni las pruebas. pruebas_tads.cpp y simulacion_partidas.cpp tienen su propio main y harían fallar g++ *.cpp, así que se quedan en el repositorio. Como el archivo de inicio no se puede incluir en el .zip, su contenido completo está en el Apéndice D para crearlo copiando y pegando. El repositorio conserva la estructura de desarrollo: src/, tads/, data/ (archivos de partida), tests/ (casos T01 a T11, pruebas unitarias, simulación, el guion ejecutar_todo.sh que corre todas las pruebas y las salidas reales), docs/ (este documento, evidencias y guiones) y risk_app.exe.

El código y su historial también están en el repositorio del proyecto (Wright Gaviria & Martínez Gómez, 2026).

Referencias

Anthropic. (2026). *Claude* [Modelo de lenguaje de gran tamaño]. https://claude.ai

Huffman, D. A. (1952). A method for the construction of minimum-redundancy codes. *Proceedings of the IRE, 40*(9), 1098–1101. https://doi.org/10.1109/JRPROC.1952.273898

Pontificia Universidad Javeriana. (2026). *Enunciado del proyecto Risk de Estructuras de Datos* [Material de clase no publicado]. Departamento de Ingeniería de Sistemas.

Wright Gaviria, I. D., & Martínez Gómez, J. S. (2026). *Proyecto Risk: Código fuente y documentación de la entrega 2* [Repositorio de código]. GitHub. https://github.com/IvanDWright/Proyecto_Estructuras

Apéndice A

Registro de prompts utilizados

Transcribimos los prompts de forma literal y en el orden en que los usamos. Corregimos la numeración del registro anterior, donde dos prompts aparecían como “Prompt 2”.

Prompt 1 (Requisitos y user paths): “Actúa como un desarrollador de software, que necesita analizar requisitos para un proyecto, basándote en los puntos 3.1 y 2.4 del documento adjunto, el Markdown debe explicar claramente cómo el código fuente cumple con los requisitos obligatorios (prompt $, validaciones, comando ayuda, sistema centralizado, mensajes exactos). También genera los flujos de usuario (User Paths) detallando paso a paso la interacción de la consola para la inicialización, turno de juego, consulta y cierre. Reglas estrictas para tu respuesta: NO incluyas saludos, introducciones, confirmaciones, ni conclusiones.”

Prompt 2 (Plantilla): “Actúa como un desarrollador de software documentando un proyecto de Estructuras de Datos. El PDF del enunciado oficial se encuentra en este archivo. Necesito que generes el contenido de un archivo en formato Markdown (.md) que sirva como una plantilla para la documentación y presentación del código específicamente para la Entrega 0 (semana 3) Teniendo en cuenta los requisitos y los user paths creados previamente. Reglas estrictas para tu respuesta: NO incluyas saludos, introducciones, confirmaciones, ni conclusiones. Tu respuesta debe ser única y exclusivamente el contenido del archivo Markdown. Añade un Checklist de Envío con las reglas estrictas de la sección 3.”

Prompt 3 (Verificación de redacción y criterios): “Realiza una verificación de que el contenido de la documentación cumple con todos los criterios solicitados en el enunciado.”

Prompt 4 (Creación de la estructura base y módulo de ayuda en C++): “Actúa como un desarrollador experto en C++. Con base en los requerimientos del enunciado oficial (numerales 2.4 y 3.1) y la especificación de los User Paths previamente estructurados, genera el archivo principal de la Entrega 0 (main.cpp). El código debe incluir: 1. Las librerías estándar necesarias (<iostream>, <string>, <vector>, <sstream>, <map>, etc.). 2. Una estructura de datos adecuada (como un std::map de structs) para registrar todos los comandos del sistema junto con su sintaxis de uso y descripción detallada. 3. Las funciones auxiliares mostrar_ayuda() y mostrar_ayuda_comando(const string& cmd) para gestionar la salida del comando ayuda. 4. Asegúrate de incluir únicamente los encabezados e implementaciones esenciales para esta fase inicial, manteniendo una arquitectura limpia y extensible.”

Prompt 5 (Verificación, corrección y auditoría del bucle REPL): “Actúa como un revisor de código (Code Reviewer) en C++. Analiza la implementación de la consola interactiva (ciclo REPL en main.cpp) y verifícala minuciosamente contra las especificaciones del enunciado oficial. Realiza las correcciones necesarias sobre el código fuente para garantizar que: 1. El prompt de la línea de comandos sea estrictamente el carácter $ seguido de un espacio (cout << "$

";). 2. Se realice una tokenización correcta de la entrada con std::stringstream para validar la cantidad exacta de parámetros. 3. Los mensajes de respuesta coincidan carácter por carácter con las plantillas del enunciado. Requisito de documentación: Al inicio del archivo o en la sección de revisión del documento, añade un bloque de comentarios detallado que documente: El código base previo sometido a revisión y la lista explícita de correcciones realizadas junto con el motivo técnico de cada ajuste."

Prompt 6 (Corrección crítica de empaquetado y ajuste fino del REPL): "Actúa como un revisor de código (Code Reviewer) en C++ y auditor de entrega. Verifica que el archivo comprimido de la entrega no contenga subarchivos ni código fuente ajeno al proyecto (por ejemplo, talleres de depuración previos), dejando únicamente el PDF de documentación y main.cpp en la raíz del .zip. Adicionalmente, corrige el bucle REPL para que una línea vacía (Enter sin texto) vuelva a mostrar el prompt $ sin imprimir mensajes de error, confirma que la tokenización con std::stringstream maneja correctamente espacios múltiples entre argumentos, y realiza una verificación carácter por carácter de los mensajes de salida contra las plantillas del enunciado."

Prompt 7 (Correcciones del proyecto y de la codificación de Huffman): "Actúa como un revisor de código (Code Reviewer) en C++ y auditor de entrega. Con base en las observaciones de la Entrega 1 (la lógica está en main y no en los TADs, nunca se verifica la victoria, los dados no cambian por falta de srand y la validación del archivo es incompleta) y en los numerales 2.1 y 2.2 del enunciado oficial, analiza el código del proyecto y corrígelo para garantizar que: 1. Se verifique la victoria cuando un jugador ocupa los 42 territorios. 2. La semilla de los dados se inicialice una sola vez con srand al arrancar la aplicación. 3. La validación del archivo de inicio rechace nombres y colores repetidos, territorios repetidos y datos sobrantes. 4. La codificación de Huffman (ColaPrioridad, ArbolHuffman y ArchivoHuffman) construya siempre el mismo árbol para las mismas frecuencias, escriba la estructura n c1 f1 ... cn fn w binary_code y al decodificar recupere exactamente el texto original. Requisito de documentación: añade un bloque de comentarios que documente el código previo sometido a revisión y la lista explícita de correcciones realizadas junto con el motivo técnico de cada ajuste. Reglas estrictas para tu respuesta: NO incluyas saludos, introducciones, confirmaciones, ni conclusiones."

Prompt 8 (Traslado de la lógica de main a los TADs): "Actúa como un desarrollador experto en C++ y en diseño de TADs. Con base en la observación "la lógica está en main y no en los TADs" y en la exigencia del enunciado de que el diseño sea totalmente coherente con la implementación, traslada la lógica de los comandos obtener_unidades, atacar y fortificar desde main.cpp a los TADs JuegoRisk, TableroRisk, Territorio y Jugador (por ejemplo calcularUnidadesBase, canjearCartas, resolverRondaCombate, conquistarTerritorio, otorgarCartaPorConquista, moverTropas, territoriosAptosParaAtacar y buscarGrupoCanjeable), de forma que main solo lea los datos que escribe el usuario y muestre los mensajes. Requisito de documentación: cada operación nueva debe quedar comentada con lo que recibe, lo que retorna y

su funcionamiento paso a paso. Reglas estrictas para tu respuesta: NO incluyas saludos, introducciones, confirmaciones, ni conclusiones."

Prompt 9 (Plantilla del documento a partir de las correcciones): "Actúa como un desarrollador de software documentando un proyecto de Estructuras de Datos. Me ayudas con una plantilla para el documento a base de correcciones: con base en las observaciones de la Entrega 1 (adjuntas en una imagen), el documento de la Entrega 0 y el código fuente actual, genera la estructura de un documento de diseño y plan de pruebas para la Entrega 2 que incluya el acta de evaluación con los comentarios literales y su corrección, la descripción de entradas, salidas y condiciones de cada comando, los TADs con la plantilla de clase, el diagrama de relación y el plan de pruebas de los comandos atacar y guardar_comprimido con su resultado obtenido. Verifica que el diseño sea coherente con el código y deja todo lo que requiera la Entrega 2. Reglas estrictas para tu respuesta: NO incluyas saludos, introducciones, confirmaciones, ni conclusiones."

Prompt 10 (Depuración del documento): "Quiero que revises críticamente todo el documento, porque mi objetivo no es tener un Word extenso y saturado de información, sino un documento completo, preciso, profesional y fácil de comprender. […] Quiero un documento de Word con normas APA 7, visualmente impecable, bien estructurado, sin páginas innecesarias, sin repeticiones y con una extensión justificada por su contenido."

Prompt 11 (Corrección final del documento): "Actúa como un editor académico universitario de excelencia, especializado en ingeniería de sistemas, estructuras de datos, programación en C++, documentación técnica y normas APA 7.ª edición. […] Tu trabajo NO es rehacerlo desde cero. Tu trabajo es perfeccionar la versión existente, corregir sus debilidades y convertirla en una entrega universitaria de calidad sobresaliente. […] No inventes condiciones, operaciones ni decisiones."

Prompt 12 (Corrección del código y del documento): "Actúa como un ingeniero de software experto en C++11, estructuras de datos, TAD, árboles de Huffman, pruebas de software y documentación académica bajo normas APA 7. Necesito que revises y corrijas mi proyecto Risk – Entrega 2, utilizando el documento Word, el código fuente del ZIP y el enunciado oficial del profesor, si está disponible. […] Comando atacar: revisa y corrige el problema por el cual puede aparecer un mensaje de finalización correcta después de introducir una cantidad inválida de tropas. […] Caso T08: revisa y corrige las respuestas s sobrantes que pueden interpretarse como comandos independientes. Ejecuta nuevamente el caso y actualiza su evidencia. […]"

Prompt 13 (Auditoría final contra el enunciado): "Actúa como un equipo de expertos en ingeniería de software, C++11, estructuras de datos lineales, árboles binarios, codificación de Huffman, diseño de TAD, pruebas de software, diagramas de flujo y documentación académica. Necesito que realices una auditoría integral, corrección y perfeccionamiento de mi Proyecto Risk – Entrega 2 […]. El enunciado oficial tiene prioridad absoluta sobre cualquier recomendación, regla general de Risk o decisión anterior del proyecto. […] 1.1. Validación de tropas iniciales: el

enunciado exige validar la cantidad exacta de unidades iniciales según el número de jugadores […] No implementes una validación que rompa la recuperación de partidas. 1.2. Cálculo de refuerzos […] Eliminar el mínimo de tres si no existe una autorización expresa para utilizarlo. 1.3. Canje de cartas […] Verifica especialmente la regla de las dos tropas adicionales por territorio […] Corrige específicamente el título de la sección 6.4, que todavía menciona T01 a T08 aunque ya existe T09. […] Prepara un ZIP diferente para la entrega académica. […] No intentes ocultar el uso de inteligencia artificial ni introducir errores para aparentar autoría humana. […] No inventes información. No ocultes errores. No declares aprobadas pruebas que no ejecutaste." Junto con este prompt adjuntamos el enunciado y pedimos: "Primero, preséntame una matriz completa de cumplimiento del enunciado, identificando lo que está correcto, lo que falta y lo que necesita corrección. Después, realiza las correcciones y prepara los archivos definitivos." También pedimos pruebas extremas y de robustez.

Prompt 14 (Correcciones de la recuperación de partidas y del formato): "Quiero que revises y corrijas mi proyecto Entrega2_Risk_Wright_Martinez.zip, utilizando como referencia obligatoria el enunciado oficial de la profesora de Estructuras de Datos. No quiero que rehagas el proyecto. Quiero que conserves todo lo que está bien y corrijas únicamente los errores, inconsistencias y aspectos que realmente necesitan mejorar. […] Corregir la recuperación de partidas guardadas […] Corregir la validación de tropas iniciales: actualmente la validación se omite para archivos .bin. Implementa una solución coherente que permita validar configuraciones iniciales y recuperar partidas guardadas, sin aceptar archivos inválidos ni rechazar partidas legítimas. […] Revisar el formato de guardado […] pero no agregues información que contradiga el formato oficial. Mejorar la detección de archivos comprimidos: revisa la dependencia de la extensión .bin […] Agrega pruebas específicas de recuperación de partidas en texto y comprimidas, validación de tropas, archivos corruptos, entradas inválidas y casos límite. […] No afirmes que merece 5,0/5,0 sin evidencias suficientes." Cuando Claude explicó que la validación de la suma y la recuperación de cualquier texto guardado no se pueden cumplir a la vez, escogimos mantener la regla de la sección 7.

Prompt 15 (Auditoría final e inicialización segura): "Quiero que trabajes sobre la última versión de Entrega2_Risk_Wright_Martinez.zip y el enunciado oficial de la profesora. Mi objetivo es alcanzar la máxima calificación posible. No quiero más mejoras estéticas innecesarias: quiero eliminar todos los errores comprobables que puedan afectar mi nota. 1. Corrección crítica: inicialización segura. Revisa JuegoRisk::cargarDesdeTexto. Actualmente existe el riesgo de que una inicialización fallida elimine o modifique una partida activa. Corrige esto mediante una inicialización segura […] 2. Resolver o documentar el conflicto de los archivos […] Si los requisitos son incompatibles sin información adicional, no inventes una solución: conserva la decisión más defendible, explica la limitación y señala la consulta que debería hacerse a la profesora. 3. Verificación de las 57 pruebas […] Agrega pruebas de: inicialización fallida sin pérdida de estado, archivos corruptos, guardado y recuperación, compresión y descompresión Huffman, ataques inválidos, fortificaciones inválidas, refuerzos y canjes, casos límite de memoria. No inventes resultados. […] 5. Sustentación para máxima nota

[…] Incluye 20 preguntas difíciles de sustentación y respuestas basadas exclusivamente en el código definitivo. […] No me digas que está perfecto por cortesía.”

Apéndice B

Salidas de consola de los casos de prueba

Cada bloque es la salida real del programa para el caso indicado, en el mismo orden de las Tablas 27 y 29. Las salidas de los casos A y G las generó el guion pruebas_doc.py, que abre el programa compilado, escribe las entradas y guarda lo que la aplicación responde; el texto completo está en docs/evidencias/resultados.json del repositorio. El programa no repite lo que se escribe, así que el guion pone cada entrada justo después de la pregunta correspondiente. Cuando una lista larga ya apareció en un caso anterior la abreviamos con una nota entre paréntesis. Los mensajes no llevan tildes porque así los imprime el programa.

Plan del comando atacar

```
$ inicializar archivo_inicio.txt
(Comando correcto) El juego se ha inicializado correctamente.
$ obtener_unidades Juan

Tienes 11 unidades para ubicar.
En cual de tus territorios deseas ubicar unidades?
*  - Alaska (Actuales: 3)
*  - Alberta (Actuales: 3)
*  - America Central (Actuales: 3)
*  - Estados Unidos Orientales (Actuales: 3)
*  - Groenlandia (Actuales: 3)
*  - Territorio Noroccidental (Actuales: 3)
*  - Ontario (Actuales: 3)
*  - Quebec (Actuales: 2)
*  - Estados Unidos Occidentales (Actuales: 2)
*  - Argentina (Actuales: 2)
*  - Brasil (Actuales: 2)
*  - Peru (Actuales: 2)
*  - Venezuela (Actuales: 2)
*  - Gran Bretana (Actuales: 2)
Ingresa el codigo del territorio (ej. 1.1): 1.1 Cuantas
unidades deseas ubicar aqui? (Maximo 11): 11
-> Se han anadido 11 tropas a Alaska.
(Comando correcto) El jugador Juan ha terminado de reclamar y ubicar sus unidades.
$
```

Precondición de los casos A-08 a A-17: Juan ubica sus 11 unidades en 1.1 Alaska

```
$ atacar Juan
(Juego no inicializado) Esta partida no ha sido inicializada correctamente.
$
```

Caso A-01

```
$ atacar
Error: Numero de parametros incorrecto.
Uso: atacar <nombre_jugador>
Descripcion: Inicia la fase de ataque para el jugador especificado.
$
```

Caso A-02

```
$ atacar Juan extra
Error: Numero de parametros incorrecto.
Uso: atacar <nombre_jugador>
Descripcion: Inicia la fase de ataque para el jugador especificado.
$
```

Caso A-03

```
$ atacar Pepe
(Jugador no válido) El jugador Pepe no forma parte de esta partida.
$
```

Caso A-04

```
$ atacar Pedro
(Jugador fuera de turno) No es el turno del jugador Pedro.
$
```

Caso A-05

```
$ atacar Juan
(Jugador no ha ubicado unidades) El jugador Juan no ha ejecutado el comando obtener_unidades.
$
```

Caso A-06

```
$ fortificar Juan
(Jugador no ha atacado) El jugador Juan no ha ejecutado el comando atacar.
$
```

Caso A-07

```
$ atacar Juan

Desde cual de tus territorios quieres atacar? (Ingresa 0 para no atacar / terminar fase de
ataque)
*  - Alaska (tienes 14 tropas)
*  - Alberta (tienes 3 tropas) [No disponible: no tiene vecinos enemigos]
*  - America Central (tienes 3 tropas) [No disponible: no tiene vecinos enemigos]
*  - Estados Unidos Orientales (tienes 3 tropas) [No disponible: no tiene vecinos
   enemigos]
*  - Groenlandia (tienes 3 tropas)
*  - Territorio Noroccidental (tienes 3 tropas) [No disponible: no tiene vecinos
   enemigos]
*  - Ontario (tienes 3 tropas) [No disponible: no tiene vecinos enemigos]
*  - Quebec (tienes 2 tropas) [No disponible: no tiene vecinos enemigos]
*  - Estados Unidos Occidentales (tienes 2 tropas) [No disponible: no tiene vecinos
   enemigos]
*  - Argentina (tienes 2 tropas) [No disponible: no tiene vecinos enemigos]
*  - Brasil (tienes 2 tropas)
*  - Peru (tienes 2 tropas) [No disponible: no tiene vecinos enemigos]
*  - Venezuela (tienes 2 tropas) [No disponible: no tiene vecinos enemigos]
*  - Gran Bretana (tienes 2 tropas)
 0 - Terminar fase de ataque y pasar a fortificar
Ingresa el codigo del territorio de origen (o 0 para terminar):
```

Caso A-08

```
Ingresa el codigo del territorio de origen (o 0 para terminar): 9.9
Error: Codigo invalido o territorio no disponible para atacar Ingresa
el codigo del territorio de origen (o 0 para terminar):
```

Caso A-09

```
Ingresa el codigo del territorio de origen (o 0 para terminar): 1.4
Error: Codigo invalido o territorio no disponible para atacar Ingresa
el codigo del territorio de origen (o 0 para terminar):
```

Caso A-10

```
Ingresa el codigo del territorio de origen (o 0 para terminar): 1.1
Con cuantas tropas quieres atacar? (1 a 13):
0
Error: Opcion invalida. El ataque no se realizo; puedes ejecutar atacar de nuevo.
$
```

Caso A-11

```
$ atacar Juan

Desde cual de tus territorios quieres atacar? (Ingresa 0 para no atacar / terminar fase de
ataque)
(lista de los 14 territorios de Juan, igual a la del caso A-08; 1.1 Alaska con 14 tropas)
Ingresa el codigo del territorio de origen (o 0 para terminar): 1.1
Con cuantas tropas quieres atacar? (1 a 13):
14
Error: Opcion invalida. El ataque no se realizo; puedes ejecutar atacar de nuevo.
$
```

Caso A-12

```
$ atacar Juan

Desde cual de tus territorios quieres atacar? (Ingresa 0 para no atacar / terminar fase de
ataque)
(lista de los 14 territorios de Juan, igual a la del caso A-08; 1.1 Alaska con 14 tropas)
Ingresa el codigo del territorio de origen (o 0 para terminar): 1.1
Con cuantas tropas quieres atacar? (1 a 13):
13

A cual territorio quieres atacar?
*  - Kamchatka (Propietario: verde - Tropas: 3)
   Ingresa el codigo del territorio a atacar: 1.2
   Error: Codigo invalido
Ingresa el codigo del territorio a atacar: 5.6

[!] Lanzamiento de dados...
Atacante tira 3 dado(s):
 -  Dado 1: 5
 -  Dado 2: 5
 -  Dado 3: 4
Defensor tira 2 dado(s):
 -  Dado 1: 2
 -  Dado 2: 2
---------------------------------

--- Resultados de los duelos ---
Duelo 1: Atacante (5) vs Defensor (2) -> El defensor pierde 1 tropa.
Duelo 2: Atacante (5) vs Defensor (2) -> El defensor pierde 1 tropa. Tropas

restantes de la invasion - Atacante: 13 | Defensor: 1

¿Deseas continuar el ataque con las tropas restantes? (s/n): n El
defensor mantiene el territorio por ahora.
Ataque finalizado. Tropas sobrevivientes devueltas a tu territorio.
(Comando correcto) El jugador Juan ha terminado de atacar.
$
```

Caso A-13

```
$ atacar Juan

Desde cual de tus territorios quieres atacar? (Ingresa 0 para no atacar / terminar fase de
ataque)
(lista de los 14 territorios de Juan, igual a la del caso A-08; 1.1 Alaska con 14 tropas)
Ingresa el codigo del territorio de origen (o 0 para terminar): 1.1
Con cuantas tropas quieres atacar? (1 a 13):
13

A cual territorio quieres atacar?
 5.6 - Kamchatka (Propietario: verde - Tropas: 1)
Ingresa el codigo del territorio a atacar: 5.6

[!] Lanzamiento de dados...
Atacante tira 3 dado(s):
 -  Dado 1: 3
 -  Dado 2: 1
 -  Dado 3: 1
Defensor tira 1 dado(s):
 -  Dado 1: 2
---------------------------------

--- Resultados de los duelos ---
Duelo 1: Atacante (3) vs Defensor (2) -> El defensor pierde 1 tropa.
```

Caso A-14

```
Tropas restantes de la invasion - Atacante: 13 | Defensor: 0

¡El atacante ha conquistado el territorio!
¡Has conquistado el territorio Kamchatka!
Tropas ubicadas en ese territorio: 13
(Comando correcto) El jugador Juan ha terminado de atacar.
$ estado_juego
--- ESTADO DEL JUEGO ---
Jugadores en partida (3):
- Juan [Color: rojo]
- Pedro [Color: azul]
- Maria [Color: verde]
Jugador con el turno actual: Juan
Lista de Territorios:
(lista de los 42 territorios; se muestra solo 5.6)
- 5.6 (Kamchatka): Controlado por 'rojo' con 13 unidades.
$
```

```
$ atacar Juan

Desde cual de tus territorios quieres atacar? (Ingresa 0 para no atacar / terminar fase de
ataque)
(lista de los 14 territorios de Juan, igual a la del caso A-08; 1.1 Alaska con 1 tropas)
Ingresa el codigo del territorio de origen (o 0 para terminar): 0
[*] Recibiste una carta por conquistar territorios en este turno.
(Comando correcto) El jugador Juan ha terminado de atacar.
$
```

Caso A-15

```
$ atacar Juan
(Jugador ya fortifico) El jugador Juan ya no puede atacar.
$
```

Caso A-16

```
$ fortificar Juan
¿Quieres fortificar algun territorio? (s/n): n
(Comando correcto) El jugador Juan ha terminado de fortificar su posición.
$ estado_juego
--- ESTADO DEL JUEGO ---
Jugadores en partida (3):
- Juan [Color: rojo]
- Pedro [Color: azul]
- Maria [Color: verde]
Jugador con el turno actual: Pedro
Lista de Territorios:
(lista de los 42 territorios; se muestra solo 5.6)
- 5.6 (Kamchatka): Controlado por 'rojo' con 13 unidades.
$
```

Caso A-17

```
$ atacar Juan
(Jugador no puede atacar) Juan no tiene territorios con mas de 1 tropa y vecinos enemigos
para atacar.
$
```

Caso A-18

Plan del comando guardar_comprimido

```
$ guardar_comprimido p.bin
(Juego no inicializado) Esta partida no ha sido inicializada correctamente.
$
```

Caso G-01

```
$ guardar_comprimido
Error: Numero de parametros incorrecto.
Uso: guardar_comprimido <nombre_archivo>
Descripcion: Guarda el estado del juego en un archivo binario comprimido (Huffman).
$
```

Caso G-02

```
$ guardar_comprimido a.bin extra
Error: Numero de parametros incorrecto.
Uso: guardar_comprimido <nombre_archivo>
Descripcion: Guarda el estado del juego en un archivo binario comprimido (Huffman).
$
```

Caso G-03

```
$ guardar_comprimido carpeta_inexistente/p.bin
(Error al codificar y/o guardar) La partida no ha sido codificada ni guardada
correctamente.
$ estado_juego
--- ESTADO DEL JUEGO ---
Jugadores en partida (3):
- Juan [Color: rojo]
- Pedro [Color: azul]
- Maria [Color: verde]
Jugador con el turno actual: Juan
Lista de Territorios:
(lista de los 42 territorios; se muestra solo 5.6)
- 5.6 (Kamchatka): Controlado por 'verde' con 3 unidades.

$
```

Caso G-04

```
$ guardar p3.txt
(Comando correcto) La partida ha sido guardada correctamente.
$ guardar_comprimido p3.bin
(Comando correcto) La partida ha sido codificada y guardada correctamente.
$
```

Caso G-05

```
(Se jugo un turno completo de Juan)
guardar t.txt
(Comando correcto) La partida ha sido guardada correctamente.
$ guardar_comprimido t.bin
(Comando correcto) La partida ha sido codificada y guardada correctamente.
$
```

Caso G-06

```
$ inicializar p3.bin
(Comando correcto) El juego se ha inicializado correctamente.
$ guardar r3.txt
(Comando correcto) La partida ha sido guardada correctamente.
$
```

Caso G-07

```
$ inicializar p3.bin
(Juego en curso) El juego ya ha sido inicializado.
$
```

Caso G-08

```
$ inicializar t.bin
(Comando correcto) El juego se ha inicializado correctamente.
$ guardar r_t.txt
(Comando correcto) La partida ha sido guardada correctamente.
$
```

Caso G-09

```
$ inicializar corrupto.bin
(Archivo sin formato) corrupto.bin no contiene información en el formato esperado.
$
```

Caso G-10

```
$ inicializar truncado.bin
(Archivo sin formato) truncado.bin no contiene información en el formato esperado.
$
```

Caso G-11

```
$ inicializar w_distinto.bin
(Archivo sin formato) w_distinto.bin no contiene información en el formato esperado.
$
```

Caso G-12

```
$ inicializar n_cero.bin
(Archivo sin formato) n_cero.bin no contiene información en el formato esperado.
$
```

Caso G-13

```
$ inicializar vacio.bin
(Archivo vacío) vacio.bin no contiene información.
$
```

Caso G-14

```
$ inicializar noexiste.bin
(Archivo erróneo) noexiste.bin no se encuentra o no puede leerse.
$
```

Caso G-15

```
p3.txt: 514 bytes
codigo en p3.bin: 268 bytes (52.1 %)
encabezado (n, ci, fi, w): 262 bytes
total p3.bin: 530 bytes
```

Caso G-16 (medición de tamaños hecha por el guion sobre p3.txt y p3.bin)

Casos de consola T08 a T11 (tests/resultados/)

```
Bienvenido al juego Risk. Pon 'ayuda' para ver los comandos disponibles.
$ (Comando correcto) El juego se ha inicializado correctamente.
$
Tienes 11 unidades para ubicar.
En cual de tus territorios deseas ubicar unidades?
```

- - Alaska (Actuales: 3)
- - Alberta (Actuales: 3)
- - America Central (Actuales: 3)
- - Estados Unidos Orientales (Actuales: 3)
- - Groenlandia (Actuales: 3)
- - Territorio Noroccidental (Actuales: 3)
- - Ontario (Actuales: 3)
- - Quebec (Actuales: 2)
- - Estados Unidos Occidentales (Actuales: 2)
- - Argentina (Actuales: 2)
- - Brasil (Actuales: 2)
- - Peru (Actuales: 2)
- - Venezuela (Actuales: 2)
- - Gran Bretana (Actuales: 2)

```
Ingresa el codigo del territorio (ej. 1.1): Cuantas unidades deseas ubicar aqui? (Maximo 11):
-> Se han anadido 11 tropas a Alaska.
(Comando correcto) El jugador Juan ha terminado de reclamar y ubicar sus unidades.
$
Desde cual de tus territorios quieres atacar? (Ingresa 0 para no atacar / terminar fase de
ataque)
(lista de los 14 territorios de Juan, igual a la del caso A-08; 1.1 Alaska con 14 tropas)
Ingresa el codigo del territorio de origen (o 0 para terminar): Con cuantas tropas quieres
atacar? (1 a 13):
```

Caso T08

```
A cual territorio quieres atacar?
*  - Kamchatka (Propietario: verde - Tropas: 3)
   Ingresa el codigo del territorio a atacar:
[!] Lanzamiento de dados...
Atacante tira 3 dado(s):
 -  Dado 1: 5
 -  Dado 2: 5
 -  Dado 3: 4
Defensor tira 2 dado(s):
 -  Dado 1: 2
 -  Dado 2: 2

--- Resultados de los duelos ---
Duelo 1: Atacante (5) vs Defensor (2) -> El defensor pierde 1 tropa.
Duelo 2: Atacante (5) vs Defensor (2) -> El defensor pierde 1 tropa. Tropas

restantes de la invasion - Atacante: 13 | Defensor: 1

¿Deseas continuar el ataque con las tropas restantes? (s/n): El defensor mantiene el
territorio por ahora.
Ataque finalizado. Tropas sobrevivientes devueltas a tu territorio.
(Comando correcto) El jugador Juan ha terminado de atacar.
$ ¿Quieres fortificar algun territorio? (s/n): (Comando correcto) El jugador Juan ha
terminado de fortificar su posición.
$ --- ESTADO DEL JUEGO ---
Jugadores en partida (3):
-  Juan [Color: rojo]
-  Pedro [Color: azul]
-  Maria [Color: verde]
Jugador con el turno actual: Pedro
Lista de Territorios:
(lista de los 42 territorios; se muestra solo 5.6)
--- 5.6 (Kamchatka) - Controlado por 'verde' con 1 unidades.

$
```

```
Bienvenido al juego Risk. Pon 'ayuda' para ver los comandos disponibles.
$ (Comando correcto) El juego se ha inicializado correctamente.
$
Tienes 11 unidades para ubicar.
En cual de tus territorios deseas ubicar unidades?
*  - Alaska (Actuales: 3)
*  - Alberta (Actuales: 3)
*  - America Central (Actuales: 3)
*  - Estados Unidos Orientales (Actuales: 3)
*  - Groenlandia (Actuales: 3)
*  - Territorio Noroccidental (Actuales: 3)
*  - Ontario (Actuales: 3)
*  - Quebec (Actuales: 2)
*  - Estados Unidos Occidentales (Actuales: 2)
*  - Argentina (Actuales: 2)
*  - Brasil (Actuales: 2)
*  - Peru (Actuales: 2)
*  - Venezuela (Actuales: 2)
*  - Gran Bretana (Actuales: 2)
Ingresa el codigo del territorio (ej. 1.1): Cuantas unidades deseas ubicar aqui? (Maximo 11):
-> Se han anadido 11 tropas a Alaska.
(Comando correcto) El jugador Juan ha terminado de reclamar y ubicar sus unidades.
$
```

Caso T09 (regresión)

```
Desde cual de tus territorios quieres atacar? (Ingresa 0 para no atacar / terminar fase de
ataque)
(lista de los 14 territorios de Juan, igual a la del caso A-08; 1.1 Alaska con 14 tropas)
Ingresa el codigo del territorio de origen (o 0 para terminar): Con cuantas tropas quieres
atacar? (1 a 13):
Error: Opcion invalida. El ataque no se realizo; puedes ejecutar atacar de nuevo.
$
Desde cual de tus territorios quieres atacar? (Ingresa 0 para no atacar / terminar fase de
ataque)
(lista de los 14 territorios de Juan, igual a la del caso A-08; 1.1 Alaska con 14 tropas)
Ingresa el codigo del territorio de origen (o 0 para terminar): Con cuantas tropas quieres
atacar? (1 a 13):
Error: Opcion invalida. El ataque no se realizo; puedes ejecutar atacar de nuevo.
$ (Jugador no ha atacado) El jugador Juan no ha ejecutado el comando atacar.
$ --- ESTADO DEL JUEGO ---
Jugadores en partida (3):
- Juan [Color: rojo]
- Pedro [Color: azul]
- Maria [Color: verde]
Jugador con el turno actual: Juan
Lista de Territorios:
(lista de los 42 territorios; se muestra solo 5.6)
----------------------
- 5.6 (Kamchatka): Controlado por 'verde' con 3 unidades.
```

```
$
```

```
Bienvenido al juego Risk. Pon 'ayuda' para ver los comandos disponibles.
$ (Archivo sin formato) inv_suma_tropas.txt no contiene información en el formato esperado.
$ (Juego no inicializado) Esta partida no ha sido inicializada correctamente.
$ (Comando correcto) El juego se ha inicializado correctamente.
$
```

Caso T10 (suma inicial incorrecta)

```
Bienvenido al juego Risk. Pon 'ayuda' para ver los comandos disponibles.
$ (Comando correcto) El juego se ha inicializado correctamente.
$
Tienes 0 unidades para ubicar en este turno.
(Comando correcto) El jugador Ana ha terminado de reclamar y ubicar sus unidades.
$
Desde cual de tus territorios quieres atacar? (Ingresa 0 para no atacar / terminar fase de
ataque)
 1.1 - Alaska (tienes 34 tropas)
 3.1 - Gran Bretana (tienes 1 tropas) [No disponible: necesita mas de 1 tropa]
 0 - Terminar fase de ataque y pasar a fortificar
Ingresa el codigo del territorio de origen (o 0 para terminar): (Comando correcto) El jugador
Ana ha terminado de atacar.
$ ¿Quieres fortificar algun territorio? (s/n): (Comando correcto) El jugador Ana ha terminado
de fortificar su posición.
$
```

Caso T11 (refuerzos en 0)

Nota. En T08 a T11 las entradas se leen del archivo .in, así que la consola no muestra lo que se escribe; por eso las respuestas no aparecen después de cada pregunta.

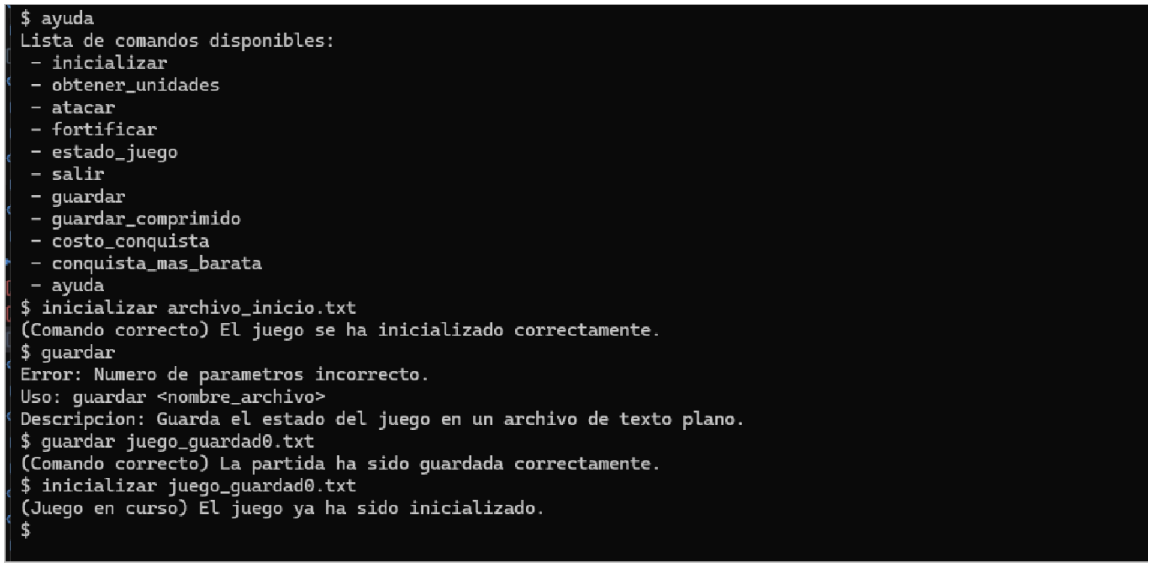
Apéndice C

Capturas de pantalla de la ejecución

Estas capturas las tomamos nosotros al ejecutar el programa: un turno completo y el ciclo de guardar y volver a cargar. Debajo de cada una indicamos qué casos respalda. Se tomaron en Windows con la versión anterior a las correcciones finales. Ninguna muestra el caso de tropas inválidas (lo cubren A-11, A-12 y T09 en el Apéndice B). Lo que aparece en las capturas C1 a C9 no cambió con las correcciones; la C10 ya no se reproduce con la versión final (ver su nota). Como los dados cambian en cada ejecución, los valores de las capturas de atacar no coinciden con los del Apéndice B; lo que interesa es que el comportamiento sea el mismo. En algunas capturas las tildes y los signos ¿ y ¡ se ven mal por la página de códigos de la consola (sección 7).

Figura C1

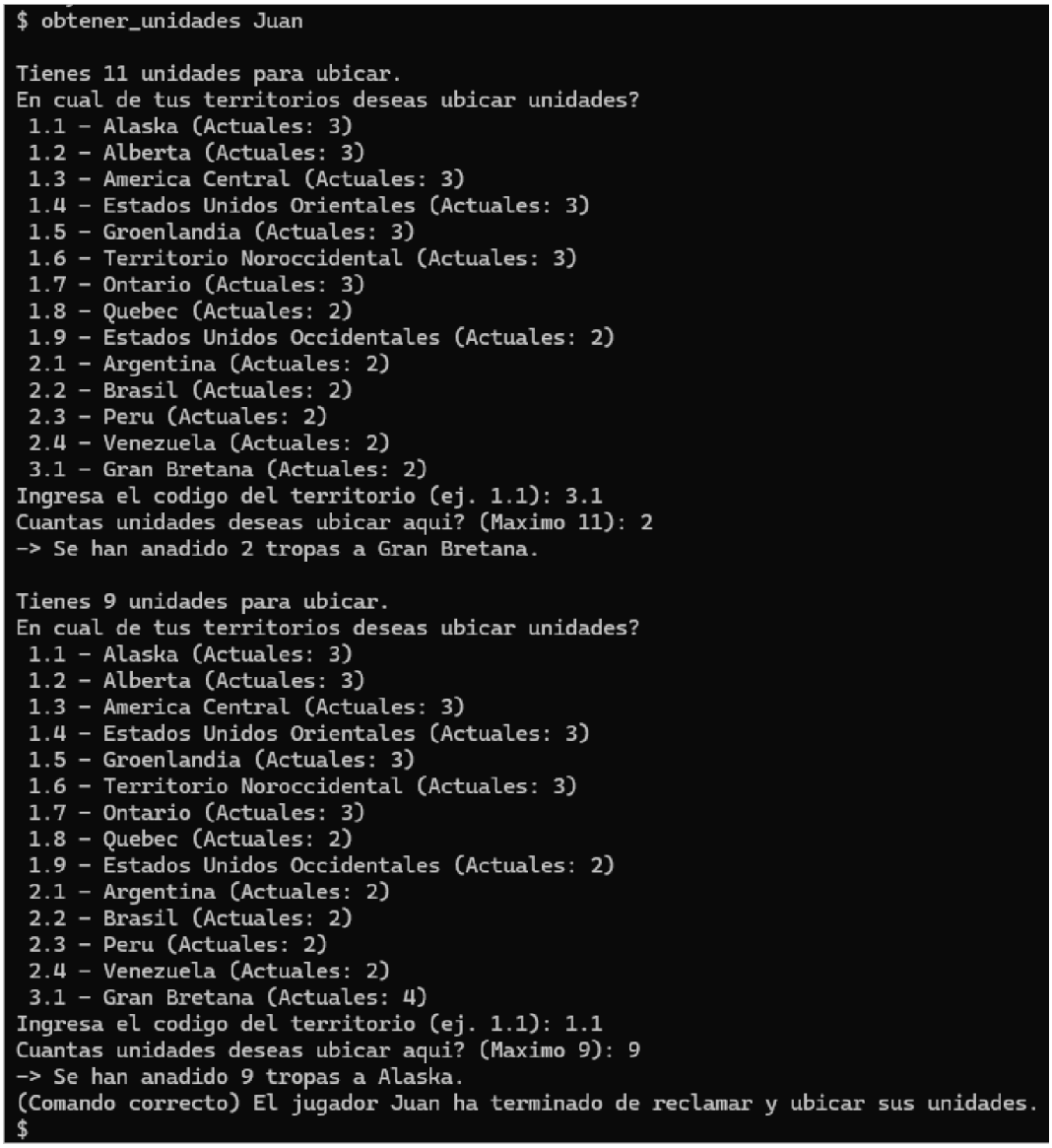
Comandos ayuda, inicializar, guardar sin parámetros, guardar juego_guardad0.txt e inicializar con el juego en curso



Nota. Respalda T05, G-02 (mismo error de parámetros) y G-08 (mismo mensaje de juego en curso).

Figura C2

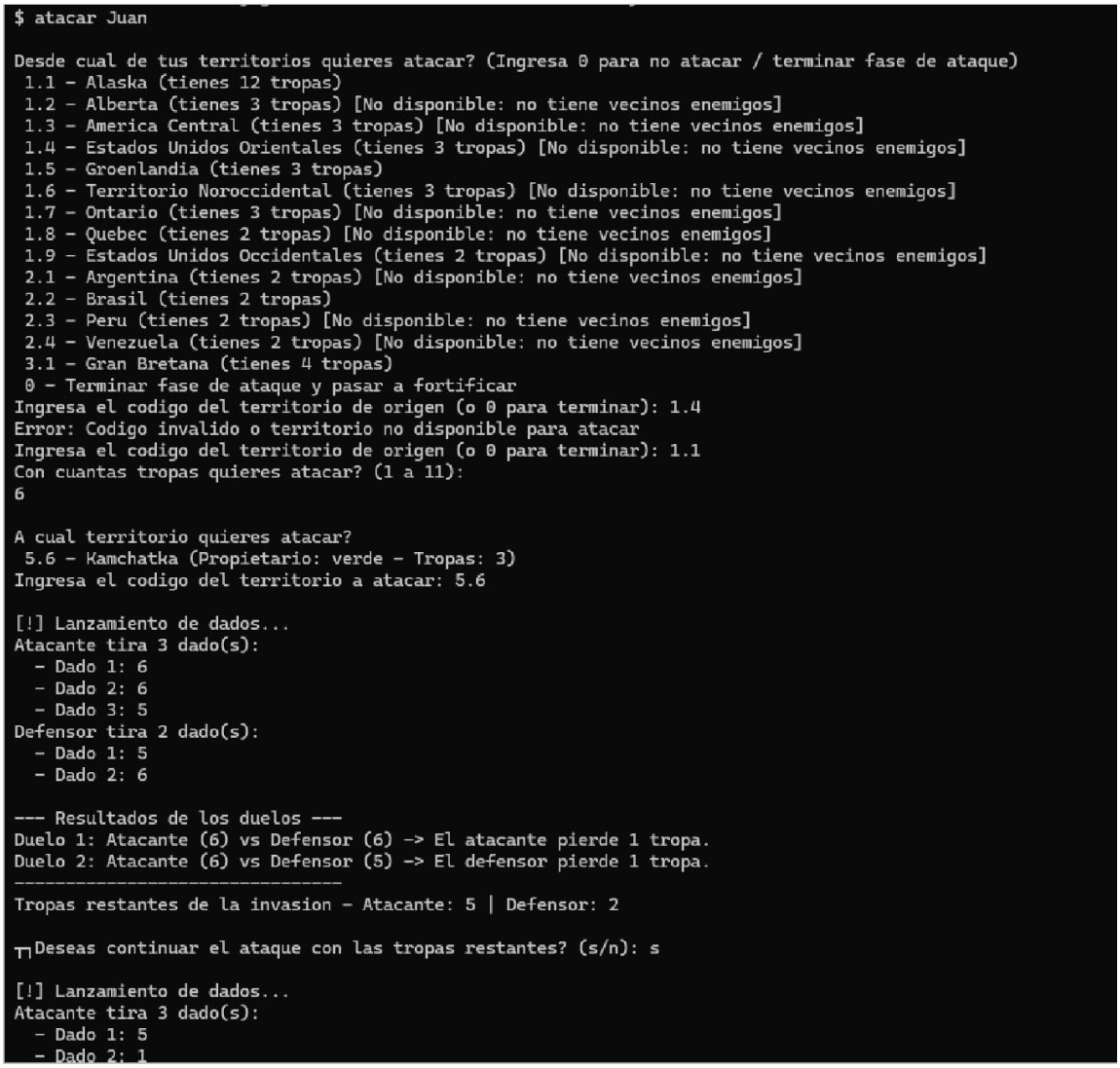
obtener_unidades Juan: 11 unidades ubicadas por código en dos rondas (2 en 3.1 y 9 en 1.1)



Nota. Precondición de los casos A-08 a A-17; respalda T07.

Figura C3

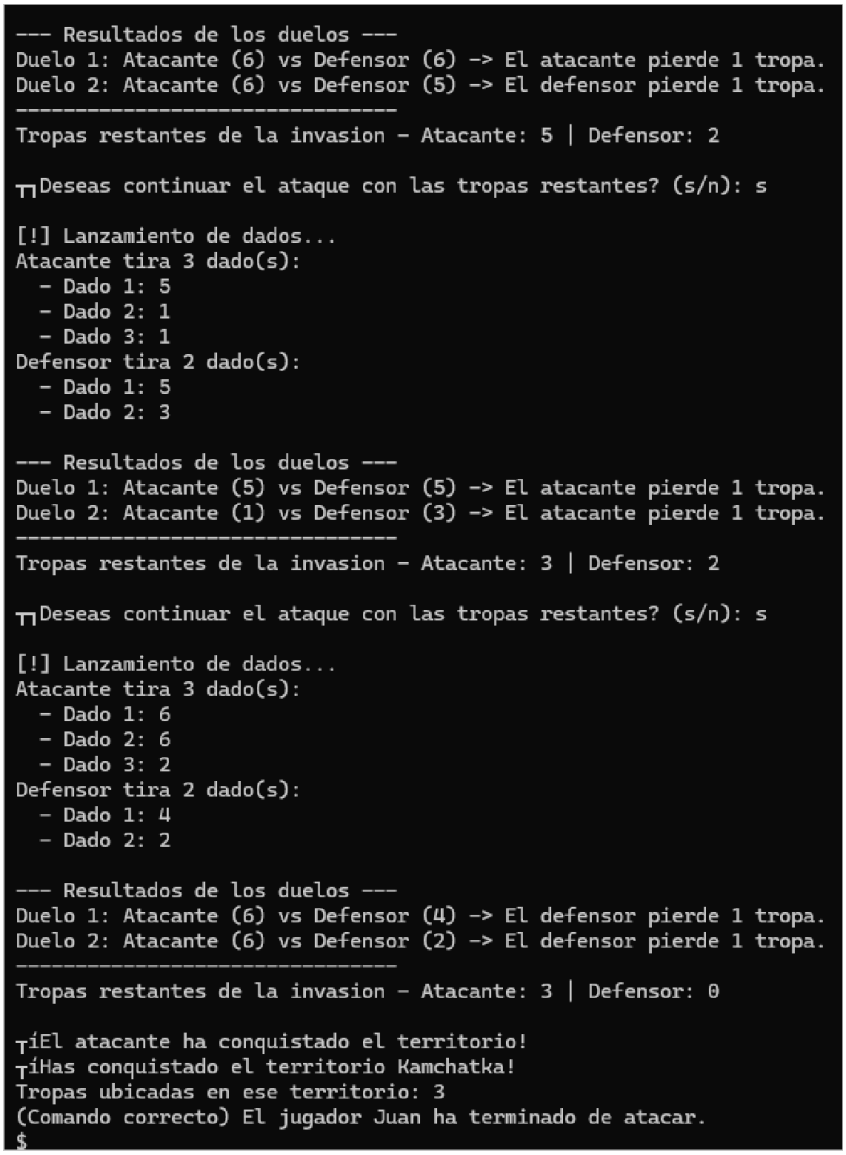
atacar Juan (primera parte): lista con códigos, rechazo del origen 1.4, origen 1.1, destino 5.6 y primer lanzamiento



Nota. Respalda A-08, A-10 y A-13.

Figura C4

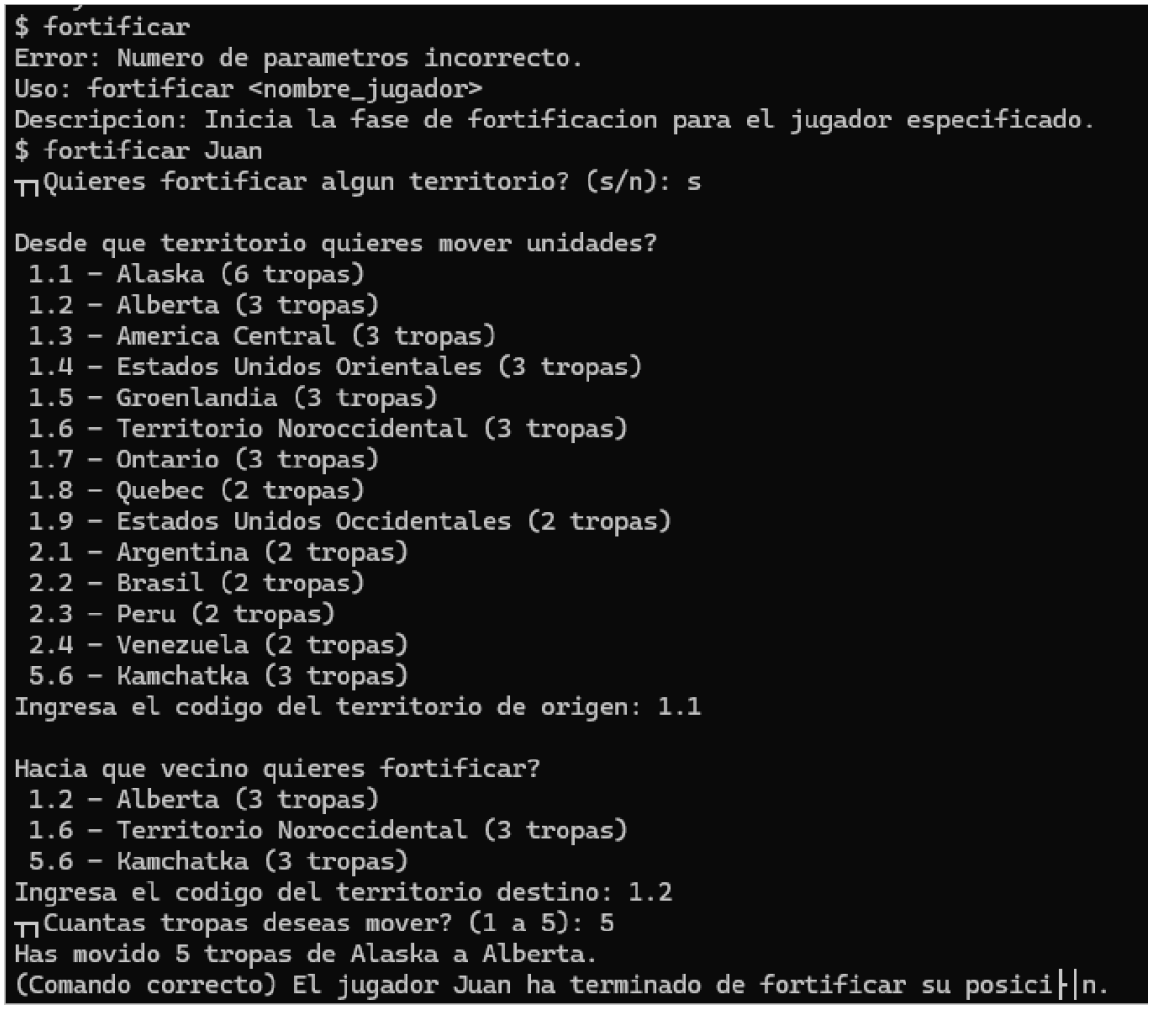
atacar Juan (segunda parte): lanzamientos hasta dejar al defensor en 0 y conquista de Kamchatka



Nota. Respalda A-14.

Figura C5

fortificar sin parámetros y fortificar Juan: 5 tropas movidas de Alaska (1.1) a Alberta (1.2)



Nota. Respalda A-17.

Figura C6

estado_juego después del turno: 5.6 pasa a rojo y el turno pasa a Pedro

```
$ estado_juego
--- ESTADO DEL JUEGO ---
Jugadores en partida (3):
 - Juan [Color: rojo]
 - Pedro [Color: azul]
 - Maria [Color: verde]

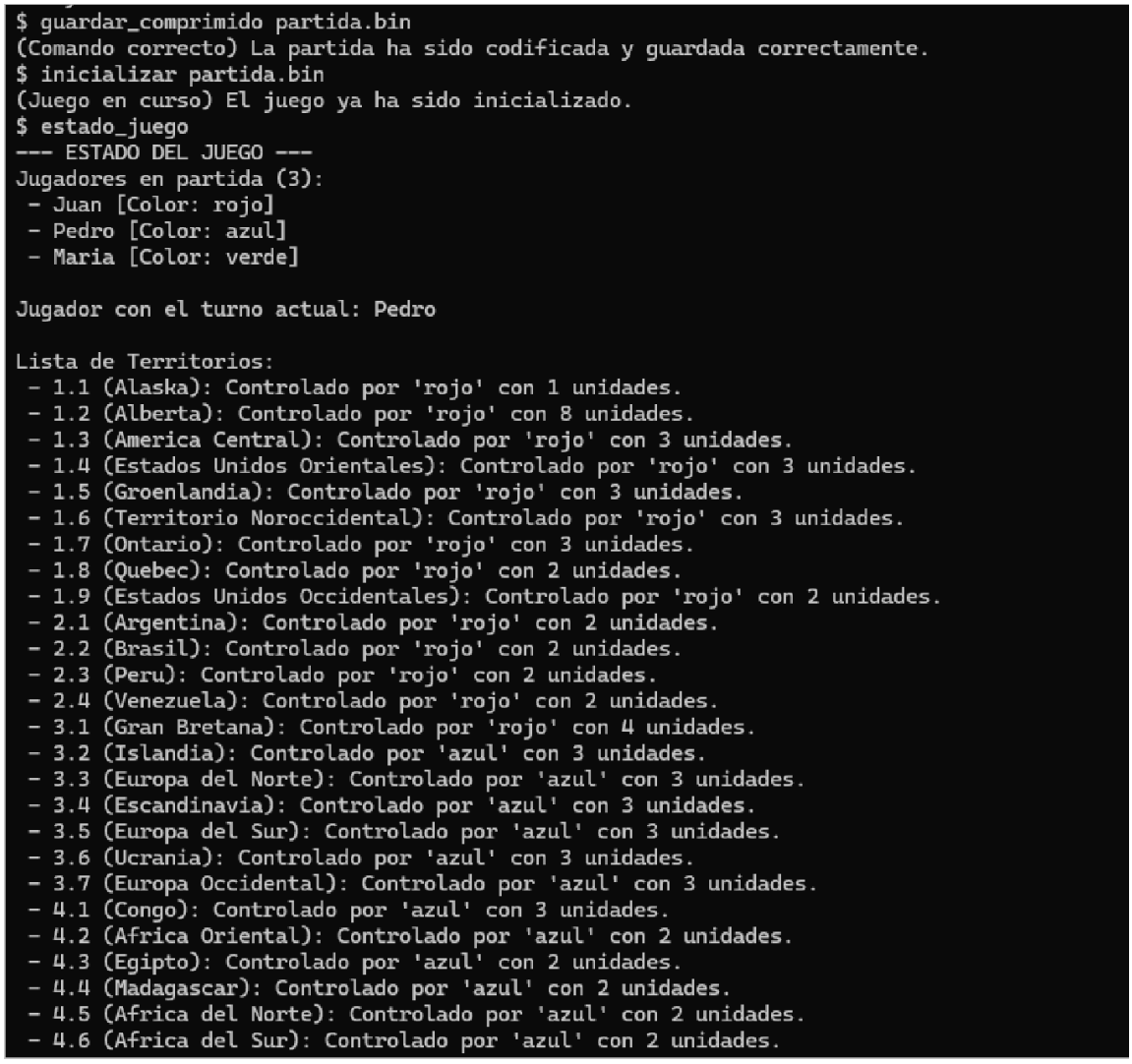
Jugador con el turno actual: Pedro

Lista de Territorios:
 - 1.1 (Alaska): Controlado por 'rojo' con 1 unidades.
 - 1.2 (Alberta): Controlado por 'rojo' con 8 unidades.
 - 1.3 (America Central): Controlado por 'rojo' con 3 unidades.
 - 1.4 (Estados Unidos Orientales): Controlado por 'rojo' con 3 unidades.
 - 1.5 (Groenlandia): Controlado por 'rojo' con 3 unidades.
 - 1.6 (Territorio Noroccidental): Controlado por 'rojo' con 3 unidades.
 - 1.7 (Ontario): Controlado por 'rojo' con 3 unidades.
 - 1.8 (Quebec): Controlado por 'rojo' con 2 unidades.
 - 1.9 (Estados Unidos Occidentales): Controlado por 'rojo' con 2 unidades.
 - 2.1 (Argentina): Controlado por 'rojo' con 2 unidades.
 - 2.2 (Brasil): Controlado por 'rojo' con 2 unidades.
 - 2.3 (Peru): Controlado por 'rojo' con 2 unidades.
 - 2.4 (Venezuela): Controlado por 'rojo' con 2 unidades.
 - 3.1 (Gran Bretana): Controlado por 'rojo' con 4 unidades.
 - 3.2 (Islandia): Controlado por 'azul' con 3 unidades.
 - 3.3 (Europa del Norte): Controlado por 'azul' con 3 unidades.
 - 3.4 (Escandinavia): Controlado por 'azul' con 3 unidades.
 - 3.5 (Europa del Sur): Controlado por 'azul' con 3 unidades.
 - 3.6 (Ucrania): Controlado por 'azul' con 3 unidades.
 - 3.7 (Europa Occidental): Controlado por 'azul' con 3 unidades.
 - 4.1 (Congo): Controlado por 'azul' con 3 unidades.
 - 4.2 (Africa Oriental): Controlado por 'azul' con 2 unidades.
 - 4.3 (Egipto): Controlado por 'azul' con 2 unidades.
 - 4.4 (Madagascar): Controlado por 'azul' con 2 unidades.
 - 4.5 (Africa del Norte): Controlado por 'azul' con 2 unidades.
 - 4.6 (Africa del Sur): Controlado por 'azul' con 2 unidades.
 - 5.1 (Afghanistan): Controlado por 'azul' con 2 unidades.
 - 5.2 (China): Controlado por 'azul' con 2 unidades.
 - 5.3 (India): Controlado por 'verde' con 3 unidades.
 - 5.4 (Irkutsk): Controlado por 'verde' con 3 unidades.
 - 5.5 (Japon): Controlado por 'verde' con 3 unidades.
 - 5.6 (Kamchatka): Controlado por 'rojo' con 3 unidades.
 - 5.7 (Medio Oriente): Controlado por 'verde' con 3 unidades.
 - 5.8 (Mongolia): Controlado por 'verde' con 3 unidades.
 - 5.9 (Siam): Controlado por 'verde' con 3 unidades.
 - 5.10 (Siberia): Controlado por 'verde' con 2 unidades.
 - 5.11 (Ural): Controlado por 'verde' con 2 unidades.
 - 5.12 (Yakutsk): Controlado por 'verde' con 2 unidades.
 - 6.1 (Australia Oriental): Controlado por 'verde' con 2 unidades.
 - 6.2 (Indonesia): Controlado por 'verde' con 2 unidades.
 - 6.3 (Nueva Guinea): Controlado por 'verde' con 2 unidades.
```

Nota. Respalda A-14 y A-17.

Figura C7

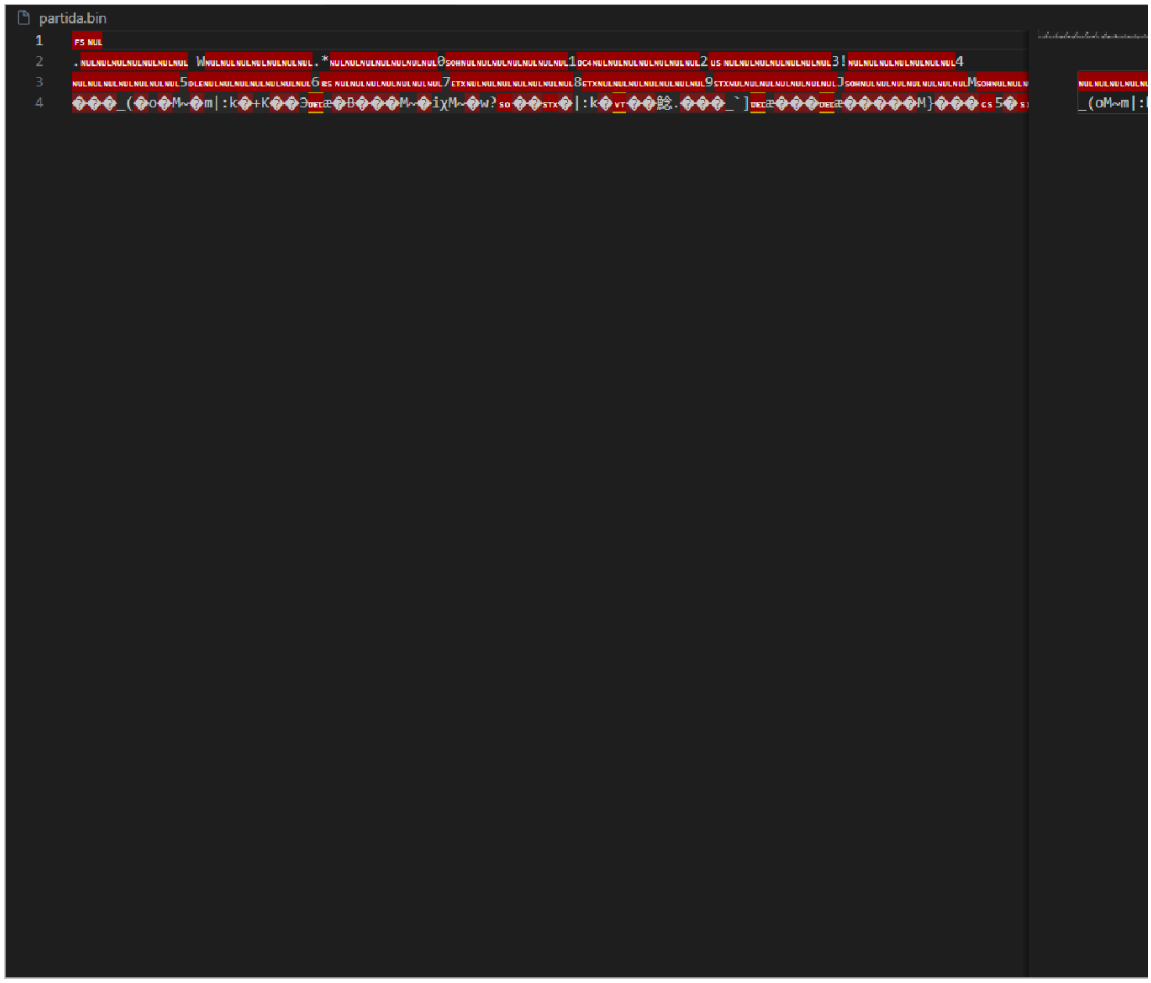
guardar_comprimido partida.bin, inicializar con el juego en curso y estado_juego



Nota. Respalda G-05 y G-08.

Figura C8

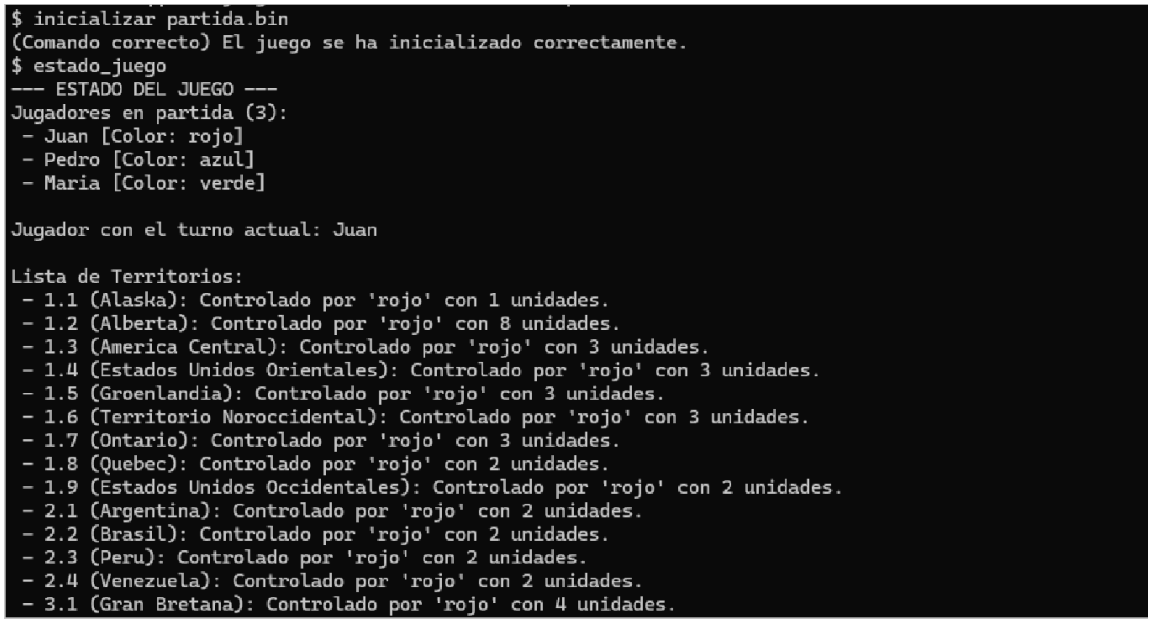
Contenido de partida.bin en un editor: la cabecera empieza con n = 0x1C 0x00 (28 caracteres distintos)



Nota. Respalda G-05; la estructura completa está en la Figura 13.

Figura C9

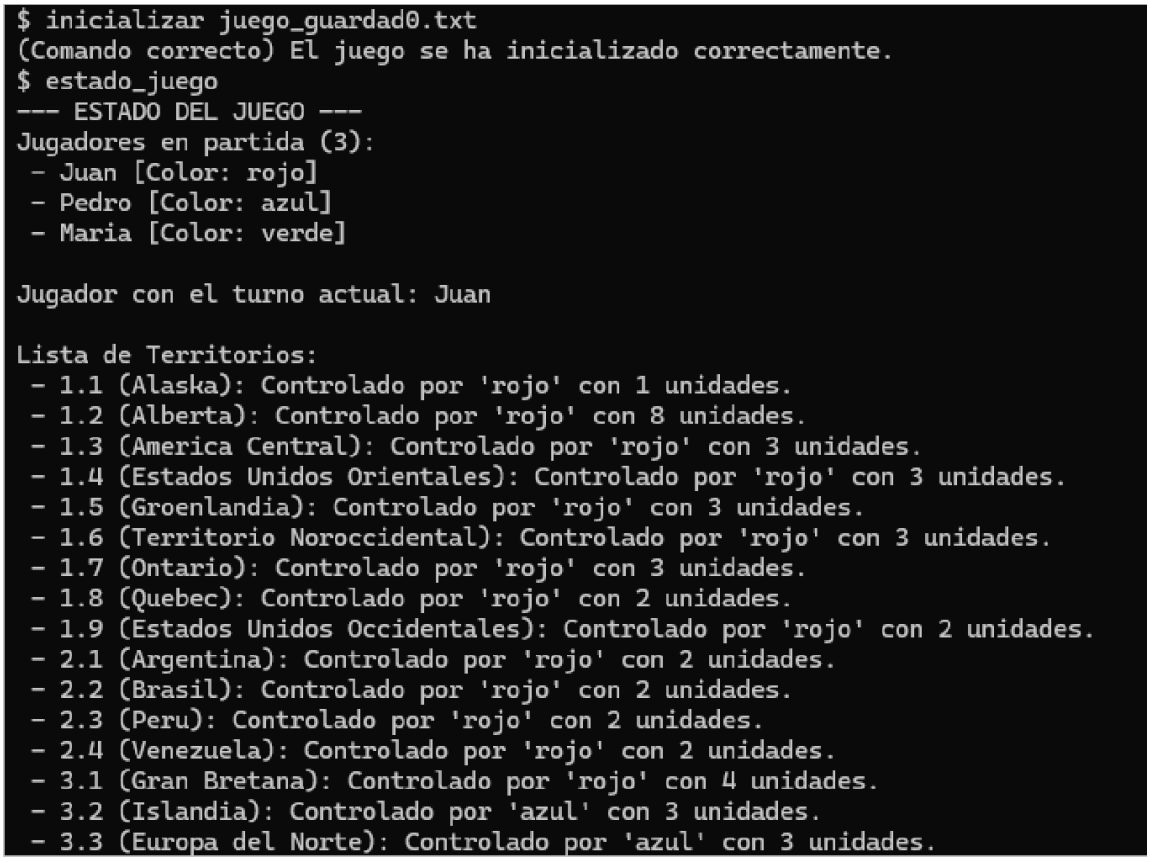
En una nueva ejecución, inicializar partida.bin reproduce la partida guardada



Nota. Respalda G-07 y G-09. El turno vuelve a Juan porque el formato no guarda el turno.

Figura C10

inicializar juego_guardad0.txt, generado por guardar, produce el mismo estado (versión anterior)



Nota. Tomada antes de validar la suma inicial. juego_guardad0.txt es una partida en curso (43, 35 y 32 unidades), así que la versión final la rechaza con (Archivo sin formato); el ciclo con partidas en curso se hace con .bin (G-07 y G-09). Ver sección 7.

Apéndice D

Archivo de inicio usado en las pruebas

El enunciado no permite incluir archivos .txt en el .zip, así que aquí está el contenido completo de archivo_inicio.txt (carpeta data/ del repositorio). Para probar el programa desde el .zip basta copiarlo en un archivo de texto con ese nombre. Cada jugador suma 35 unidades: Juan (rojo) en América del Norte, América del Sur y Gran Bretaña; Pedro (azul) en el resto de Europa, África y parte de Asia, y Maria (verde) en el resto de Asia y Australia.

```
3
Juan rojo
Pedro azul
Maria verde
- rojo 3
- rojo 3
- rojo 3
- rojo 3
- rojo 3
- rojo 3
- rojo 3
- rojo 2
- rojo 2
- rojo 2
- rojo 2
- rojo 2
- rojo 2
- rojo 2
- azul 3
- azul 3
- azul 3
- azul 3
- azul 3
- azul 3
- azul 3
```

archivo_inicio.txt (líneas 1 a 25)

```
azul 2
azul 2
azul 2
azul 2
azul 2
azul 2
azul 2
verde 3
verde 3
verde 3
verde 3
verde 3
verde 3
verde 3
verde 2
verde 2
verde 2
verde 2
verde 2
verde 2
verde 2
```

archivo_inicio.txt (líneas 26 a 46)